Deggendorf Institute of Technology
Faculty of Mechanical Engineering and Mechatronics

Faculty Applied Natural Sciences and Industrial Engineering

Mechatronics and Cyber-Physical Systems

Master's Thesis

Optimizing Acoustic Event Detection for Challenging Data and Label Availability

A Master's thesis submitted in partial fulfillment of the requirements for the academic degree of

Master of Science

Author name	**Omkar Akolkar**
Examiner	Prof. Dr.-Ing. Mathias Hartmann THD - Technologie Campus Hutthurm
Company Supervisor	M.Sc. Jan Baumann IAV GmbH
Date of submission	Deggendorf, 29.06.2026

Deggendorf Institute of Technology
Faculty Applied Natural Sciences and Industrial Engineering
Technology Campus Hutthurm
Dieter-Görlitz-Platz 1

Tel.: +49 (0) 991 / 3615 - 0
Fax: +49 (0) 991 / 3615 - 297
Email: info@th-deg.de
Web: www.th-deg.de

Abstract

Sound event detection (SED) systems require strongly-labelled audio recordings annotated with the onset and offset of every event, yet such labels are costly and slow to produce, whereas weakly-labelled and unlabelled audio are comparatively abundant. This thesis investigates whether semi-supervised learning (SSL) can exploit this cheap data to offset the scarcity of strong labels, and thereby approach the performance of a fully strongly-supervised system on synthetic SED data.

Because no public corpus matched the required sound classes and acoustic conditions, three synthetic mixture-generation pipelines were developed, producing single-event, non-overlapping multi-event, and overlapping multi-event soundscapes with exact temporal annotations. A Frequency Dynamic Convolutional Recurrent Neural Network was trained within a Mean Teacher framework across fourteen experiments that systematically varied dataset size, label partitioning, class count, and mixture type. The study was complemented by three reference systems on a matched overlapping configuration: a supervised strong-label baseline, a fine-tuned ATST-SED model initialised from an externally pre-trained audio transformer, and an event-to-background ratio (EBR) sweep that evaluated a fixed model across test sets of decreasing event prominence.

With respect to guiding question, the results were not matching the expectations. The in-domain semi-supervised model reached an event-based macro F1 of 0.358, roughly 63% of the supervised baseline (0.571) trained on the same data, and reallocating training data from strong labels to unlabelled clips degraded every metric. A source-disjoint evaluation protocol further revealed that earlier, higher scores were partly inflated by train-test source overlap, establishing a lower but more credible performance ceiling. The EBR sweep showed monotonic degradation as events grew quieter, with precise temporal localisation lost first and frame-level presence detection most resilient.

In contrast, transferring knowledge from external pre-training proved more effective than exploiting in-domain unlabelled data. On the identical configuration, ATST-SED raised event-based macro F1 to 0.492, recovering most of the gap to the supervised baseline about 86% of its event-based F1 while still using only 40% of the training pool as strong labels. Within the regimes studied, in-domain semi-supervised training did not compensate for limited strong supervision, whereas external pre-training did improve it. Together these findings delineate where the standard Mean Teacher recipe ceases to help on synthetic SED data and identify transfer from large-scale pre-training as the more promising route when strong labels are scarce.

Contents

Abstract i

List of Figures vi

List of Tables viii

List of Abbreviations ix

1 Introduction 1

1.1 Background and Motivation 1

1.2 Problem Statement 1

1.3 Thesis Outline 2

2 Groundwork and Fundamental Principles 4

2.1 Definition and Formalization of Sound Event Detection 4

2.1.1 Sound Events In Our Daily Environment 4

2.2 Challenges In Sound Event Detection 5

2.2.1 Main Challenges This Work Is Trying To Tackle 6

2.3 Sound Event Labels: Strong, Weak, and Unlabeled Data 6

2.4 Acoustic Feature Extraction: From Raw Audio to Log Mel Spectrograms for Sound Event Detection 11

2.5 Machine Learning for Sound Event Detection 15

2.5.1 From Traditional Methods to Deep Learning 15

2.5.2 Neural Network's for Sound Event Detection 16

2.5.3 Convolutional Recurrent Neural Network (CRNN) 18

2.6 Evaluation Metrics for Sound Event Detection 22

2.6.1 Segment-Based Evaluation 22

2.6.2 Event-Based Evaluation 25

2.6.3 Polyphonic Sound Detection Score (PSDS) 28

3 State of the Art in Sound Event Detection 31

3.1 Semi-Supervised Learning Paradigms 31

3.1.1 Consistency Regularization 31

3.1.2 Pseudo-Labeling . . . 32
3.1.3 The Mean Teacher Framework . . . 33
3.2 Semi-Supervised Learning Applied to SED . . . 34
3.2.1 DCASE Task 4 Evolution . . . 35
3.2.2 Mean Teacher for SED . . . 37
3.2.3 Data Augmentation Strategies . . . 38
3.3 Architectural Advances for Sound Event Detection . . . 40
3.3.1 Frequency Dynamic Convolution for Sound Event Detection . . . 41
3.3.2 Attention and Transformer-based Approaches . . . 42

4 Methodology **44**
4.1 Public Data Sets . . . 44
4.1.1 FSD50K Dataset for Isolated Sound Events . . . 45
4.1.2 TUT Sound Events 2017, Development dataset for Background Noise 46
4.2 Data Generation . . . 46
4.2.1 Synthetic Mixture Generation With Single Isolated Event . . . 47
4.2.2 Synthetic Mixture Generation with Multiple Non-Overlapping Events 53
4.2.3 Synthetic Mixture Generation with Multiple Overlapping Events . 56
4.3 Supervised Model . . . 58
4.4 Model and Training Framework for Semi-supervised model . . . 59
4.5 ATST-SED: Fine-tuning a Pre-trained Audio Transformer for SED . . . 60

5 Experimental Setup **63**
5.1 Common Configuration . . . 63
5.2 Hardware . . . 63
5.3 Data Setups . . . 64
5.4 Setup for supervised learning . . . 67
5.5 Setup for ATST-SED . . . 67
5.6 Event-to-Background Ratio (EBR) Test Sweep . . . 68

6 Results & Discussion **69**
6.1 Results on Semi-Supervised method FDY-SED model . . . 69
6.1.1 Group A: Three-Class Single-Event . . . 69

6.1.2 Group B: Five-Class Single-Event 72
6.1.3 Group C: Five-Class Non-Overlapping Multi-Event 74
6.1.4 Group D: Five-Class Overlapping Multi-Event 76
6.1.5 Group E: Overlapping Multi-Event, Increased Unlabelled Proportion 77
6.2 Supervised method with strong labeled Overlapping events 79
6.3 ATST-SED Results 80
6.4 Cross-Model Comparison 80
6.5 Event-to-Background Ratio (EBR) Test Sweep Results 82
7 Conclusion and Future Work **84**
7.1 Conclusion 84
7.2 Future Work 85
Bibliography **87**

List of Figures

2.1 Sound event detection: Acoustic labels describing the source of the sound and visually showcasing the source. Source: [1] 4

2.2 Comparison of weak labels (tagging, left) and strong labels (polyphonic annotation, right) for sound events. Source: [2] 11

2.3 Overview of the audio preprocessing pipeline for generating log-Mel spectrograms from raw waveform input. 13

2.4 Graphical representation of conversion from Raw audio signal to Mel-spectrogram [3] 14

2.5 Overview of the CRNN method. (1): Multiple convolutional layers with max pooling in frequency axis, (2): The outputs of the last convolutional layer stacked over frequency axis and fed to multiple stacked recurrent layers, (3): feedforward layer as output layer and (4): binarization of event activity probabilities [4] 19

2.6 Segment-based evaluation example illustrating the comparison between ground truth and system predictions across temporal segments, highlighting correct detections (TP), missed detections (FN), false alarms (FP), and correct rejections (TN). 24

2.7 Tolerance windows (collars) used in event-based evaluation for onset and offset matching, where predictions within the onset and offset tolerance are considered correct detections (if 50% or more overlap). 27

4.1 General Flowchart for Data set generation pipeline 47

4.2 Buffers and permissible space for event placements. 50

4.3 Single isolated event mixture representation 51

4.4 Example Audio 1 mix000004.wav,0.2771,4.9716,Rattle 52

4.5 Example Audio 2 mix000300.wav,3.7916,9.5524,Hiss 52

4.6 Multiple event non overlapping mixture representation 54

4.7 Multiple events overlapping mixture representation 57

6.1 Group A: teacher $PSDS_1$ and event-based macro F1 versus dataset size at fixed 45/5/15/30/5 partitioning (E1–E3); both peak at 6,440 mixtures. Error bars show ± one standard deviation. 70

6.2 Group A: effect of partitioning at fixed 8,050 mixtures. Raising the unlabelled share from 35% (E5) to 40% (E4) lowers every teacher metric. Error bars show ± one standard deviation. 72

6.3 Effect of class count at matched 45/5/15/30/5 partitioning and comparable dataset size: three-class E1 (4,832 clips) versus five-class E6 (4,000 clips). Every teacher metric drops with five classes, most sharply on the event-based metrics. Error bars show ± one standard deviation. 73

6.4 Group C: monotonic improvement in teacher $PSDS_1$ and event-based macro F1 with dataset size at fixed 30/5/15/45/5 partitioning (E8–E10). Error bars show ± one standard deviation. 75

6.5 Groups D vs E: raising the unlabelled training share from 45% to 55% lowers every teacher metric at both 12,000 and 18,000 mixtures. Values from Table 6.15. 79

6.6 Event-based macro F1 on the identical 18,000-mixture five-class overlapping dataset (E12 configuration). ATST-SED narrows the gap between the FDY-CRNN semi-supervised system and the fully-supervised baseline. . . 81

6.7 Detection performance of the fixed E12 model as a function of Event-to-Background Ratio. All metrics degrade monotonically as events become quieter relative to the background; PSDS1 collapses most steeply, while segment-based F1 is the most resilient. 82

List of Tables

4.1 Overview of public datasets and reasons for rejection 44

4.2 Per-class sample counts and durations of the isolated event set. 45

4.3 Comparison of the three synthetic mixture generation pipelines. 58

5.1 Group A: three-class single-event experiments. The partitioning column gives strong/val/weak/unlabelled/test percentages of the total. 65

5.2 Group B: five-class single-event experiments. 65

5.3 Group C: five-class multi-event non-overlapping experiments. 65

5.4 Group D: five-class multi-event overlapping experiments. Validation column gives size as a fraction of the strongly-labelled training subset. 66

5.5 Group E: five-class multi-event overlapping experiments with increased unlabelled training proportion. Partitioning is expressed relative to the training pool (the 90% of clips remaining after the held-out test set is removed). 66

6.1 E1 test results on three-class single-event mixtures (*Rattle*, *Hiss*, *Bell*) with 4,832 clips and 45/5/15/30/5 partitioning. Mean ± standard deviation over $n = 7$ runs. 69

6.2 E2 test results on three-class single-event mixtures with 6,440 clips. Mean ± standard deviation over $n = 6$ runs. 70

6.3 E3 test results on three-class single-event mixtures with 10,273 clips. Mean ± standard deviation over $n = 7$ runs. 70

6.4 E4 test results on three-class single-event mixtures with 8,050 clips and partitioning 35/5/15/40/5. Mean ± standard deviation over $n = 4$ runs. . . 71

6.5 E5 test results on three-class single-event mixtures with 8,050 clips and partitioning 40/5/15/35/5. Mean ± standard deviation over $n = 4$ runs. . . 71

6.6 E6 test results on five-class single-event mixtures with 4,000 clips and partitioning 45/5/15/30/5. Mean ± standard deviation over $n = 7$ runs. . . 72

6.7 E7 test results on five-class single-event mixtures with 6,000 clips and partitioning 30/5/15/45/5. Mean ± standard deviation over $n = 7$ runs. . . 73

6.8 E8 test results on five-class non-overlapping multi-event mixtures with 2,000 clips. Mean ± standard deviation over $n = 4$ runs. 74

6.9 E9 test results on five-class non-overlapping multi-event mixtures with 4,000 clips. Mean ± standard deviation over $n = 4$ runs. 74

6.10 E10 test results on five-class non-overlapping multi-event mixtures with 8,000 clips. Mean ± standard deviation over $n = 2$ runs. 75

6.11 E11 test results on five-class overlapping multi-event mixtures with 12,000 clips. Mean ± standard deviation over $n = 6$ runs. 76

6.12 E12 test results on five-class overlapping multi-event mixtures with 18,000 clips. Mean ± standard deviation over $n = 6$ runs. 76

6.13 E13 test results on five-class overlapping multi-event mixtures with 18,000 clips and a 55% unlabelled training share. Mean ± standard deviation over $n = 7$ runs. 77

6.14 E14 test results on five-class overlapping multi-event mixtures with 12,000 clips and a 55% unlabelled training share. Mean ± standard deviation over $n = 7$ runs. 78

6.15 Effect of raising the unlabelled training share from 45% (Group D) to 55% (Group E) on teacher metrics at matched total dataset size. 78

6.16 Supervised strong-label baseline: overall event-based F1 (%) by split, at the F1-optimal validation threshold. Single run on the strong labelled 40% part of 18,000-mixture five-class overlapping dataset. 79

6.17 ATST-SED test results on the 18,000-mixture five-class overlapping dataset (E12 configuration). 80

6.18 Comparison of the three models on the identical 18,000-mixture five-class overlapping dataset (E12 configuration). Best value in each column in bold. PSDS1 and segment-based F1 were not available for the supervised baseline. 81

6.19 Performance of the fixed E12 semi-supervised model evaluated on test sets of decreasing Event-to-Background Ratio. 82

List of Abbreviations

AED Acoustic Event Detection

ATST Audio Teacher-Student Transformer

BCE Binary Cross-Entropy

BEATs Bidirectional Encoder representation from Audio Transformers

CNN Convolutional Neural Network

CRNN Convolutional Recurrent Neural Network

dBFS Decibels relative to Full Scale

DCASE Detection and Classification of Acoustic Scenes and Events

DESED Domestic Environment Sound Event Detection

DFT Discrete Fourier Transform

eFPR effective False Positive Rate

EMA Exponential Moving Average

FDY-Conv Frequency Dynamic Convolution

FDY-CRNN Frequency Dynamic Convolutional Recurrent Neural Network

FFT Fast Fourier Transform

FN False Negative

FNN Feedforward Neural Network

FP False Positive

FP32 32-bit (single-precision) Floating Point

GMM Gaussian Mixture Model

GRU Gated Recurrent Unit

HMM Hidden Markov Model

KL Kullback–Leibler (divergence)

LSTM Long Short-Term Memory

MFCC Mel-Frequency Cepstral Coefficient

MLP Multilayer Perceptron

MSE Mean Squared Error

NMF Non-negative Matrix Factorization

PCM Pulse Code Modulation

PSD-ROC Polyphonic Sound Detection

PSDS Polyphonic Sound Detection Score

ReLU Rectified Linear Unit

RMS Root Mean Square

RNN Recurrent Neural Network

ROC Receiver Operating Characteristic

SED Sound Event Detection

SNR Signal-to-Noise Ratio

SSL Semi-Supervised Learning

STFT Short-Time Fourier Transform

SVM Support Vector Machine

TDP Thermal Design Power

TN True Negative

TP True Positive

VAT Virtual Adversarial Training

1 Introduction

1.1 Background and Motivation

Sound carries information about the physical events that produce it. A rattle from a loose trim panel, the squeak of a worn bearing, a horn from an adjacent vehicle, or breaking glass each correspond to a distinct physical source, and humans interpret these acoustic events effortlessly as part of everyday listening. Replicating this ability computationally detecting which sound events occur in an audio signal and when, is the task of Sound Event Detection (SED).

The motivation for automated SED is its applicability wherever a system must react to acoustic events without human supervision. Sound often reveals the state of a process or environment before any visual cue is available: rattles, squeaks, hisses, and similar transient sounds are characteristic indicators of mechanical wear, faults, or changes in operating condition, and comparable cues arise across domestic, industrial, surveillance, and monitoring settings. A system able to detect and temporally localise such events from audio would complement vision-based sensing, which cannot observe sound, and would support applications ranging from condition and quality monitoring to acoustic scene analysis and event-triggered alerting. Unlike speech recognition or music analysis, which exploit strong domain structure such as a fixed phoneme inventory or harmonic regularity, environmental SED must contend with an open-ended set of sources, wide variation in event duration and spectral character, and overlapping events in realistic acoustic scenes [1].

The shift from hand-crafted features with shallow classifiers to deep neural networks made SED practical at this level of difficulty, enabling models to learn discriminative spectro-temporal representations directly from data [1]. This capability, however, comes with a corresponding demand: deep models require large quantities of suitably annotated audio to reach their potential, and the cost of producing that annotation is precisely where SED becomes constrained.

1.2 Problem Statement

The supervision a SED model can learn from is not uniform in cost. Strong labels, which specify the onset and offset of every event, provide the temporal information needed for localisation but are expensive to obtain: annotating event boundaries is slow, requires trained annotators, and suffers from inter-annotator disagreement on events with gradual onsets or offsets. Weak labels, which record only the presence of event classes within a clip,

and unlabelled audio are by comparison cheap and abundant [1]. The central difficulty addressed in this work is this asymmetry: the most useful form of supervision for temporal localisation is the scarcest, while less informative data is plentiful.

This difficulty is compounded at the level of available datasets. No public corpus was found that combined a sufficient quantity of strongly-labelled audio with the sound classes and acoustic conditions relevant to this work, which necessitates generating labelled data synthetically in order to obtain precise temporal annotations at scale. The question that follows is whether the abundant weak and unlabelled data can be exploited to offset the limited strong supervision, that is, whether semi-supervised learning can approach the performance of a fully strongly-supervised system on synthetically generated SED data [5].

Accordingly, this thesis investigates semi-supervised SED on synthetic data under controlled variation of dataset size, label partitioning, and acoustic complexity. The formal definition of the task, the strong/weak/unlabelled label taxonomy, and the specific challenges targeted are developed in Chapter 2; the concrete modelling and training approach is deferred to the methodology Chapter 4.

1.3 Thesis Outline

The remainder of this thesis is organised as follows.

Chapter 2 - **Groundwork and Fundamental Principles** establishes the groundwork. It formally defines sound event detection, introduces the strong, weak, and unlabelled label categories that lie at the heart of the data-availability problem, and describes the log-mel feature extraction pipeline, the convolutional recurrent architectures used for SED, and the evaluation metrics (segment-based F1, event-based F1, and PSDS) on which all later results are reported.

Chapter 3 - **State of the art in sound event detection** reviews the state of the art. It covers the semi-supervised learning paradigms relevant to this work — consistency regularisation, pseudo-labelling, and the Mean Teacher framework — and traces how they have been applied to SED through the evolution of the DCASE Task 4 challenge, before surveying the architectural advances (frequency dynamic convolution and transformer-based pre-trained models) that the methodology builds upon.

Chapter 4 - **Methodology** presents the methodology. It motivates the use of synthetically generated data, details the three mixture-generation pipelines developed for this work (single-event, non-overlapping multi-event, and overlapping multi-event), and specifies the models employed: the FDY-CRNN trained within a Mean Teacher framework, the supervised strong-label baseline, and the ATST-SED model that fine-tunes a pre-trained audio transformer for SED.

Chapter 5 - **Experimental Setup** defines the experimental protocol. It fixes the shared training configuration and hardware, and lays out the experiment groups that systematically vary dataset size, label partitioning, class count, and mixture type, together with the supervised-baseline and event-to-background ratio (EBR) setups.

Chapter 6 - **Results and Discussion** reports and discusses the results group by group, analyses the effects of the variables above, compares the semi-supervised, supervised, and ATST-SED models on identical data, and characterises the robustness of a trained model to event prominence through the EBR sweep.

Chapter 7 - **Conclusion and Future Work** concludes the thesis. It summarises the principal findings, returns to the research question posed above, and outlines directions for future work.

2 Groundwork and Fundamental Principles

2.1 Definition and Formalization of Sound Event Detection

A Sound Event refers to a specific sound produced by a distinct physical source, such as siren or a car passing by, typically characterised by a well-defined, brief duration[2].

2.1.1 Sound Events In Our Daily Environment

Sound events or acoustic events are as described above, are everywhere in our environments. Think about sitting in a car, while driving we use our senses like eyes, ears and touch or feel. While computer vision is implemented for the object detection but, the systems should also be able to detect audio signals like glass breaking, a car honking behind, squeak and rattle sounds of mechanical wear and tear in the car. Although the process seems natural enough for us but it is not so easy for machines to understand and differentiate between these audio signals. People learned to connect the sounds to their origins through centuries of evolutionary developments and learning.

Multiple sounds from different sources in the environment make up a complex mixture of audio. The process of separating multiple sound sources and concentrating on a particular sound source is highly selective and complex in nature. Just like concentrating on one's conversation with noise in a party, it is called the "cocktail party effect."[1] In order for us to perceive sounds or a group of sounds as a sound event, perception integrates the time-frequency information from the acoustic signal to an object producing it. This determines whether a complex series of sound events, such as "rattle" or "alarm," are perceived as one sound event occurrence.



Figure 2.1: Sound event detection: Acoustic labels describing the source of the sound and visually showcasing the source. Source: [1]

Automated sound event detection is a goal that attempts to classify from an algorithmic perspective, the capability possessed by humans of interpreting an acoustic scene. Automated sound event detection seeks to detect not only the nature but also the exact

time intervals of the sound events in an audio signal. The task would be detecting the time periods when specific sound events occur in the signal. An example demonstrating the temporal localization of sound events in an audio signal is illustrated in Figure 2.1 Where we can see a a urban street with pedestrians and vehicles, and continuous sound of raining.

The set of sound classes that a detection system can detect will depend upon the intended use of the technology. For the case of cars SEDs, there is a tendency to concentrate on mechanical and urban sounds such as sirens, vehicles, and footstep sounds.

These sounds are described as non-speech and non-musical sounds, thereby differentiating the study of environmental sound from other well-researched fields of sound study, including speech recognition and music information retrieval.The goals of the SED process differ from conventional methods of speech and music analysis in several ways. This discrepancy emerges from the different perceptual tendencies of human hearing in each of the fields. For instance, in the field of music listening, human perception tends to be oriented toward aspects such as melody, harmony, and rhythm. On the other hand, speech recognition focuses on linguistic information such as meaning and emotion.

The everyday perception of sound in the natural environment is primarily directed at the task of detecting and understanding the source of sounds. This process of auditory perception relies more on perceiving what is going on in the environment than the sound's characteristics. Thus, sound event detection tries to emulate human hearing in that respect and focuses on detecting sound events in an audio signal. [1]

2.2 Challenges In Sound Event Detection

Creating automatic systems for sound event detection is often by multiple challenges, some related to the nature of the sounds to be detected and how they appear in natural environments, others related to data collection and annotation procedures. These determine the challenges that must be overcome by the machine learning techniques in the learning process.

Sound events have very diverse acoustic characteristics. Some sounds are very short, transient-like, others may have longer duration and be harmonic, while events such as "footsteps" or "bird singing" may arise from temporal integration of a sequence of elementary components.

In typical applications of sound event detection, the target sound events are far away from the microphone, so the events are significantly affected by the acoustic transfer function. Also because of the distance to the microphone, the sound pressure level of the target sound event when received by the microphone can be often lower than that of other

sounds occurring in the environment, adding to the difficulty of detection.

Natural environments are **polyphonic**, meaning that multiple sounds may be active at the same time. Polyphony exists also in music, although often in a constrained way where overlapping sound events exhibit harmonic relations with each other, so that fundamental frequencies form small integer ratios. For sound events, there are no predefined rules on how sounds can co-occur, and the way to model co-occurrence is through statistics from the data itself, such as direct counts, and degree of overlap of different classes in the data.

The number of possible sound event classes is unlimited, since any object or being may produce a sound as a naturally occurring event. This is entirely different from other audio classification domains such as speech recognition, where a fixed set of classes (phonemes) is possible in a language, and language models can provide support to acoustic models in the recognition task.[1]

2.2.1 Main Challenges This Work Is Trying To Tackle

In the case of the sound event detection (SED) problem considered in this work, a primary challenge is the scarcity of strongly labeled data. Strong labels require precise temporal annotations indicating the onset and offset of each sound event. Acquiring such annotations at scale is difficult, as it involves significant manual effort. The annotation process is time-consuming and often requires human annotators to listen to large volumes of audio data for extended periods, which can lead to fatigue and reduced consistency.

Also, collecting and annotating real-world audio data introduces additional challenges. Variability in human perception can lead to disagreements among annotators regarding the exact timing or presence of sound events. Ensuring consistency and agreement across annotators is therefore non-trivial and adds to the complexity of building reliable strongly labeled datasets.

Weakly labeled data is comparatively easier to obtain, as it only requires clip-level annotations indicating the presence or absence of sound events without specifying their temporal boundaries. However, systems solely on weak labels often exhibit inferior performance in SED tasks, since they lack explicit temporal information necessary for accurate event localization.

An in depth explanation, about strong and weak labels can be found in chapters Section 2.3

2.3 Sound Event Labels: Strong, Weak, and Unlabeled Data

Sound event labels serve as descriptive annotations that capture how humans perceive and identify individual sounds within acoustic environments. These labels are fundamentally

rooted in the concept of everyday listening, where sounds are recognized and categorized based on their physical source or causal origin such as "dog barking," "car horn," or "footsteps" [2]. This approach contrasts with musical listening, which emphasizes the analysis of acoustic properties including pitch, timbre, rhythm, and harmonic structure, independent of the sound's origin.

The labeling of sound events enables the systematic description and analysis of acoustic scenes by identifying their constituent components. Within a given acoustic environment, such as a home, office, or street, multiple sound sources may be active simultaneously. Sound event labels provide a structured means of documenting these individual occurrences for instance, "people talking," "children shouting," "dishes clattering," or "traffic noise" thereby enabling both human understanding and machine learning of complex auditory scenes [2, 1].

The granularity and precision of sound event annotations can vary significantly based on application requirements, annotation resources, and practical constraints. This section examines three principal categories of labeled and unlabeled data used in sound event detection: weak labels, strong labels, and unlabeled data.

Weak Labels (Clip-Level Annotations)

Weak labels, also referred to as clip-level labels or tags, represent the most basic form of sound event annotation. A weak label indicates the presence or absence of specific sound event classes within an entire audio recording without providing any information about when these events occur, how long they last, or how many times they appear [5, 6].

Formally, for an audio clip $\mathbf{x}$ of duration T, a weak label annotation can be represented as a binary vector $\mathbf{y} \in \{0,1\}^C$, where C is the number of event classes in the taxonomy, and $y_c = 1$ indicates that event class c is present somewhere within the clip, while $y_c = 0$ indicates its absence. Alternatively, weak labels may be represented as a set of tags $\mathcal{L} = \{l_1, l_2, \ldots, l_k\}$, where each l_i denotes an event class present in the recording.

The annotation format for weak labels typically consists of two fields:

- **Filename**: Unique identifier of the audio file
- **Label**: Event class label(s) present in the clip

A typical weak label annotation file may be structured as follows:

```
filename,label
audio_001.wav,Speech
audio_001.wav,Dog
audio_001.wav,Dishes
audio_002.wav,Alarm_bell_ringing
audio_002.wav,Speech
audio_003.wav,Cat
```

In this example, `audio_001.wav` contains three types of sound events (Speech, Dog, and Dishes), but the annotation provides no information about the temporal arrangement, duration, or number of occurrences of these events.

Advantages of Weak Labels Weak labels offer several significant advantages that make them particularly attractive for large-scale dataset construction:

- Low annotation cost: Annotators need only listen to an entire clip and identify which sound events are present, without the time-consuming task of marking precise temporal boundaries
- Faster annotation speed: Studies have shown that weak labeling can be 5-10 times faster than strong labeling for the same amount of audio [1]
- Scalability: The reduced annotation effort enables the creation of much larger training datasets, which can improve model generalization
- Lower expertise requirement: Weak labeling requires less training and expertise compared to precise temporal annotation

Limitations of Weak Labels Despite their practical advantages, weak labels have inherent limitations:

- No temporal information: The absence of timing information makes it impossible to train models directly for temporal localization tasks
- Ambiguous supervision: During training, the model must infer which portions of the audio contain the labeled events, introducing additional learning complexity
- Limited evaluation capability: Systems trained on weak labels cannot be directly evaluated on their temporal localization accuracy

Weak labels are particularly prevalent in datasets derived from web sources, collaborative tagging platforms, and crowdsourcing efforts, where obtaining detailed temporal annotations would be prohibitively expensive [5].

Strong Labels (Event-Level Annotations)

Strong labels, also known as event-level labels or temporal annotations, provide precise temporal information about sound events by specifying their exact onset and offset times. Strong labels enable not only the identification of which sound events are present but also their temporal localization within the audio stream [2, 1].

Formally, a strong label annotation for an audio clip can be represented as a set of tuples:

$$\mathcal{A} = \{(t_{\text{on},i}, t_{\text{off},i}, c_i) \mid i = 1, 2, \ldots, N\} \tag{2.1}$$

where $t_{\text{on},i}$ denotes the onset time (in seconds) of the i-th event occurrence, $t_{\text{off},i}$ denotes its offset time, c_i denotes its event class label, and N is the total number of event occurrences in the clip. Each tuple represents a single event instance, and multiple instances of the same or different event classes may be present.

The standard annotation format for strong labels consists of four essential fields:

- **Filename**: Unique identifier of the audio file
- **Onset**: Start time of the event in seconds
- **Offset**: End time of the event in seconds
- **Label**: Event class label

A typical strong label annotation file is structured as follows:

```
filename,onset,offset,event_label
audio_001.wav,0.5,2.3,Speech
audio_001.wav,1.8,4.2,Dog
audio_001.wav,3.5,6.1,Speech
audio_001.wav,5.2,7.8,Dishes
audio_002.wav,0.0,1.5,Alarm_bell_ringing
audio_002.wav,0.8,3.2,Speech
audio_003.wav,2.1,5.4,Cat
audio_003.wav,4.8,8.3,Cat
```

This format explicitly captures temporal overlaps between events. For instance, in `audio_001.wav`, the "Dog" event (1.8–4.2s) overlaps with the first "Speech" event (0.5–2.3s) between 1.8 and 2.3 seconds, and also overlaps with the second "Speech" event (3.5–6.1s) between 3.5 and 4.2 seconds.

Advantages of Strong Labels Strong labels provide several critical benefits for sound event detection research and applications:

- Precise temporal localization: Enable training and evaluation of systems that can identify exactly when events occur
- Rich supervisory information: Provide detailed information about event duration, frequency of occurrence, and temporal relationships
- Direct evaluation: Allow for comprehensive evaluation using temporal localization metrics such as event-based F1-score and PSDS Section 2.6
- Application suitability: Essential for applications requiring real-time event monitoring, such as surveillance, healthcare monitoring, and industrial safety systems

Limitations of Strong Labels The detailed nature of strong labels introduces several practical challenges:

- High annotation cost: Requires significantly more time and effort, often 5-10 times longer than weak labeling [1]

- Annotator expertise: Demands trained annotators who can consistently identify event boundaries
- Limited dataset size: The high annotation cost typically results in smaller datasets compared to weakly labeled collections
- Inter-annotator variability: Different annotators may mark boundaries differently, particularly for events with unclear transitions [7]

Unlabeled Data

Unlabeled data consists of audio recordings without any annotations neither weak nor strong labels. While unlabeled data provides no direct supervisory signal, it has become increasingly important in modern sound event detection due to its abundance and the development of sophisticated semi-supervised and self-supervised learning techniques [5, 8].

Unlabeled audio data is relatively easy to collect at scale. Environmental recordings, web audio, and surveillance footage can provide virtually unlimited amounts of unlabeled data at minimal cost. The key challenge is leveraging this data effectively for training sound event detection models without explicit supervision.

The DCASE Challenge Task 4 has specifically emphasized the combination of strongly labeled, weakly labeled, and unlabeled data, demonstrating that modern sound event detection systems can effectively leverage all three data types to achieve superior performance compared to systems trained on strongly labeled data alone [5].

Polyphonic Sound Events

Real-world acoustic environments frequently exhibit polyphony, where multiple sound sources are active simultaneously, producing overlapping sound events. Polyphonic sound event detection represents one of the most challenging scenarios in computational auditory scene analysis [2].

In polyphonic scenarios, strong label annotations must capture not only the presence and timing of individual events but also their temporal overlaps. The annotation format described previously naturally supports polyphonic annotation: when multiple event instances have overlapping time intervals $[t_{\text{on},i}, t_{\text{off},i}]$ and $[t_{\text{on},j}, t_{\text{off},j}]$ such that the intervals intersect, the audio contains simultaneous sound events during the overlap period.

Figure 2.2 illustrates the fundamental difference between weak labels (tagging) and strong labels (polyphonic annotation). The left panel shows weak labels, which simply indicate that certain sound events are present in the audio clip without temporal information. The right panel demonstrates polyphonic annotation with strong labels, where multiple overlapping events are precisely marked with their onset and offset times, revealing the

temporal structure and concurrent activity of different sound sources [2].

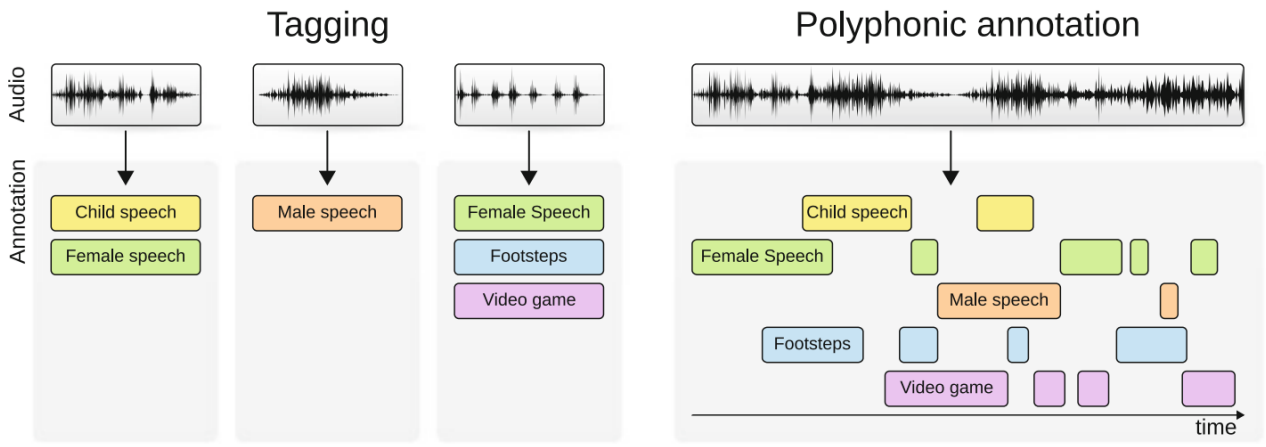


Figure 2.2: Comparison of weak labels (tagging, left) and strong labels (polyphonic annotation, right) for sound events. Source: [2]

Challenges of Polyphonic Detection Polyphonic sound event detection introduces several technical challenges:

- Source separation ambiguity: When multiple events overlap, their acoustic features may interact in complex ways, making it difficult to attribute spectro-temporal patterns to specific sources
- Class imbalance: Some event classes may occur more frequently than others, and polyphonic scenarios may be less common than monophonic ones in training data
- Evaluation complexity: Matching predicted events to ground truth becomes more complex when multiple events overlap, requiring sophisticated evaluation metrics
- Label dependency: The presence of one event may affect the perception and labeling of concurrent events, introducing potential inconsistencies

Modern deep learning approaches, particularly those based on recurrent neural networks and attention mechanisms, have shown promising results in handling polyphonic sound event detection by learning to simultaneously track multiple event classes over time [1].

2.4 Acoustic Feature Extraction: From Raw Audio to Log Mel Spectrograms for Sound Event Detection

The conversion of raw audio signals in the .wav format to log mel spectrograms is the foundational stage of most modern Sound Event Detection (SED) systems. While a waveform contains all the acoustic information, it is nearly impossible to differentiate

specific sound events from the time-domain pressure oscillations alone. Feature extraction transforms the signal into a compact, non-redundant representation that maximizes recognition performance by highlighting relevant physical properties like energy distribution and temporal evolution [1, 2]. The Overview is illustrated in Figure 2.3.

Signal Acquisition and Pre-processing:

The process begins with the acquisition of an analog signal x(t), which is converted to a digital discrete-time signal x[n] through discretization in both amplitude and time. [2]

Sampling and the Nyquist Criterion:

To ensure the digital signal faithfully represents the analog source, the sampling frequency f_s must satisfy the Sampling Theorem[9]. It must be at least twice the highest frequency component $f_{\max}$ to be captured:

$$f_s > 2f_{\max} \tag{2.2}$$

In practice, common sampling rates for SED are 16 kHz or 44.1 kHz. If data is collected from non-uniform sources, it is typically re-sampled to a fixed frequency and down-mixed to a single channel (mono) to ensure consistency [2]

Frame and Windowing:

Audio signals are non-stationary; their statistical properties change rapidly over time. To analyze them, we utilize short-time processing, slicing the signal into "quasi-stationary" segments called analysis frames [2]

Framing:

The signal is divided into frames of length L (typically 20 to 60 ms). For a general-purpose SED system, a window size is dependent on the transients and label accuracy [1].

To ensure temporal continuity, frames are shifted by a fixed timestep H (hop size), ensuring an overlap of at least 50 percent [2]. For example, in a system using a 512-sample hop at 22.05 kHz, the hop length is approximately 23 ms.[10]

Windowing:

Directly slicing a frame creates abrupt discontinuities at the boundaries, leading to spectral leakage (distortions) in the frequency domain [2]. To mitigate this, a windowing function w[k] is applied to each frame, tapering the signal to zero at the edges[11].

$$w[n] = \frac{1}{2}[1 - cos(\frac{2\pi n}{N-1})] \tag{2.3}$$

Windowing performs a trade-off: it reduces ripples in the spectrum at the expense of

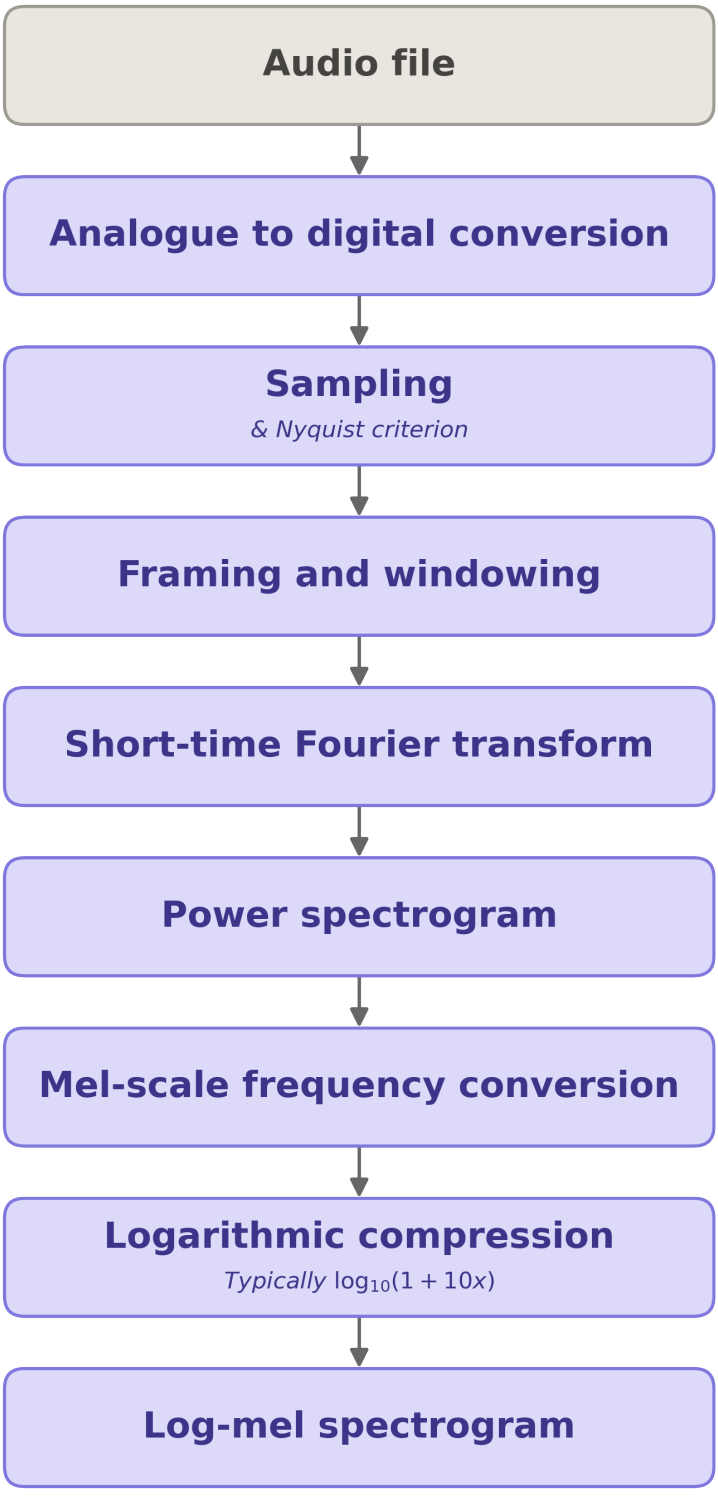


Figure 2.3: Overview of the audio preprocessing pipeline for generating log-Mel spectrograms from raw waveform input.

slightly smoothing the frequency resolution[9]

Short-Time Fourier Transform (STFT):

Each windowed frame is transformed into the frequency domain using the Discrete Fourier Transform (DFT) . This represents the signal as a superposition of sinusoidal base functions characterized by magnitude and phase.

The f-th component of the DFT for the t-th frame of signal $x[n]$ is computed as:

$$X[t, f] = \sum_{k=0}^{N-1} w[k]\, x[tN + k]\, e^{-i2\pi kf/N} \tag{2.4}$$

where N is the FFT size (usually the smallest power of two greater than the frame length). In practice, this is computed efficiently via the Fast Fourier Transform (FFT).

Power Spectrogram:

We compute magnitude squared:

$$S(m, k) = |X(m, k)|^2 \tag{2.5}$$

Mel Scale Frequency Warping:

The linear frequency scale (Hz) provided by the STFT does not match human perception. Humans are more sensitive to changes in low frequencies than in high frequencies. To mimic this, we warp the frequency axis to the Mel Scale [2]. Shown in Figure 2.4

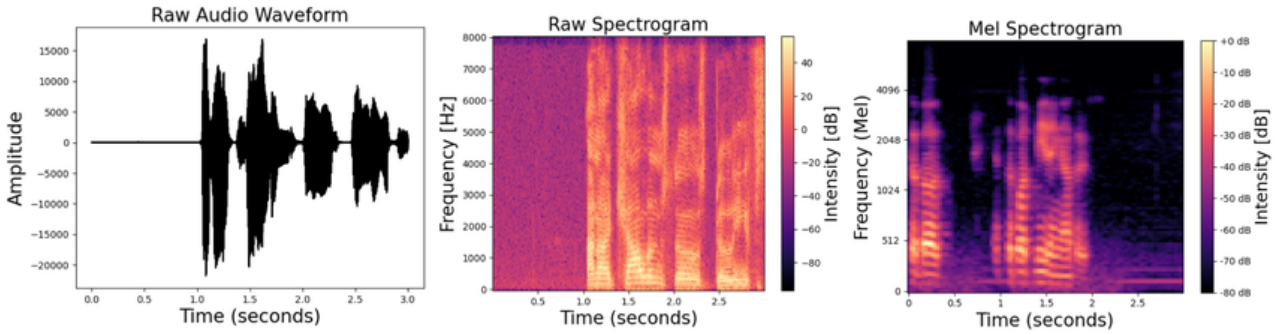


Figure 2.4: Graphical representation of conversion from Raw audio signal to Mel-spectrogram [3]

The Mel Scale Mapping:

The Mel scale is a perceptual scale where pitches judged by listeners to be equal in distance are spaced equally. The reference point is 1000 mels = 1000 Hz[1]. A common analytical approximation is given by[2]:

This mapping expands details in low frequencies and compresses it in high frequencies[11].

The name "Mel" comes from the word melody to indicate that the scale is based on pitch comparisons. The relationship between frequency in Hertz f and the Mel scale m is given by the following formula:

$$m = 2595 * log_{10}(1 + \frac{f}{700}) \tag{2.6}$$

As the formula shows, the mapping is nearly linear for frequencies below 1000 Hz but becomes increasingly logarithmic for higher frequencies. This effectively mirrors how our ears work, grouping high frequencies together while preserving resolution in the lower, more perceptually important frequency ranges for speech.

Mel Filterbank:

To obtain the Mel spectrum the power spectrum is passed through a bank of triangular band-pass filters equally spaced on the Mel scale. These filters integrate the spectrum over non-linear ranges narrow at low frequencies and wider at higher frequencies.[2, 11]

The energy $QX_m(t)$ in the m-th Mel band at time t is the weighted sum of the spectral amplitudes[1, 2]:

$$QX_m(t) = \sum |X(t, f)|^2 H_m(f) \tag{2.7}$$

where $H_m(f)$ is the weight defined by the triangular shape of the m-th filter .[2] While early systems (like MFCCs for speech) used 20–40 filters, modern SED systems often use up to 128 filters to resolve overlapping sounds at close frequencies.[1, 10]

Logarithmic Compression:

The final step in creating the log mel spectrogram is applying a logarithmic compression to the Mel spectrograms (typically log10)

2.5 Machine Learning for Sound Event Detection

Machine learning provides the computational framework that enables sound event detection systems to automatically learn discriminative patterns from audio data and make predictions about the presence and timing of sound events. This section presents an overview of the fundamental principles of machine learning as applied to sound event detection, progressing from general concepts to the specific deep learning architectures that have become the standard in the field.

2.5.1 From Traditional Methods to Deep Learning

Early approaches to sound event detection relied on **traditional machine learning** methods that employed a two-stage pipeline: first, hand-crafted features such as Mel-Frequency Cepstral Coefficients (MFCCs), spectral centroid, zero-crossing rate, and other statistical descriptors were extracted from audio signals; then, these features were fed into classifiers

such as Gaussian Mixture Models (GMMs), Hidden Markov Models (HMMs), Support Vector Machines (SVMs), or Non-negative Matrix Factorization (NMF) [1, 12].

While these traditional methods achieved reasonable performance for simple classification tasks, they faced several fundamental limitations for sound event detection:

- **Feature engineering bottleneck**: Hand-crafted features require significant domain expertise to design and are not guaranteed to capture the most discriminative information for a given task. Features optimized for one type of sound may perform poorly for others.
- **Limited representational capacity**: Shallow classifiers such as GMMs and SVMs have limited ability to model the complex, non-linear relationships between acoustic features and sound event labels, particularly in polyphonic scenarios where multiple events overlap.
- **Weak temporal modeling**: Traditional classifiers treat individual feature frames independently or rely on simplistic temporal models (e.g., HMMs with limited state complexity), failing to capture the rich temporal dynamics of real-world sound events.

Deep learning overcame these limitations by introducing neural networks with multiple layers that jointly learn feature representations and classification from unprocessed input data. Instead of relying on pre-defined features, deep neural networks learn hierarchical representations where each successive layer captures increasingly abstract and task-relevant patterns. This end-to-end learning paradigm has proven far more effective for complex audio analysis tasks, including polyphonic sound event detection [4].

2.5.2 Neural Network's for Sound Event Detection

Several neural network architectures have been explored for sound event detection, each bringing distinct capabilities. This subsection reviews the three foundational architectures feedforward neural networks, convolutional neural networks, and recurrent neural networks before presenting their integration in the Convolutional Recurrent Neural Network (CRNN).

Feedforward Neural Networks (FNNs)

The simplest deep learning architecture is the feedforward neural network (FNN), also known as the multilayer perceptron (MLP). An FNN consists of an input layer, one or more hidden layers, and an output layer, where information flows in a single direction from input to output. Each neuron in a hidden layer computes a weighted sum of its inputs, adds a bias term, and applies a non-linear activation function:

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{2.8}$$

where $\mathbf{h}^{(l)}$ is the output of the l-th hidden layer, $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weight matrix and bias vector of that layer, and $\sigma(\cdot)$ is a non-linear activation function such as the Rectified Linear Unit (ReLU), defined as $\sigma(x) = \max(0, x)$ [13].

While FNNs can learn non-linear mappings from spectral features to event labels, they treat each input frame independently without exploiting the two-dimensional structure of spectrograms or the temporal relationships between consecutive frames. This makes them suboptimal for sound event detection, where both spectral patterns and temporal context are crucial [4].

Convolutional Neural Networks (CNNs)

Originally developed for image recognition, Convolutional Neural Networks (CNNs) have been successfully applied to audio analysis by treating spectrograms as 2D image-like inputs. CNNs introduce two key innovations over FNNs: *local connectivity* through convolutional filters that capture local patterns, and *weight sharing* where the same filter is applied across the entire input, dramatically reducing the number of parameters while enabling translation-invariant feature detection.

A convolutional layer applies a set of learnable two-dimensional filters to the input. For a filter of size $K_t \times K_f$, the convolution operation at position (m, j) is:

$$\mathbf{Z}_p(m, j) = \sigma\left(\sum_{i=0}^{K_t-1}\sum_{l=0}^{K_f-1} \mathbf{W}_p(i, l) \cdot \mathbf{X}'(m + i, j + l) + b_p\right) \tag{2.9}$$

where $\mathbf{W}_p$ is the p-th convolutional filter, $\mathbf{X}'$ is the input feature map, and b_p is the bias term [14].

When applied to spectrograms, CNN filters learn to detect spectro-temporal patterns such as harmonic structures, onset transients, and frequency modulations. Lower layers capture fine-grained local features, while deeper layers combine these into higher-level representations corresponding to more complex acoustic patterns. However, standard CNNs have a limited receptive field and struggle to capture long-range temporal dependencies that span beyond the size of the convolutional filters, which limits their effectiveness for detecting sound events with extended or irregular temporal structures [4].

Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNN) are specifically designed for sequential data processing, maintaining an internal hidden state that is updated at each time step to capture temporal

dependencies. At time step t, a basic RNN updates its hidden state as:

$$\mathbf{h}_t = \sigma\left(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h\right) \tag{2.10}$$

where $\mathbf{x}_t$ is the input at time t, $\mathbf{h}_{t-1}$ is the previous hidden state, $\mathbf{W}_{xh}$ and $\mathbf{W}_{hh}$ are learnable weight matrices, and $\mathbf{b}_h$ is the bias vector [13].

Standard RNNs suffer from the vanishing gradient problem, where gradients diminish exponentially during backpropagation through time, making it difficult to learn long-range dependencies. This limitation is addressed by gated architectures, most notably the **Long Short-Term Memory** (LSTM) [15] and the **Gated Recurrent Unit** (GRU) [16].

The LSTM cell introduces a cell state $\mathbf{c}_t$ regulated by three gates:

$$\mathbf{f}_t = \sigma_g(\mathbf{W}_f\mathbf{x}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{b}_f) \quad \text{(forget gate)} \tag{2.11}$$

$$\mathbf{i}_t = \sigma_g(\mathbf{W}_i\mathbf{x}_t + \mathbf{U}_i\mathbf{h}_{t-1} + \mathbf{b}_i) \quad \text{(input gate)} \tag{2.12}$$

$$\mathbf{o}_t = \sigma_g(\mathbf{W}_o\mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{b}_o) \quad \text{(output gate)} \tag{2.13}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c\mathbf{x}_t + \mathbf{U}_c\mathbf{h}_{t-1} + \mathbf{b}_c) \quad \text{(candidate state)} \tag{2.14}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{(cell state update)} \tag{2.15}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{(hidden state)} \tag{2.16}$$

where σ_g is the sigmoid function and $\odot$ denotes element-wise multiplication. The forget gate $\mathbf{f}_t$ controls what information to discard from the cell state, the input gate $\mathbf{i}_t$ controls what new information to store, and the output gate $\mathbf{o}_t$ controls what information to output. This gating mechanism enables LSTMs to selectively retain or forget information over long sequences, making them well-suited for modeling temporal patterns in audio [15].

While RNNs excel at modeling temporal dependencies, they operate on pre-extracted feature vectors at each time step and do not inherently exploit the two-dimensional spectro-temporal structure of spectrograms, relying on the input features to provide adequate spectral information [4].

2.5.3 Convolutional Recurrent Neural Network (CRNN)

The Convolutional Recurrent Neural Network (CRNN) was introduced for polyphonic sound event detection by Çakır et al. [4], combining the complementary strengths of CNNs and RNNs into a unified architecture. The fundamental insight behind the CRNN is that CNNs excel at extracting local spectro-temporal features that are invariant to small shifts in frequency and time, while RNNs are powerful in capturing long-term temporal context and that sound event detection fundamentally requires both capabilities [4, 17].

The CRNN architecture consists of three principal stages arranged sequentially: a

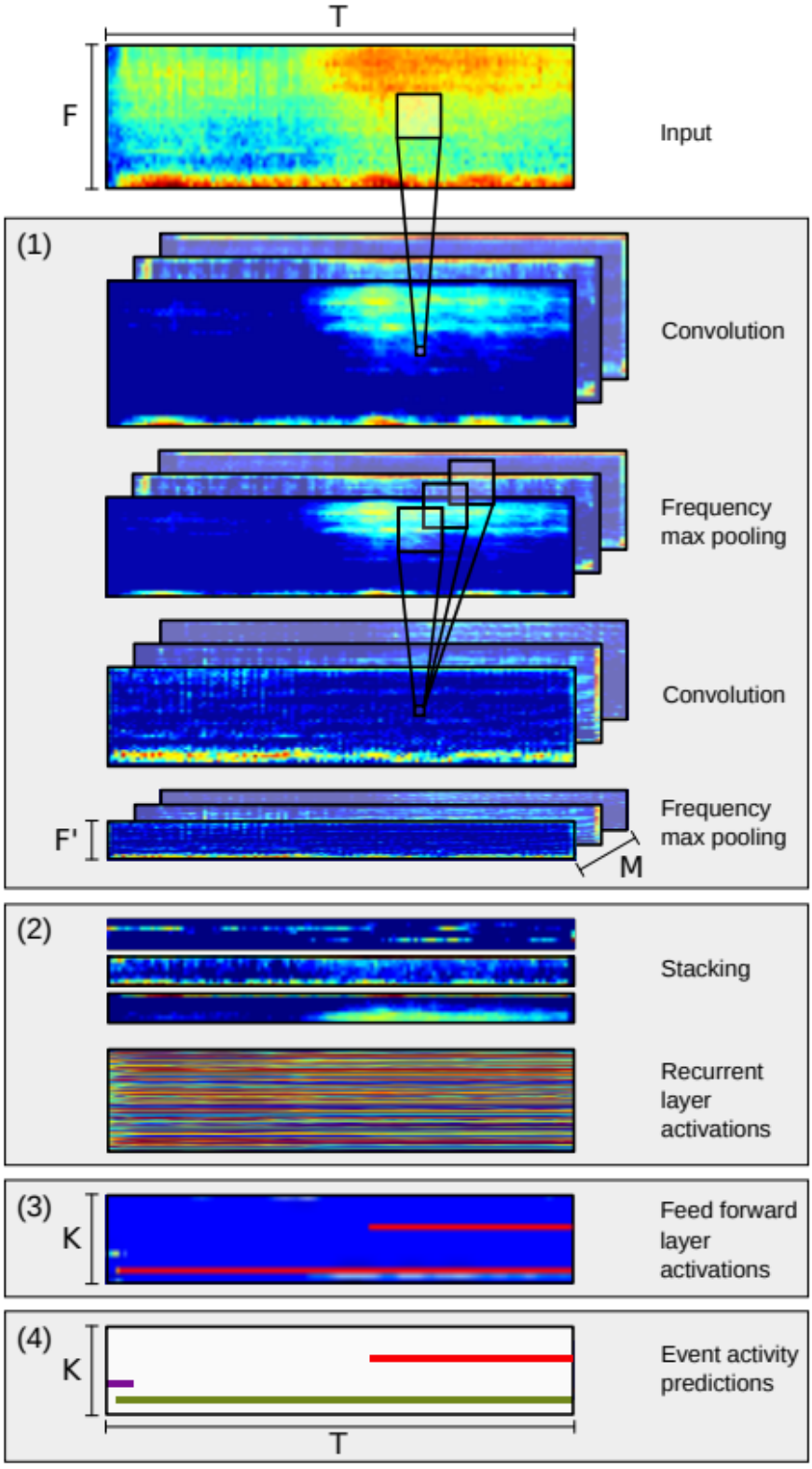


Figure 2.5: Overview of the CRNN method. (1): Multiple convolutional layers with max pooling in frequency axis, (2): The outputs of the last convolutional layer stacked over frequency axis and fed to multiple stacked recurrent layers, (3): feedforward layer as output layer and (4): binarization of event activity probabilities [4]

convolutional feature extraction front end, a recurrent temporal modeling module, and a fully connected classification layer. Representation of CRNN in Figure 2.5

Convolutional Feature Extraction

The convolutional block forms the front end of the CRNN and processes the input log-mel spectrogram $\mathbf{X} \in \mathbb{R}^{T \times F}$, where T is the number of time frames and F is the number of mel frequency bands. Multiple convolutional blocks are stacked sequentially, each typically comprising a 2D convolutional layer with P filters of kernel size 3×3, followed by batch normalization, ReLU activation, max pooling, and dropout [4, 17].

A critical design choice in CRNN architectures for sound event detection is the application of max pooling *only along the frequency axis* (e.g., with pooling size $1 \times n$), while preserving the temporal resolution unchanged. This is essential because the temporal dimension directly corresponds to the frame-level predictions required for precise event onset and offset detection [4, 18].

Through successive convolutional layers, the network learns a hierarchy of features: early layers capture low-level patterns such as spectral edges, harmonic structures, and onset transients, while deeper layers combine these into higher-level representations corresponding to more abstract and discriminative sound characteristics. After processing through all convolutional blocks, the frequency dimension is substantially reduced through pooling, while the temporal dimension is retained, producing feature maps of shape $T \times F' \times P'$, where $F' \ll F$ and P' is the number of filters in the final convolutional layer [4].

Recurrent Temporal Modeling

After convolutional feature extraction, the feature maps are reshaped by collapsing the frequency and channel dimensions into a single feature vector for each time frame, producing a sequence of feature vectors $\{\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_T\}$. This sequence is then passed to one or more recurrent layers that model the temporal dependencies between consecutive frames [4].

In most CRNN implementations for sound event detection, **bidirectional** recurrent layers are employed. A bidirectional layer processes the sequence in both forward and backward directions simultaneously using two separate sets of recurrent units, and the outputs are concatenated, added or multiplied:

$$\mathbf{h}_t^{\text{bi}} = [\overrightarrow{\mathbf{h}_t} \, ; \, \overleftarrow{\mathbf{h}_t}] \tag{2.17}$$

where $\overrightarrow{\mathbf{h}_t}$ and $\overleftarrow{\mathbf{h}_t}$ represent the hidden states from the forward and backward passes, respectively. This bidirectional processing allows the network to incorporate both past and future context when making predictions for each time frame, which is particularly beneficial

for accurately determining event boundaries and also removes real time capabilities of inference. [4, 17].

Classification Layer and Output

The final component of the CRNN is a fully connected layer with sigmoid activation that produces frame-level predictions for each sound event class. For each time frame t, the output is:

$$\hat{y}_{t,c} = \sigma\left(\mathbf{w}_c^\top \mathbf{h}_t^{\text{bi}} + b_c\right), \quad c = 1, 2, \ldots, C \tag{2.18}$$

where $\hat{y}_{t,c} \in [0, 1]$ represents the predicted probability that event class c is active at time frame t, and C is the total number of event classes.

The sigmoid activation function is used instead of softmax because sound event detection is inherently a multi-label classification problem: multiple event classes can be active simultaneously in polyphonic scenarios. Each output neuron independently predicts the probability for its corresponding class, allowing the network to detect overlapping events [4].

The network is trained by minimizing the **binary cross-entropy** (BCE) loss, computed independently for each class across all time frames:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{T \cdot C} \sum_{t=1}^{T} \sum_{c=1}^{C} \left[y_{t,c} \log(\hat{y}_{t,c}) + (1 - y_{t,c}) \log(1 - \hat{y}_{t,c}) \right] \tag{2.19}$$

where $y_{t,c} \in \{0, 1\}$ is the ground truth label indicating whether event class c is active at frame t [4].

Post-Processing

During inference, the continuous sigmoid output probabilities must be converted into binary detection decisions. This is achieved by applying a decision threshold γ_c to each event class:

$$\hat{d}_{t,c} = \begin{cases} 1 & \text{if } \hat{y}_{t,c} \geq \gamma_c \\ 0 & \text{otherwise} \end{cases} \tag{2.20}$$

Consecutive frames where $\hat{d}_{t,c} = 1$ are then merged into detected events characterized by their onset time, offset time, and class label. Additionally, **post-processing** techniques such as median filtering are commonly applied to the frame-level predictions before or after thresholding to suppress short-duration false alarms and fill brief gaps within detected events, producing more temporally coherent predictions [19].

Advantages of the CRNN Architecture

The CRNN architecture offers several important advantages that have established it as the dominant approach for sound event detection:

- **Complementary feature learning**: The CNN component extracts shift-invariant local spectro-temporal features, while the RNN component captures long-range temporal dependencies. Together, they provide a more comprehensive representation than either component alone [4].
- **Parameter efficiency**: Experimental evaluations have shown that the CRNN achieves superior performance with fewer total parameters compared to standalone CNNs or RNNs, indicating a fundamental architectural advantage in combining both approaches [4]..

Experimental evaluations on benchmark datasets have demonstrated that the CRNN architecture outperforms both standalone CNN and RNN approaches, with reported improvements of approximately 6.6% and 13.6% in frame-based F1 scores over CNN and RNN relatively, respectively [4]. This consistent improvement across multiple datasets has established the CRNN as the foundational architecture for modern sound event detection systems and has served as the basis for subsequent innovations incorporating attention mechanisms, dilated convolutions [18], and pre-trained audio embeddings.

2.6 Evaluation Metrics for Sound Event Detection

Sound Event Detection systems are evaluated using two types of metrics: *segment-based* metrics and *event-based* metrics. Each family measures detection performance under different assumptions about what constitutes a correct prediction, and both are necessary to obtain a complete picture of a model's behaviour. The choice of metric directly reflects the intended use case: segment-based metrics are appropriate when the system must identify what is happening at every moment in time, while event-based metrics are appropriate when the system must detect and localise discrete occurrences of events. This section describes both families in detail, following the formulations standardised by the Detection and Classification of Acoustic Scenes and Events (DCASE) challenge [7].

2.6.1 Segment-Based Evaluation

Definition and Motivation

Segment-based evaluation divides the entire audio signal into short, fixed-length time frames typically one second in duration and treats each frame as an independent classification

decision. For each frame and each sound class, the system must decide whether that class is active or inactive during that frame. The frame is then compared against the ground-truth annotation for the same interval: if the ground truth indicates that the class was active for any portion of that frame, the frame is labelled as a ground truth positive (true) example for that class; otherwise it is a negative (false) example.

This frame-wise formulation transforms the temporal detection problem into a standard multi-label classification problem, where each frame may belong to zero, one, or multiple classes simultaneously. The evaluation then proceeds by computing precision, recall, and F1-score across all frames in the test set, aggregated either per class or across all classes. Figure 2.6 represents this mertic graphically.

Cases

Four types are defined for each frame and each class:

- **True Positive (TP):** The system predicts the class is active in the frame, and the ground truth confirms that the class was active during any part of that frame.
- **False Positive (FP):** The System predicts the class is active, but the ground truth indicates the class was not active at all during that frame (Error Type).
- **True Negative (TN):** The system correctly predicts that the class is not active, and the ground truth confirms this.
- **False Negative (FN):** It predicts the class is not active, but the ground truth shows the class was active during some portion of the frame (Error Type).

Metrics

From these counts, the standard classification metrics are computed. For a single class c, precision and recall are defined as:

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c} \tag{2.21}$$

$$\text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c} \tag{2.22}$$

Precision measures the proportion of the system's positive predictions that were correct, while recall measures the proportion of ground-truth positive frames that the system successfully identified. The F1-score is the harmonic mean of precision and recall:

$$F1_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{2.23}$$

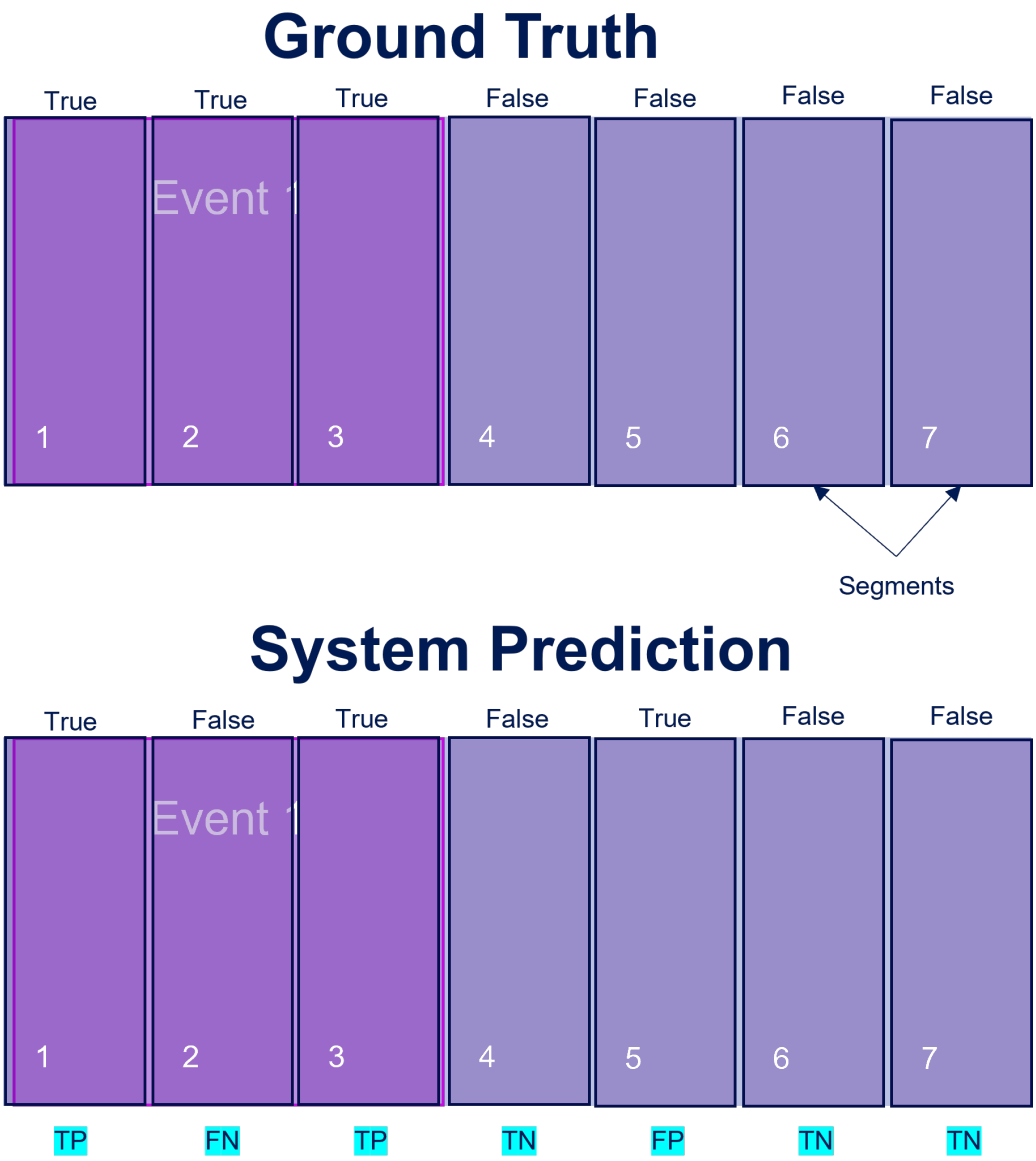


Figure 2.6: Segment-based evaluation example illustrating the comparison between ground truth and system predictions across temporal segments, highlighting correct detections (TP), missed detections (FN), false alarms (FP), and correct rejections (TN).

To obtain an overall performance measure across all classes, two aggregation strategies are commonly used. **Micro-averaging** computes a single set of TP, FP, and FN counts by summing across all classes before computing precision, recall, and F1. This gives equal weight to every frame-class pair, so classes that occupy more total time in the dataset contribute more to the final score. **Macro-averaging** computes precision, recall, and F1 for each class independently, then takes the arithmetic mean of these per-class scores. This gives equal weight to each class regardless of how frequently it appears, which is often preferred when the dataset contains imbalanced class distributions.

Advantages and Limitations

Segment-based evaluation has the advantage of conceptual and computational simplicity. It requires no matching algorithm or tolerance parameters, every frame is evaluated independently and the resulting metrics are directly interpretable in the language of binary classification. The fixed temporal granularity (typically one second) also makes the metrics robust to small jitter in the predicted onset and offset times: if a predicted event boundary is off by a few hundred milliseconds, the affected frames may still be counted as correct as long as the event occupies the majority of the frame.

However, this frame-wise independence is also a limitation. Segment-based metrics do not measure whether the system has correctly identified discrete event *instances*. A system that predicts a single long event spanning ten seconds and a system that predicts ten separate one-second events in the same interval will receive identical segment-based scores, even though the second system has fragmented the detection into faulty boundaries. Similarly, a system that predicts the correct class but with onset and offset errors of several seconds may still achieve a high segment-based F1 if most frames overlap with the ground truth, even though the temporal localisation is poor. For applications where the number and timing of discrete events matter such as counting occurrences, measuring inter-event intervals, or triggering time-sensitive actions segment-based metrics alone are insufficient.

2.6.2 Event-Based Evaluation

Definition and Motivation

Event-based evaluation treats each predicted event and each ground-truth event as a discrete temporal object, defined by an onset time, an offset time, and a class label. The evaluation algorithm explicitly matches predicted events to ground-truth events based on temporal overlap and class agreement, then counts how many ground-truth events were successfully detected and how many predicted events were not. This formulation directly measures the system's ability to localise individual occurrences of sound events in time, which is the defining task of Sound Event Detection.

The Matching Algorithm

The standard event-based matching procedure, as defined by the DCASE challenge, proceeds as follows. For each class independently:

1. Collect all ground-truth events of that class and all predicted events of that class.
2. For each predicted event, compute its temporal overlap with every ground-truth

event of the same class. Overlap is measured as the intersection of the two time intervals.

3. A predicted event is considered a true positive if it overlaps with at least one ground-truth event by more than a specified threshold. The standard threshold is 50% of the shorter of the two events' durations, though some evaluation frameworks use stricter or looser thresholds.

4. Each ground-truth event may be matched to at most one predicted event. If multiple predicted events overlap the same ground-truth event, only the one with the largest overlap is counted as a true positive; the others are counted as false positives.

5. Any predicted event that does not meet the overlap threshold with any ground-truth event is a false positive.

6. Any ground-truth event that is not matched by any predicted event is a false negative.

This procedure is applied separately for each class, and the resulting TP, FP, and FN counts are then aggregated across classes using either micro- or macro-averaging, as in the segment-based case.

Metrics

Precision, recall, and F1 are computed from the event-level counts using the same formulae as in the segment-based case:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.24}$$

However, the semantics are different: precision now measures the proportion of predicted *event instances* that correspond to real events, and recall measures the proportion of real events that were successfully detected. A system with high event-based recall detects most of the discrete occurrences in the test set, while a system with high event-based precision makes few false detections.

Onset-Only Evaluation

A variant of event-based evaluation, sometimes reported alongside the full onset-offset matching, is *onset-only* evaluation. In this mode, a predicted event is considered correct if its onset time falls within a tolerance window (or a Collar) Figure 2.7 (typically ±500 ms or ±200 ms) of a ground-truth event's onset, regardless of the predicted offset. This metric isolates the system's ability to detect *when* an event begins, which is often the most critical piece of information for applications such as real-time alerting or event counting.

Onset-only F1 is generally higher than full onset-offset F1 because it does not penalise the system for imprecise offset predictions, which are inherently harder to estimate due to gradual decay characteristics of many real-world sounds.

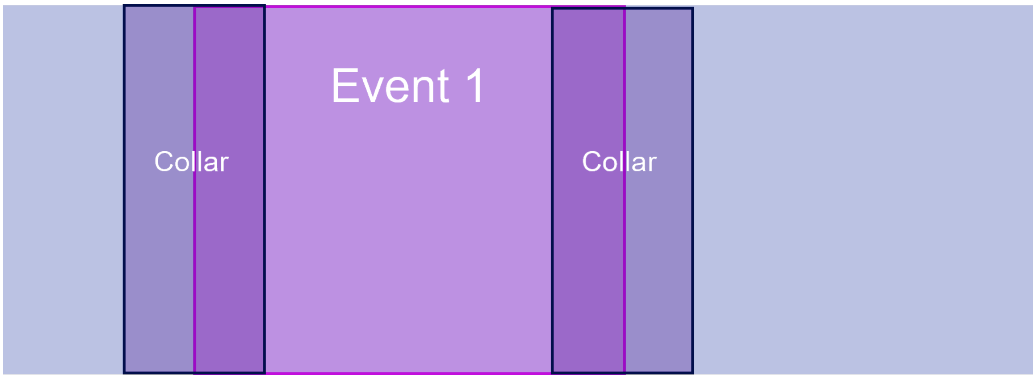


Figure 2.7: Tolerance windows (collars) used in event-based evaluation for onset and offset matching, where predictions within the onset and offset tolerance are considered correct detections (if 50% or more overlap).

Advantages and Limitations

Event-based evaluation directly measures the task that Sound Event Detection systems are designed to perform: identifying discrete sound occurrences in time. It penalises fragmentation (predicting multiple events where only one exists) and penalises missed detections in a way that reflects the actual number of events lost. It is also sensitive to temporal precision: a predicted event that overlaps a ground-truth event by only 30% of its duration will not be counted as correct under the standard 50% threshold, encouraging systems to learn tight temporal boundaries.

The primary limitation is sensitivity to the matching threshold. A system that predicts events with 48% overlap will score zero, while one with 51% overlap will score perfectly, even though the two systems' outputs may be perceptually indistinguishable. The choice of threshold is therefore somewhat arbitrary, and results should ideally be reported at multiple thresholds to understand the system's behaviour across the precision-recall trade-off. Additionally, event-based metrics can be unstable on datasets with very few events per class, where a single missed detection can cause large swings in recall or precision.

Complementarity of the Two Metrics

Segment-based and event-based metrics provide complementary views of detection performance. A system may achieve high segment-based F1 by correctly identifying the *presence* of a class in most time frames, but still achieve low event-based F1 if it fragments the detections into many spurious short events or fails to localise event boundaries accurately.

Conversely, a system that produces very few predictions perhaps only detecting the most salient, high-SNR events may achieve high event-based precision but low segment-based recall because it leaves many frames unlabelled.

In practice, both metrics are reported. Segment-based F1 is appropriate when the goal is frame-level tagging or activity detection for example, summarising which classes were present in a recording. Event-based F1 is appropriate when the goal is instance detection and temporal localisation for example, counting how many sirens occurred during a ten-minute recording, or determining the precise moment a car horn sounded. The best systems achieve strong performance on both metrics, demonstrating both accurate temporal localisation (high event-based scores) and comprehensive coverage of all active classes at all times (high segment-based scores).

Implementation and Tools

The reference implementation for both segment-based and event-based evaluation in Sound Event Detection is provided by the `sed_eval` Python library [7], which is the official evaluation toolkit for the DCASE challenge. This library accepts predictions and ground truth in a standardised CSV format (with columns for filename, onset, offset, and label) and computes all standard metrics with user-specified segment lengths and overlap thresholds. Using this library ensures that results are directly comparable to those reported in the published DCASE literature.

2.6.3 Polyphonic Sound Detection Score (PSDS)

While segment-based and event-based F1 scores provide useful metrics for evaluating sound event detection systems at specific operating points, they fail to capture the complete picture of system performance across different operational modes. To address this limitation, the polyphonic sound detection score (PSDS) was introduced as a comprehensive evaluation metric that summarizes system performance over a range of operating points [20, 21].

The PSDS metric is designed to overcome the bias introduced by threshold tuning and to better represent the diverse application requirements of sound event detection systems. In many real-world applications, the optimal balance between precision and recall varies significantly. For instance, in security monitoring systems, missed detections (false negatives) may be much more critical than false alarms, whereas in quality-of-life applications, excessive false alarms may be unacceptable [19].

PSD-ROC Curves

The computation of PSDS begins with the construction of single-class PSD-ROC (Receiver Operating Characteristic) curves $r_c(e)$ for each event class c. These curves are obtained through a continuous "staircase-type" interpolation of true positive rates $r_{c,\tau}$ plotted against corresponding effective false positive rates (eFPR) $e_{c,\tau}$ for different operating parameters τ [20].

The effective false positive rate is defined as:

$$e_{c,\tau} = \frac{N_{FP,c,\tau}}{T_{ds}} + \alpha_{CT} \frac{1}{C-1} \sum_{\substack{k \\ k \neq c}} \frac{N_{CT,c,k,\tau}}{T_k} \tag{2.25}$$

where $N_{FP,c,\tau}$ is the number of false positive detections, T_{ds} is the total duration of the evaluation dataset, $N_{CT,c,k,\tau}$ represents the number of cross-triggers (substitutions) where class c is detected instead of class k, T_k is the total duration of activity for class k, and α_{CT} is a weighting parameter for the cross-trigger penalty.

To ensure meaningful comparison, only best-case operating parameters are considered in the PSD-ROC construction:

$$\mathcal{T}_c^b = \{\tau \mid \nexists \lambda \text{ with } e_{c,\lambda} \leq e_{c,\tau} \text{ and } r_{c,\lambda} > r_{c,\tau}\} \tag{2.26}$$

This constraint ensures that for any given false positive rate, the operating point with the highest true positive rate is selected.

Overall PSD-ROC and PSDS Computation

The overall PSD-ROC curve aggregates performance across all event classes and is defined as the effective true positive rate:

$$r(e) = \mu_{TP}(e) - \alpha_{ST} \sigma_{TP}(e) \tag{2.27}$$

where $\mu_{TP}(e)$ represents the average per-class true positive rate and $\sigma_{TP}(e)$ is the standard deviation of true positive rates across classes:

$$\mu_{TP}(e) = \frac{1}{C} \sum_{c=1}^{C} r_c(e) \tag{2.28}$$

$$\sigma_{TP}(e) = \sqrt{\frac{1}{C} \sum_{c=1}^{C} \left(r_c(e) - \mu_{TP}(e)\right)^2} \tag{2.29}$$

The parameter α_{ST} controls the penalty applied to systems with inconsistent perfor-

mance across different event classes, encouraging balanced performance.

Finally, PSDS is computed as the normalized area under the PSD-ROC curve:

$$\text{PSDS} = \frac{1}{e_{max}} \int_0^{e_{max}} r(e)\, de \tag{2.30}$$

where e_{max} is a metric parameter that defines the maximum false positive rate up to which operating points are considered relevant. This parameter allows the metric to be tailored to specific application requirements.

Evaluation Scenarios

In practical applications, PSDS can be configured with different parameters to reflect different evaluation scenarios. For example, in the DCASE Challenge of sound event detection [19]:

- **PSDS1** ($\rho_{DTC} = 0.7$, $\rho_{GTC} = 0.7$, $\alpha_{CT} = 0$, $\alpha_{ST} = 1$, $e_{max} = 100$/hour) evaluates the model's capability to temporally localize sound events with high precision.
- **PSDS2** ($\rho_{DTC} = 0.1$, $\rho_{GTC} = 0.1$, $\rho_{CTTC} = 0.3$, $\alpha_{CT} = 0.5$, $\alpha_{ST} = 1$, $e_{max} = 100$/hour) focuses on evaluating reliable recognition of event classes within an audio clip, with more lenient temporal matching requirements.

The PSDS metric has been widely adopted in the sound event detection community, particularly in benchmark challenges, as it provides a more complete and less biased evaluation of system performance compared to single operating point metrics [22].

3 State of the Art in Sound Event Detection

3.1 Semi-Supervised Learning Paradigms

Supervised deep learning has achieved remarkable success across a wide range of tasks, but its reliance on large, carefully annotated datasets remains a fundamental bottleneck. In many real-world domains and in sound event detection in particular obtaining accurate strong labels is expensive, time-consuming, and often requires expert annotators, whereas unlabeled data are abundant and cheap to collect. Semi-supervised learning (SSL) addresses this asymmetry by leveraging a typically small labeled set $\mathcal{D}_l = \{(x_i, y_i)\}_{i=1}^{N_l}$ jointly with a much larger unlabeled set $\mathcal{D}_u = \{x_j\}_{j=1}^{N_u}$, with $N_u \gg N_l$, during training [23].

For SSL to be effective, the distribution of the unlabeled data must carry useful information about the conditional distribution $p(y \mid x)$. Classical SSL theory formalizes this through three assumptions that underpin essentially all modern algorithms [23]: the *smoothness assumption* (points close in input space should receive similar predictions), the *cluster (or low-density separation) assumption* (decision boundaries should lie in low-density regions, so points within the same cluster likely share a label), and the *manifold assumption* (high-dimensional data concentrate on a lower-dimensional manifold along which the label varies smoothly). The deep SSL methods discussed below can be viewed as practical instantiations of these principles, exploiting either local input perturbations (smoothness) or confident model predictions (low-density separation) to extract a learning signal from unlabeled data [24].

This section reviews the four families of SSL methods that are most relevant to this thesis. Sections 3.1.1 and 3.1.2 introduce the two foundational paradigms consistency regularization and pseudo-labeling whose combination underlies almost all current state-of-the-art SSL systems. Section 3.1.3 then describes the Mean Teacher framework in detail, as it forms the methodological backbone of the DCASE Task 4 baseline 2021 and of the systems developed in this work. Section 3.3 discusses more recent SSL methods that build on these ideas.

3.1.1 Consistency Regularization

Consistency regularization is rooted directly in the smoothness assumption: a classifier f_θ should produce stable predictions under small, semantics-preserving perturbations of its input [25, 26]. Given an unlabeled sample $x \in \mathcal{D}_u$ and two stochastic perturbations η, η' (e.g., data augmentation, dropout, Gaussian noise), a typical consistency loss takes the

form

$$\mathcal{L}_{\text{cons}}(x) = d(f_\theta(x, \eta),\ f_\theta(x, \eta'))\,, \tag{3.1}$$

where $d(\cdot, \cdot)$ is a divergence such as the mean squared error (MSE) or the Kullback Leibler (KL) divergence. Critically, this loss is unsupervised: it depends only on the model's own predictions and can therefore be evaluated on the entire unlabeled set. The Π-model of Laine and Aila [26] is the canonical example, applying two independent stochastic forward passes per unlabeled sample and minimizing the squared distance between the resulting probability vectors. Their Temporal Ensembling variant replaces one of these passes by an exponential moving average (EMA) of the model's per-sample predictions across past epochs, yielding a more stable target at the cost of having to store a prediction vector for every training example [26].

A complementary line of work generates perturbations adversarially rather than stochastically. Virtual Adversarial Training (VAT) [27] computes, for each input, the direction in input space that most changes the model's output distribution, and penalizes the divergence between predictions on the original and the adversarially perturbed input. Because this direction is determined purely from the model's own output and not from any label, VAT naturally extends to the unlabeled set, and it can be combined with entropy minimization for further gains [27]. From a theoretical standpoint, consistency regularization implements the low-density separation principle: by forcing predictions to be invariant within local neighborhoods of unlabeled points, the decision boundary is pushed into regions of low data density [23, 24].

3.1.2 Pseudo-Labeling

Pseudo-labeling, sometimes called self-training, is conceptually simpler than consistency regularization. The model is first trained on the labeled set $\mathcal{D}_l$, then used to predict labels for the unlabeled set; predictions that exceed a confidence threshold are treated as if they were ground truth and added to the training pool, and the process is iterated [28]. Formally, for an unlabeled example x_u, the pseudo-label is obtained as

Despite its simplicity, Lee's original work showed that pseudo-labeling combined with dropout was competitive with much more elaborate generative SSL methods of its time [28]. From a probabilistic viewpoint, training on hard pseudo-labels is closely related to entropy minimization: it pushes the model toward producing low-entropy (confident) predictions on unlabeled data, which again aligns with the low-density separation assumption.

The main weakness of pseudo-labeling is *confirmation bias*: once the model commits to an incorrect prediction on an unlabeled sample, training on that pseudo-label reinforces the error, which can degrade performance in low-label regimes [29]. Several remedies have been proposed, including confidence thresholding, class-balanced sampling, mixup-based

regularization, and re-initializing the model between self-training rounds (curriculum labeling) [29]. Modern SSL methods rarely use pseudo-labeling in isolation; instead, they couple it with consistency regularization so that the model is forced to produce consistent predictions across perturbations of the same input, which acts as an implicit filter against unreliable pseudo-labels [30].

3.1.3 The Mean Teacher Framework

The Mean Teacher framework [31] is the methodological foundation of the DCASE Task 4 baseline systems and of the work presented in this thesis. It combines consistency regularization with a particularly stable target-generation mechanism based on weight averaging, and it is therefore worth describing in detail.

Motivation. The Π-model evaluates the consistency loss between two stochastic passes of the *same* network, which means that the "target" used to supervise the unlabeled samples is itself noisy and changes at every gradient step. Temporal Ensembling improves on this by averaging predictions across epochs, but it requires maintaining a per-example prediction buffer and updates each target only once per epoch [26]. Tarvainen and Valpola observed that both shortcomings can be addressed by averaging in *weight space* rather than in prediction space, which yields target updates after every minibatch without any per-example storage [31].

Architecture. Mean Teacher maintains two networks with identical architecture:

- a *student* f_θ, with parameters θ updated by stochastic gradient descent on the total loss; and
- a *teacher* $f_{\theta'}$, whose parameters θ' are an exponential moving average (EMA) of the student's parameters across training iterations:

$$\theta'_t = \alpha\,\theta'_{t-1} + (1-\alpha)\,\theta_t, \tag{3.2}$$

  where $\alpha \in [0, 1)$ is the EMA decay (typically $\alpha = 0.999$).

The teacher therefore implements an implicit temporal ensemble of student snapshots, weighted exponentially toward the most recent iterations.

Training objective. Given a labeled batch $\{(x_i, y_i)\}_{i=1}^{B_l} \subset \mathcal{D}_l$ and an unlabeled batch $\{x_j\}_{j=1}^{B_u} \subset \mathcal{D}_u$, the total loss is the sum of a supervised term and a consistency term:

$$\mathcal{L} = \underbrace{\frac{1}{B_l} \sum_{i=1}^{B_l} \mathcal{L}_{\text{sup}}(f_\theta(x_i, \eta_i), y_i)}_{\text{supervised}} + \lambda(t) \underbrace{\frac{1}{B_u} \sum_{j=1}^{B_u} d\big(f_\theta(x_j, \eta_j),\ f_{\theta'}(x_j, \eta'_j)\big)}_{\text{consistency}}, \tag{3.3}$$

where η_i, η_j, η'_j denote independent stochastic perturbations (data augmentation, dropout, Gaussian noise), $\mathcal{L}_{\text{sup}}$ is typically cross-entropy (or binary cross-entropy for multi-label problems such as SED), $d(\cdot, \cdot)$ is the MSE between probability vectors, and $\lambda(t)$ is a weight that ramps up from zero according to a sigmoid schedule to prevent the consistency loss from dominating early in training when the teacher's predictions are essentially random [31].

Crucially, gradients flow only through the student. The teacher's predictions act as "soft targets" that the student must match, but the teacher itself is updated solely through Equation 3.2 it never receives a gradient.

Why weight averaging helps. Three properties make Mean Teacher more effective than its predecessors [31]. First, because θ' is an average over many past values of θ, the teacher's predictions are substantially smoother in time than the student's, providing a more stable target and reducing the noise of the consistency signal. Second, weight averaging is a form of model ensembling that is known to generalize better than any individual snapshot, so the teacher typically produces *higher-quality* predictions than the student on its own, which is precisely what is needed for self-training to work. Third, the teacher is updated after every minibatch rather than once per epoch, which makes it scale gracefully to large datasets where Temporal Ensembling becomes impractical [31, 26].

Empirically, Tarvainen and Valpola reported substantial improvements over Temporal Ensembling on standard SSL image benchmarks, including a reduction in error rate to 4.35% on SVHN dataset with only 250 labels [31]. Beyond image classification, the framework has been adopted as the default semi-supervised paradigm for DCASE Task 4 since 2018, where it is used to exploit large unlabeled in-domain audio collections in conjunction with limited strongly-labeled data a use case that will be discussed in detail in Section 3.2.2.

3.2 Semi-Supervised Learning Applied to SED

This section reviews how the SSL paradigms of Section 3.1 have been adapted to the SED setting: Section 3.2.1 traces the evolution of the DCASE Task 4 challenge, which has effectively defined the SED benchmark for the past several years; Section 3.2.2

describes how the Mean Teacher framework is instantiated in the standard SED baseline; and Section 3.2.3 surveys the data-augmentation strategies that have become essential complements to the consistency loss.

3.2.1 DCASE Task 4 Evolution

The Detection and Classification of Acoustic Scenes and Events (DCASE) challenge has organized a dedicated Task 4 on sound event detection in domestic environments since 2018. The task has been remarkably consistent in spirit but has evolved substantially in the datasets, evaluation metrics, and modelling assumptions that it adopts; this evolution provides a convenient timeline of the state of the art in semi-supervised SED.

2018: Weak Labels and Large-Scale Unlabeled Audio.

The first edition of Task 4 framed SED explicitly as a large-scale, weakly-labeled, semi-supervised problem in domestic environments [32]. Training data consisted of a small weakly-labeled set (clip-level tags only) drawn from AudioSet together with a much larger *unlabeled* in-domain set, and systems were asked to produce strong, frame-level predictions on the evaluation set. The setting therefore combined three difficulties that remain at the heart of the task today: weak supervision, semi-supervised training, and the mismatch between clip-level training labels and event-level evaluation. The Mean Teacher framework [31], introduced for image classification only a year before, was the basis of the best-performing submission [33] and became the de-facto baseline architecture in subsequent editions.

2019 2020: DESED and synthetic soundscapes.

The DCASE 2019 edition introduced the Domestic Environment Sound Event Detection (DESED) dataset [34], which augmented the AudioSet-derived weak and unlabeled subsets with a new *synthetic, strongly-labeled* subset generated with the Scaper soundscape-synthesis library. The result is the three-stream training configuration that has been standard ever since: a small strongly-labeled synthetic set, a weakly-labeled real set, and a large unlabeled real set. This configuration is also the natural fit for the Mean Teacher framework, because each stream is processed differently in the loss (strong supervision on synthetic data, weak supervision on weakly-labeled data, consistency loss on all data). DCASE 2020 maintained this structure and additionally introduced source separation as an optional preprocessing stage and the synthetic evaluation analysis of Serizel et al. [35].

2020 2021: The PSDS metric.

A second important shift in this period concerned evaluation. Which we have already covered in Section 2.6.3 The event-based F1-score traditionally used in DCASE depends on a collar parameter and a single operating point, both of which were shown to bias the comparison of SED systems with different temporal resolution and class-balance behavior [36]. To address this, the Polyphonic Sound Detection Score (PSDS) [37] was adopted as the primary metric from DCASE 2020 onward: it is computed as the area under a *polyphonic* ROC curve and is therefore independent of the operating point, and its intersection-based definition of a correct detection avoids the duration-dependent strictness of collar-based F1 [37, 36]. From 2021 onward, the challenge has reported two PSDS variants PSDS_1 (temporally strict, penalizing localization errors) and PSDS_2 (audio-tagging oriented, more tolerant of localization error) which together summarize different deployment profiles of an SED system.

2022: Data Augmentation and Improved Baselines.

The DCASE 2022 baseline introduced a number of incremental improvements over previous editions, including stronger data augmentation (MixUp [38] and FilterAugment [39] in the augmentation pipeline) and refinements to the Mean Teacher training schedule [40]. The same edition saw the wider adoption of frequency-aware architectural components, which will be discussed in Section 3.3.1.

2023: Pre-Trained Audio Embeddings.

The DCASE 2023 edition incorporated pre-trained large audio models (notably BEATs) into the baseline by late-fusing their frame-level embeddings with the CRNN encoder, reflecting a broader shift in the audio community from training-from-scratch SED systems toward adapting strong self-supervised audio representations [41]. Mean Teacher was retained as the SSL backbone, with the pre-trained model kept frozen during training to limit overfitting on the small labeled set.

2024: Heterogeneous Data and Missing Labels.

The most recent edition formalizes the heterogeneity that has been implicit in earlier editions, by jointly using DESED and the MAESTRO real dataset for training and evaluation [41]. The two datasets differ in their class taxonomies, in the temporal granularity of their labels (strong vs. weak), and in their domain. The challenge therefore explicitly targets the setting where some classes are present in the data but *not annotated* in a given clip, requiring the loss to be masked over potentially missing labels. The

semi-supervised backbone, however, remains Mean Teacher combined with strong data augmentation and frozen pre-trained embeddings [41].

Across these seven editions, the trend is clear. The basic SSL recipe introduced in 2018 Mean Teacher with a CRNN backbone has not been fundamentally replaced; rather, each edition has refined the data, metrics, augmentation, and architectural components around this core, and the most successful systems have tended to be those that combine many of these refinements rather than those that propose a radically different SSL paradigm.

3.2.2 Mean Teacher for SED

The adaptation of the Mean Teacher framework [31] to SED was first demonstrated in the DCASE 2018 baseline and is most clearly described in the system of Jiakai [33], which has been carried over with only minor modifications into every subsequent DCASE Task 4 baseline. The principle is identical to that of Section 3.1.3: a student network is trained by gradient descent, and a teacher network with the same architecture is maintained as an exponential moving average of the student's weights (Equation 3.2). The teacher's predictions on unlabeled and weakly-labeled data act as soft targets that the student is regularized to match. Three SED-specific elements deserve to be made explicit, however, because they affect the design of the loss in ways that do not arise in image classification.

Two-headed output. Because the available labels come at two granularities, the CRNN backbone is given two output heads: a *strong head* that produces per-frame, per-class probabilities $\hat{y}^{(s)} \in [0,1]^{T\times C}$ via sigmoid activations, and a *weak head* that produces clip-level probabilities $\hat{y}^{(w)} \in [0,1]^{C}$ by aggregating the strong predictions across time. The aggregation operator is typically attention-weighted pooling, which is differentiable and allows the network to learn which frames are most informative for each class. The supervised loss is then a sum of (i) a binary cross-entropy on the strong head, applied only to the synthetic strongly-labeled clips, and (ii) a binary cross-entropy on the weak head, applied to both strongly-labeled and weakly-labeled clips:

$$\mathcal{L}_{\text{sup}} = \mathcal{L}_{\text{BCE}}^{\text{strong}}(\hat{y}^{(s)}, y^{(s)}) + \mathcal{L}_{\text{BCE}}^{\text{weak}}(\hat{y}^{(w)}, y^{(w)}). \tag{3.4}$$

Two consistency losses. The consistency term in Equation 3.3 is likewise applied at both output levels. Given a (possibly unlabeled) clip x_u and two independent stochastic augmentations η, η', the loss is

$$\mathcal{L}_{\text{cons}} = \text{MSE}\Big(f_{\theta}^{(s)}(x_u,\eta),\, f_{\theta'}^{(s)}(x_u,\eta')\Big) + \text{MSE}\Big(f_{\theta}^{(w)}(x_u,\eta),\, f_{\theta'}^{(w)}(x_u,\eta')\Big), \tag{3.5}$$

where $f_\theta^{(s)}, f_\theta^{(w)}$ denote the student's strong and weak heads and $f_{\theta'}$ denotes the EMA teacher. Critically, this consistency loss is computed on *all* clips in the batch strongly-labeled, weakly-labeled, and unlabeled which means that the unlabeled set, despite contributing nothing to $\mathcal{L}_{\text{sup}}$, drives most of the gradient through the consistency term and is what allows the model to generalize beyond the very small synthetic strongly-labeled subset.

Sigmoid outputs and MSE consistency. A subtle but important consequence of the multi-label nature of SED is that the model's output for each class is an independent Bernoulli probability rather than a categorical distribution over classes. The common practice [33, 34] is to apply the MSE consistency loss directly on these sigmoid outputs (rather than KL divergence on a softmax, as in image SSL); this is a natural choice when classes are mutually compatible, and it has the practical advantage of producing gradients that decay smoothly as the student approaches the teacher.

Why Mean Teacher works well for SED. The empirical robustness of Mean Teacher in SED can be attributed to the same properties that motivated it in image classification, but they happen to be especially well matched to the domain [31]. The EMA teacher produces smooth temporal targets, which are essential when the student's per-frame predictions are inherently noisier than its clip-level predictions; the weight-space averaging is a form of ensembling that compensates for the small effective size of the strongly-labeled set; and the consistency loss naturally *propagates* the available weak and strong labels into the unlabeled portion of the dataset through invariance to augmentation. For these reasons, every winning DCASE Task 4 submission since 2018 has used Mean Teacher as its SSL backbone, even as the surrounding architecture, augmentation, and pre-training have changed substantially.

3.2.3 Data Augmentation Strategies

In any consistency-based SSL framework the choice of augmentation is not a cosmetic detail: it defines the equivalence class within which the model is required to be invariant, and therefore the inductive bias that the unlabeled data are used to enforce. Three augmentation strategies have become standard in SED, in increasing order of audio-specificity: MixUp, SpecAugment, and FilterAugment.

MixUp

MixUp [38] constructs synthetic training examples by taking convex combinations of pairs of inputs and of their labels. For two clips (x_i, y_i) and (x_j, y_j) and a mixing coefficient

$\lambda \sim \mathrm{Beta}(\alpha, \alpha)$:

$$\tilde{x} = \lambda\, x_i + (1 - \lambda)\, x_j, \qquad \tilde{y} = \lambda\, y_i + (1 - \lambda)\, y_j. \tag{3.6}$$

This linearly interpolates between training samples and encourages the model to behave linearly between them, which has been shown to improve generalization and to reduce sensitivity to corrupted labels [38]. In SED, MixUp can be applied either in the waveform domain or, more commonly, in the log-mel domain, and the label interpolation $\tilde{y}$ is taken element-wise on the multi-hot label matrix. Because SED is multi-label, mixing two clips typically produces a clip with *more* simultaneous events than either source, which is precisely the kind of polyphonic scenario the model is asked to handle at test time this property makes MixUp particularly natural for SED and explains why it appears in the default augmentation pipeline of the DCASE Task 4 baseline [41].

SpecAugment

SpecAugment [42], originally proposed for end-to-end speech recognition, operates directly on the time frequency representation (typically log-mel spectrograms). It consists of three simple operations: *time warping*, which displaces a column of the spectrogram along the time axis by a random distance; *time masking*, which sets a contiguous block of t time frames to a constant value, with t drawn from $\mathrm{Uniform}(0, T_0)$; and *frequency masking*, which similarly zeros a contiguous block of f mel channels. SpecAugment's appeal lies in its simplicity it requires no audio synthesis, has no learnable parameters, and is trivial to apply on the GPU together with the strong regularization it provides: by forcing the model to predict correctly even when substantial portions of the input have been hidden, it discourages overfitting to narrow time–frequency cues and improves robustness to the partial occlusion that is common in real recordings [42]. In the SED context, SpecAugment is typically applied to the input of the student network only; the teacher receives the unaugmented (or weakly augmented) clip.

FilterAugment

FilterAugment [39] is an SED-specific augmentation designed to address a limitation of SpecAugment's frequency masking. Real-world acoustic environments rooms, microphones, and recording chains do not *remove* entire frequency bands; they apply smooth, frequency-dependent gains. Masking entire bands therefore introduces a distribution shift that the model is forced to learn to invert, but is not representative of the deployment conditions. FilterAugment instead approximates an arbitrary acoustic filter by partitioning the frequency axis into a small number of random bands and applying an independent random gain (in dB) to each band, optionally with linear interpolation between the band centers to produce a smooth equalization curve. The resulting augmentation simulates

the kind of channel and room variation that a deployed SED system actually encounters, and Nam et al. [39] reported a 6.50% absolute improvement in PSDS over frequency masking alone on DCASE 2021 Task 4. A prototype of FilterAugment was used in their third-place submission to DCASE 2021, and the technique has since been adopted into the augmentation pipelines of subsequent DCASE baselines [43].

Why augmentation matters more in SSL than in supervised SED. The augmentations described above all improve supervised SED to some extent, but their effect is amplified in the semi-supervised setting. This is because, in Equation 3.5, the augmentations η, η' are exactly what defines the consistency target: a model trained with weak augmentations is only required to be invariant to weak perturbations, and most of its capacity is therefore not constrained by the unlabeled data. Combining strong augmentations on the student side (SpecAugment, FilterAugment, MixUp) with weak or no augmentations on the teacher side, in the spirit of FixMatch [30], has emerged as the most effective recipe in recent DCASE systems [39, 41], and it forms the augmentation backbone of the systems developed later in this thesis.

3.3 Architectural Advances for Sound Event Detection

The Convolutional Recurrent Neural Network introduced in Section 2.5.3 has served as the foundational architecture for sound event detection since its proposal by Çakır et al. [**?**], and remains the backbone of the DCASE Task 4 baseline system to this day. Its strength lies in combining shift-invariant local spectro-temporal feature extraction (via the CNN front end) with long-range temporal modelling (via the bidirectional RNN), producing frame-level predictions suitable for both segment-based and event-based evaluation. However, two limitations of the baseline CRNN have motivated substantial recent research. First, the use of standard 2D convolution treats the frequency axis as translation-equivariant, which is physically inconsistent with how sound events are perceived (see Section 3.3.1). Second, purely convolutional-recurrent backbones learn only from the labelled and unlabelled audio of the target task, without leveraging the large body of weakly-labelled audio already collected at internet scale through AudioSet.

The remainder of this section examines two complementary directions that address these limitations. Section 3.3.1 covers frequency-adaptive convolution, which modifies the CNN front end to respect the non-stationary nature of the frequency axis. Section 3.3.2 covers attention- and transformer-based approaches, which either replace the CNN-RNN backbone entirely or supply the CRNN with semantically rich features pre-trained on AudioSet.

3.3.1 Frequency Dynamic Convolution for Sound Event Detection

Two-dimensional convolution has been widely adopted in deep learning-based sound event detection systems for recognizing spectro-temporal patterns in log-mel spectrograms. Originally designed for image recognition, 2D convolution enforces *translation equivariance* across both spatial dimensions of its input, meaning that the same convolutional filter is applied uniformly at every position. In computer vision, this property is desirable because the visual appearance of an object remains largely unchanged regardless of its position within an image [14, 43].

However, when applied to audio spectrograms, this uniform treatment introduces a fundamental physical inconsistency. A spectrogram is a two-dimensional representation with time on the horizontal axis and frequency on the vertical axis, but these two dimensions possess fundamentally different properties with respect to shift invariance. Along the time axis, a time-frequency pattern retains its acoustic identity when shifted hearing a dog bark one second earlier or later does not change the nature of the sound. In contrast, shifting the same pattern along the frequency axis alters its frequency components, thereby changing how the sound is perceived. A pattern that represents speech at one frequency region would sound entirely different if transposed to a significantly higher or lower frequency region [43]. Therefore, while translation equivariance is beneficial along the time axis, it is physically inconsistent along the frequency axis for audio data.

Standard 2D convolution is unable to account for this frequency-dependent nature of sound events because it applies identical learned filters across all frequency regions of the spectrogram. This limitation is particularly significant given that sound events exhibit distinctive spectro-temporal patterns across different frequency regions. Prior work on FilterAugment [39] demonstrated that frequency dependency is a critical factor in sound event detection, showing that regularizing models across varying frequency regions substantially improves detection performance.

To address this limitation, Nam et al. [43] proposed **Frequency Dynamic Convolution** (FDY-Conv), a method that replaces the fixed convolutional kernels of standard 2D convolution with *frequency-adaptive kernels* that vary according to the frequency component of the input. The central idea is to maintain translation equivariance along the time axis while releasing it along the frequency axis, thereby achieving improved physical consistency between the model and the spectro-temporal characteristics of sound events.

Frequency dynamic convolution achieves this frequency adaptivity through a mechanism based on *basis kernels* and *attention weights*. Instead of learning a single fixed kernel, the method maintains a set of K learnable basis kernels $\{(\mathbf{W}_1, b_1), (\mathbf{W}_2, b_2), \ldots, (\mathbf{W}_K, b_K)\}$. For each frequency bin f of the input, frequency-adaptive attention weights $\pi_i(f, \mathbf{x})$ are computed that determine how the basis kernels should be combined to form the optimal

kernel for that particular frequency region. The output of a frequency dynamic convolution layer is then obtained as:

$$y_i(t, f) = \mathbf{W}_i * x(t, f) + b_i \tag{3.7}$$

$$y(t, f, \mathbf{x}) = \sum_{i=1}^{K} \pi_i(f, \mathbf{x})\, y_i(t, f) \tag{3.8}$$

where t and f denote the time and frequency dimensions, $\mathbf{x}$ and y are the input and output of the layer, y_i is the output from the i-th basis kernel, and $\pi_i(f, \mathbf{x})$ is the frequency-adaptive attention weight for the i-th basis kernel at frequency f [43].

The attention weights are extracted from the input by first applying average pooling over the time axis to isolate the frequency profile, followed by two 1D convolutional layers along the channel axis with batch normalization and ReLU activation in between. The use of 1D convolution rather than fully connected layers allows the attention mechanism to consider adjacent frequency components, capturing local frequency relationships. A softmax function with a temperature parameter of $\frac{1}{31}$ is applied to ensure that the attention weights sum to one across basis kernels and that all basis kernels are learned uniformly during training [43, 44].

This design draws inspiration from two precursor methods. **Dynamic convolution** [44] enhances representation capability by adapting the convolution kernel to the entire input, producing an input-dependent kernel but without spatial specificity. **Temporal dynamic convolution** [45], developed for speaker verification, applies kernels that adapt to each time frame of the input, capturing the rapid temporal variations of phonemes in speech. Frequency dynamic convolution extends this line of work by applying kernels that adapt to each frequency bin, which is particularly suited for sound event detection where patterns are inherently frequency-dependent [43].

The frequency dynamic convolution approach is especially advantageous for detecting *non-stationary sound events* such as speech, alarm bells, cat and dog vocalizations, and dishes which exhibit intricate and rapidly changing spectro-temporal patterns across diverse frequency regions. In contrast, quasi-stationary sound events such as vacuum cleaners and blenders produce relatively simple horizontal patterns on the spectrogram that are similar across frequency regions, and thus benefit less from frequency-adaptive processing. This distinction highlights that frequency dynamic convolution enhances SED performance precisely by improving the model's ability to recognize frequency-dependent patterns that standard 2D convolution fails to capture [43].

3.3.2 Attention and Transformer-based Approaches

Although the CRNN architecture combines local feature extraction with temporal modelling, training it from scratch requires substantial labelled data for the target task. A

complementary direction in recent sound event detection research is the use of transformer-based models that are first pre-trained on large quantities of unlabelled audio and then used either as a backbone or as a frozen feature extractor for downstream SED. Two such models, BEATs and ATST-Frame, are particularly relevant to this thesis because both are central to the current DCASE Task 4 baseline and both employ teacher-student training schemes conceptually related to the Mean Teacher framework used in this work (Section 3.1.3).

BEATs. BEATs (Bidirectional Encoder representation from Audio Transformers), proposed by Chen et al. [46], introduces an iterative self-supervised pre-training scheme that does not require any AudioSet labels during the early iterations. The method jointly trains an acoustic tokeniser and a transformer encoder: the tokeniser converts audio patches into discrete tokens, and the encoder is trained to predict these tokens from masked portions of the input, in the style of masked image modelling. After each iteration, the tokeniser is replaced with a new one distilled from the current encoder, and the cycle repeats. This bootstrapping procedure produces audio representations that capture semantic content far beyond what supervised labels can provide. For sound event detection, BEATs is typically used as a frozen feature extractor whose frame-level embeddings are concatenated with the output of a CRNN backbone before the recurrent layer; this hybrid design has been the official DCASE Task 4 baseline since 2023 [**?**].

ATST-Frame. A limitation of BEATs and similar models is that they are pre-trained primarily for clip-level tasks, while sound event detection requires precise frame-level localisation. Li et al. [47] addressed this gap with the Audio Teacher-Student Transformer (ATST), which exists in two variants: ATST-Clip for clip-level pre-training and ATST-Frame for frame-level pre-training. Both variants use a teacher-student scheme inspired by BYOL (Bootstrap Your Own Latent) [48] , in which the student network is trained to match the representations produced by an exponential moving average teacher - mirroring the Mean Teacher framework discussed in Section 3.1.3, but applied to representation learning rather than semi-supervised consistency regularisation. ATST-Frame additionally integrates frame-wise data augmentation and masking, encouraging the model to learn representations that are stable to perturbations at the frame level. The resulting embeddings have been reported to outperform BEATs and other pre-trained transformers on frame-level sound event detection benchmarks [47].

4 Methodology

4.1 Public Data Sets

Table 4.1: Overview of public datasets and reasons for rejection

No.	Dataset	Description / Reason for Rejection
1	MAVD-traffic dataset[49]	Strongly labeled dataset with 21 classes and approximately 1 GB of data. Rejected due to insufficient dataset size.
2	Google AudioSet Strong [50]	Contains partially strong labels with 356 classes. Rejected due to limited accessibility, as audio data is sourced from YouTube videos with copyright restrictions.
3	DCASE 2018 Task 4 Dataset [51]	Includes 10 classes and is suitable for SED tasks. However, it is derived from AudioSet and could not be accessed due to the same restrictions.
4	GISE-51 [52]	Dataset with 51 classes and both weak and strong labels. Rejected due to the limited amount of strongly labeled data.
5	MELAUDIS Dataset [53]	Acoustic dataset for urban intelligent transport systems. Rejected due to insufficient dataset size.
6	DataSEC Dataset [54]	Environmental noise dataset of approximately 4 GB. Rejected due to insufficient size and diversity for robust model training.
7	ESC-50 Dataset [55]	Environmental sound classification dataset (2 GB). Rejected due to limited dataset size.
8	SINGA:PURA Dataset [56]	Polyphonic urban audio dataset with 40 classes (4.5 GB). Rejected due to insufficient dataset size.
9	URBAN-SED Dataset [57]	Contains 20 classes and approximately 8 GB of data. Rejected due to insufficient scale for the intended application.
10	NIGENS Dataset [58]	Strongly labeled dataset with 14 classes (2.5 GB). Rejected due to limited dataset size.
11	FSD-Mix Clips [59]	Large-scale dataset (450 GB) with 89 classes and weak labels. Rejected due to the absence of strong temporal annotations.

As suitable data set was not found we decided to generate it synthetically using 2 different data sets, namely FSD50K and TUT sound events 2017, as explained further. While collecting the data set is out of scope of this thesis.

4.1.1 FSD50K Dataset for Isolated Sound Events

The FSD50K dataset [60] is an open-source, massive-size database intended to be used in the recognition of sound events. It includes around 51,000 audio clips with total duration surpassing 100 hours. This database is based on recordings gathered from the Freesound website, which allows the user to upload audio files licensed under the Creative Commons License, thus making them open to usage in research studies. Every audio file is annotated according to the hierarchy including 200 sound classes defined by the ontology of AudioSet.

The key feature of the FSD50K dataset is that it operates under the paradigm of weak labeling. So the annotation of the audio clip includes labels for several events existing in the audio recording without any specification of the exact boundaries of those events. While this feature makes this dataset extremely relevant for tagging tasks, it brings some problems for SED since it requires determining the temporal location of the events. And a considerable number of audio recordings include overlapping sound sources.

To avoid these disadvantages a carefully selected set of FSD50K has been used in the experiments. Clips containing only one label have been picked for use in our study. This way of picking audio clips could be considered an attempt to approximate strongly labeled data, as in the case of single-label clips there should be only one sound event that is present. Even though temporal labels are not present, this method makes the dataset much clearer and more suitable for SED tasks.

From the chosen set of data, we managed to select several sound events that were labeled with only one class. Namely, such sound events as *rattle*, *squeak*, *bell*, *fart*, *hiss*, and others have been chosen. Summary in Section 4.1.1

Class	No. of Samples	Total duration (s)	Avg. duration (s)
Bell	190	867.955	4.568
Fart	527	1035.099	1.964
Hiss	63	248.939	3.951
Rattle	69	244.771	3.547
Squeak	232	748.644	3.227
Total	1,081	3145.408	2.910

Table 4.2: Per-class sample counts and durations of the isolated event set.

There are a number of benefits to utilizing FSD50K in this way. Firstly, there is an abundance of real-life sound samples to learn from and use abundant data. Secondly, because it is freely available, it removes all of the disadvantages of a proprietary dataset like AudioSet, including potential bias and other issues related to lack of access to data.

And finally, through filtering for single-label clips, it can serve as a dataset for tasks with higher label requirements. Later we will see how these clips were used in mixture generation.

It should be noted, however, that this method does not entirely substitute strong labeling, as exact times of onset and offset of events are missing in this dataset. This makes additional actions necessary to address this problem.

4.1.2 TUT Sound Events 2017, Development dataset for Background Noise

The TUT Sound Events 2017 dataset[61] has been used for modeling realistic backgrounds for noise. The TUT Sound Events 2017 dataset has been proposed by Mesaros *et al.*. This is a real life audio database comprising audio clips recorded in residential and urban environments; both indoor and outdoor. The notable feature of this dataset is its well annotated nature because each sound event within the data set is annotated with the accurate time of onset and offsets. So, it is suitable for research related to sound event detection.

The audio clips of the database have been recorded using binaural microphones and provide a highly realistic audio environment. The sounds included in this dataset include noises and acoustical events taking place in residential or urban environments, like traffic noises, human activities, and other sounds.

In our experiments the TUT Sound Events dataset was not employed as annotated datasets. It was used only as a source of noise. It means that recorded sound events were considered as unlabeled environmental noise. The reason why this was chosen is related to the necessity to expose the system to real-world noise to increase the robustness of its performance. The role of real noise should be emphasized since the complexity of sound event detection increases significantly when the system is faced with real-world background noise.

4.2 Data Generation

In this step we mixed the FSD50K isolated events and TUT Sound Events in 10 secs clips. This mixture generation was crucial for training models on generated synthetic data. There are three different types of datasets.

1. Single isolated events in one clip
2. Multiple isolated events in one clip which are non overlapping and,
3. Multiple isolated events in one clip which are overlapping

4.2.1 Synthetic Mixture Generation With Single Isolated Event

This section describes the data augmentation pipeline developed to construct the training dataset used in this work. The pipeline synthetically mixes isolated sound events into background noise, producing labelled soundscapes under controlled conditions. The following describe each stage of the pipeline in detail. Figure 4.1 gives overview of the data generation pipeline structure.

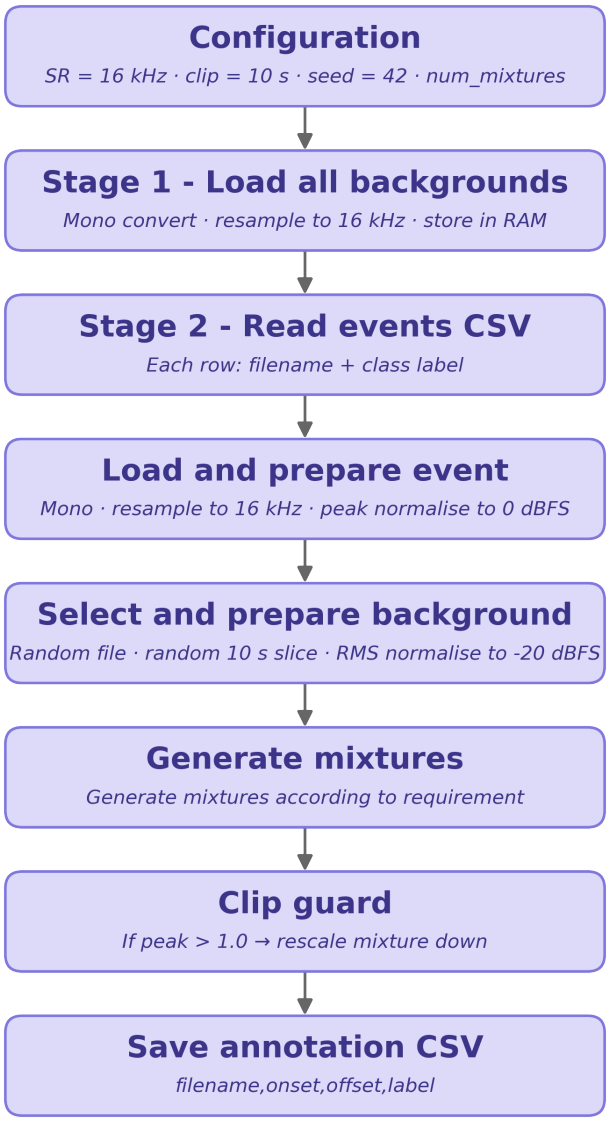


Figure 4.1: General Flowchart for Data set generation pipeline

Configuration and Design Parameters

Before any audio processing begins, the pipeline establishes a set of global parameters that govern the behaviour of the entire system.

The target sampling rate is set to 16,000 Hz. This means that every second of audio is represented by 16,000 numerical samples. All input files regardless of their original recording rate are resampled to this common rate, ensuring that the arithmetic of mixing operates in a consistent unit system throughout.

Every output mixture is fixed at exactly ten seconds in duration. This constraint is decided by refering to DCASE task 4 challenges.

The background noise level is set to a target of -20 dBFS (decibels relative to Full Scale), where dBFS is the standard unit of signal level in digital audio. This value is expressed in terms of the Root Mean Square (RMS) energy of the background signal, rather than its instantaneous peak. The RMS measure reflects the *average power* of a signal over time. A random seed of 42 is fixed for reproducibility.

Finally, the number of augmentations per isolated event is set to a variable. Which can be decided before the training.

Audio Utility Functions

The pipeline defines several audio processing functions that serve as the mathematical building blocks for all subsequent mixing operations. Each function encapsulates a single, well-defined signal processing operation. We will look into it as follows:

Mono Conversion

Audio recordings may be captured in mono (a single channel) or stereo (two channels, representing left and right). Since mixing stereo signals introduces unnecessary complexity and doubles memory consumption without adding any information relevant to this task, all audio is collapsed to a single channel before processing. This is done by discarding the second channel and retaining only the first. A more energy-preserving alternative would be to average the two channels, but the single-channel approach is a widely accepted simplification.

Peak Normalisation to 0 dBFS

Before any event recording is mixed into a background, it is normalised so that its loudest moment, its peak amplitude, sits precisely at the digital ceiling of 1.0 (equivalently, 0 dBFS). This is achieved by dividing every sample in the recording by the maximum absolute sample value across the entire waveform. The effect is to scale the entire signal up or down uniformly, preserving its internal dynamics while placing all events on a common loudness reference.

This step is crucial for consistency. Without it, a faint recording of one event class and a loud recording of another would contribute very differently to the mixture, making the relative loudness of classes an artefact of the recording conditions rather than a meaningful

acoustic property.

RMS Normalisation

While peak normalisation is appropriate for isolated events which have well-defined transient peaks, it is less suitable for continuous background noise, where the perceptually relevant quantity is the sustained average energy rather than the loudest moment. For background signals, the pipeline therefore applies RMS normalisation.

The RMS value of a discrete audio signal x of length N is defined as:

$$x_{\text{RMS}} = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x[n]^2} \tag{4.1}$$

To convert a target level from decibels to a linear amplitude value, the standard formula is applied:

$$A_{\text{linear}} = 10^{\,d_{\text{target}}/20} \tag{4.2}$$

where the denominator of 20 (rather than 10, as used for power in watts) arises because amplitude is proportional to the square root of power, introducing a factor of $\frac{1}{2}$ into the logarithm that doubles the denominator. The gain required to bring the background to the target level is then simply the ratio of the target linear amplitude to the current RMS value. A small constant of 10^{-9} is added to the denominator as a numerical guard against division by zero in the case of a silent signal.

Random Background Slicing

Since background noise files may be of arbitrary duration, a random 10-second window must be extracted from each one. The start position of this window is sampled uniformly at random. This random slicing is the primary mechanism by which each augmentation iteration receives a distinct acoustic environment, even when the same background file is selected more than once. If a background file is shorter than ten seconds, it is zero-padded (extended with silence) to reach the required length, which is a graceful fallback for short recordings.

Conversion to 16-bit PCM

All internal processing is performed in floating-point arithmetic, with sample values in the range $[-1.0, +1.0]$. The standard format for WAV audio files is 16-bit Pulse Code Modulation, which stores samples as signed integers in the range $[-32{,}767, +32{,}767]$. The value 32,767 is equal to $2^{15} - 1$, the maximum positive value representable by a signed 16-bit integer. Before saving, any floating-point values that have strayed outside the valid range are clipped to ± 1.0, and the waveform is then scaled and cast to the 16-bit integer type.

Stage 1: Loading Background Recordings

All background noise recordings are loaded from disk into memory before any mixing begins. Each file is read, converted to mono, and resampled to the target sampling rate of 16,000 Hz if necessary. Preloading all backgrounds into RAM, rather than reading from disk on demand during the mixing loop, is an important efficiency consideration.

Stage 2: Reading the Isolated Events

The second stage reads a CSV file that enumerates all isolated event recordings and their class labels. Each row in this file associates an audio filename with a semantic category such as *rattle*, *alarm*, or *fart* etc. The pipeline iterates over every row, loading and processing the corresponding audio file and generating number of mixtures.

Stage 3: The Core Mixing Procedure

The third and most involved stage implements the mixture generation loop. For each isolated event, the pipeline performs the following sequence of operations.

Event Preparation

The event audio is read from disk, converted to mono (if necessary) , resampled if necessary, and then peak-normalised to 0 dBFS. After normalisation, a temporal buffer constraint is computed. A margin of 100 milliseconds is reserved at both the start and the end of the 10-second clip (explained in the Figure 4.2). This buffer ensures that no event begins within the first 100 ms or ends within the last 100 ms of any mixture. The rationale is to prevent the model from learning to associate clip boundaries with event onsets or offsets.

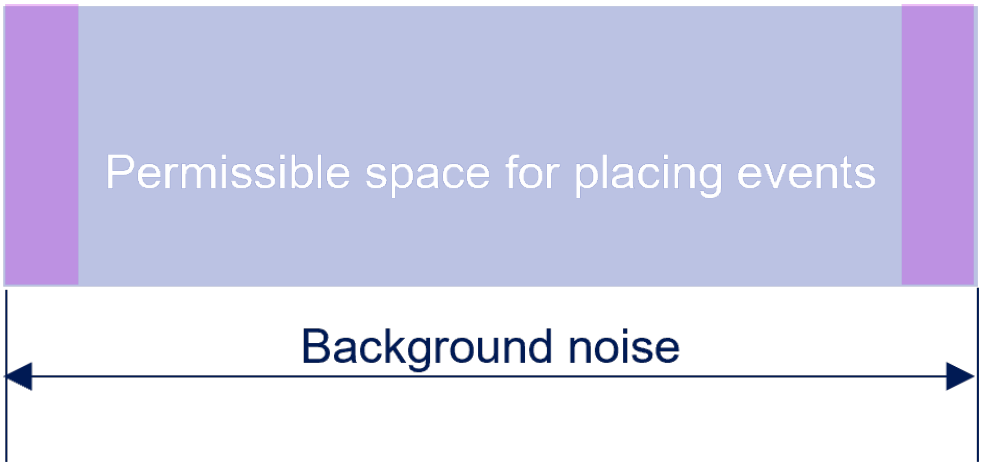


Figure 4.2: Buffers and permissible space for event placements.

If an event recording is longer than the maximum duration that can fit within the

clip while respecting these buffers, it is hard-cropped to the maximum permitted length and a warning is logged. As the isolated events were filtered with maximum duration of 9.9 secs this was just a safety feature.

The authors note this as a temporary measure; a more principled approach in future work would split such events into multiple clips rather than discarding their tails.

Augmentation Loop

For each of the augmentation iterations, a background file is selected at random from the preloaded pool, a random 10-second slice is extracted, and its RMS level is set to -20 dBFS. A random start position for the event is then drawn uniformly from the valid range that is, any position within the clip at which the event fits while respecting the 100 ms buffers on each side.

The mixture is formed by linear superposition: the event waveform is arithmetically added, sample by sample, to the corresponding region of the background waveform. This operation is physically accurate in the real world, sounds from independent sources combine additively at a microphone.

After mixing, the combined waveform may exceed the digital ceiling of ± 1.0 if the positive peaks of the event and background happen to coincide. This phenomenon, known as constructive interference, would produce clipping distortion if the waveform were saved without correction. To prevent this, the pipeline checks whether the mixture peak exceeds 1.0 and, if so, rescales the entire mixture downward so that the peak sits exactly at 1.0. This rescaling marginally alters the Signal-to-Noise Ratio (SNR) of the affected mixture but is far preferable to allowing audible distortion. Figure 4.3 illustrates the event placement example.

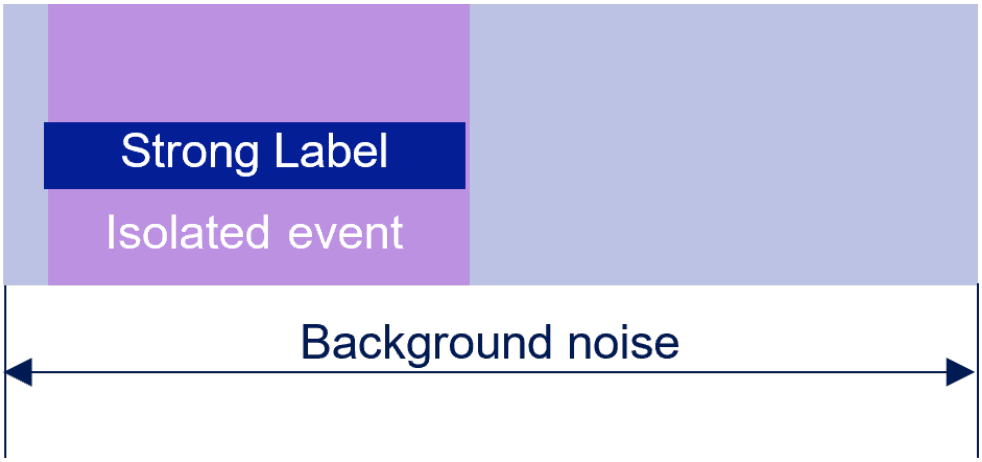


Figure 4.3: Single isolated event mixture representation

Metadata Recording

Each mixture is saved as a 16-bit PCM WAV file with a sequentially numbered

filename. Simultaneously, the precise onset and offset times of the embedded event are computed by dividing the start and end sample indices by the sampling rate, yielding timestamps in seconds accurate to four decimal places. These timestamps, together with the class label inherited from the source event, constitute the ground-truth annotation for that mixture file. The ability to record exact onset and offset times is one of the principal advantages of synthetic data generation: in a real-world recording scenario, obtaining annotations of this precision would require laborious manual work, whereas here they are a natural by-product of the mixing process.

Stage 4: Saving the Annotation File

Once all mixtures have been generated, the accumulated metadata one row per mixture file, containing the filename, onset time, offset time, and class label, is written to a single CSV file. This annotation file is directly compatible with standard Sound Event Detection evaluation frameworks, such as those used in the Detection and Classification of Acoustic Scenes and Events (DCASE) challenge, enabling straightforward integration with established evaluation workflows.Figure 4.4 and Figure 4.5 shows examples of the spectrogram with strong labels/annotations (filename, onset, offset, label)

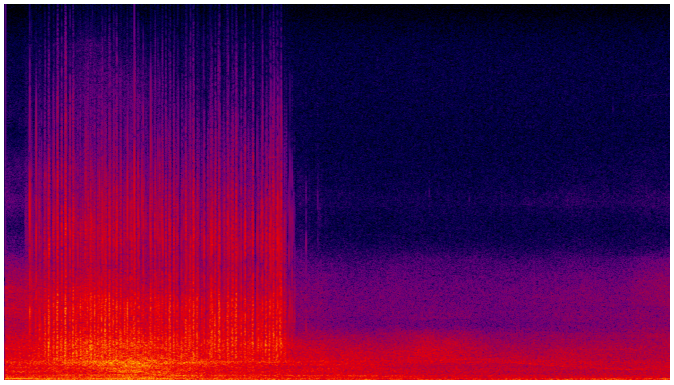


Figure 4.4: Example Audio 1 mix000004.wav,0.2771,4.9716,Rattle

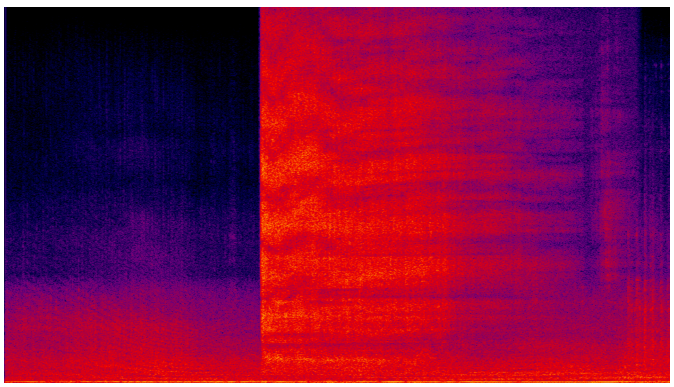


Figure 4.5: Example Audio 2 mix000300.wav,3.7916,9.5524,Hiss

4.2.2 Synthetic Mixture Generation with Multiple Non-Overlapping Events

This section describes the second and more sophisticated version of the data generation pipeline. Whereas the first pipeline places exactly one isolated sound event into each background clip, this pipeline places *multiple events* into a single ten-second clip while enforcing the constraint that no two events overlap in time. The result is a dataset that far more closely resembles real urban acoustic scenes, where several distinct sound-producing sources may be active at different moments within the same recording. Here we enter a stage near to polyphonic data. The following subsections describe every stage of the pipeline in detail.

Configuration and Design Parameters

The pipeline is governed by a set of global parameters, all defined in one place for reproducibility and ease of experimentation. Most parameters carry the same meaning as in the single-event pipeline: the target sampling rate is 16,000 Hz, each output clip is exactly ten seconds long, the background noise is normalised to a target of -20 dBFS (Root Mean Square), and a fixed random seed of 42 is used throughout. Three parameters are new and specific to the multi-event design.

`MAX_EVENTS_PER_CLIP` caps the number of events that may be placed in any single clip at five. The actual number per clip will often be lower, because the placement algorithm stops naturally when no further event fits in the remaining temporal space. The minimum is always one, meaning no clip is generated with background noise alone.

`BUFFER_SEC` and `GAP_SEC` govern the temporal layout of events within the clip. A hard silence buffer of 100 milliseconds is reserved at both the start and the end of every clip, so no event may begin in the first 100 ms or end in the last 100 ms. In addition, a mandatory gap of 500 milliseconds must separate consecutive events from each other and from the buffer boundary. These two parameters transform into sample-domain constants - `BUFFER_SAMPLES` and `GAP_SAMPLES` - by multiplying by the sampling rate, and the usable region of the clip is formally defined as the interval [`BUFFER_SAMPLES`, `CLIP_LEN_SAMPLES` − `BUFFER_SAMPLES`). Figure 4.6 illustrates the graphical representation.

Audio Utility Functions

The utility functions used in this pipeline are identical in purpose to those in the single-event pipeline: mono conversion, peak normalisation to 0 dBFS, RMS normalisation to a target dBFS level, random background slicing with zero-padding, and conversion to 16-bit PCM. The reader is referred to Section 4.2.1 for a detailed mathematical treatment of each function.

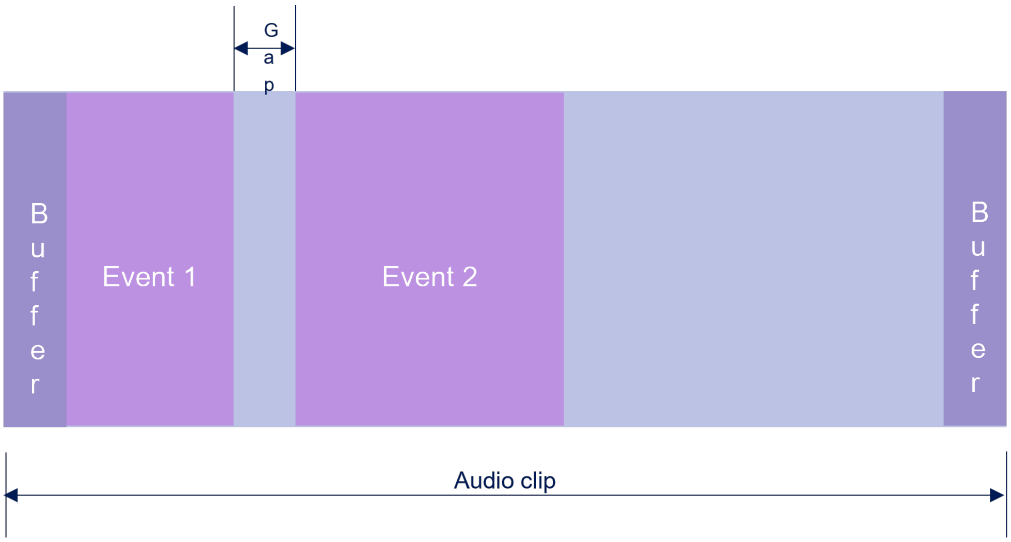


Figure 4.6: Multiple event non overlapping mixture representation

The Non-Overlapping Placement Algorithm

The central algorithmic contribution of this pipeline, and the mechanism that distinguishes it from the single-event version is this placement method. Its purpose is to determine whether a event of a given duration can be inserted into the clip without violating any of the temporal constraints, and if so, to return the sample index at which it should begin.

First Event Placement

When the occupied list is empty, the first event is always placed at a fixed position. Its start index is set to `USABLE_START` + `GAP_SAMPLES`, which is the position immediately following the left buffer and the mandatory inter-event gap. The function then checks whether the event fits before the right wall, defined as `USABLE_END` − `GAP_SAMPLES`. If it does, the fixed start index is returned; if the event is so long that it cannot fit even in an otherwise empty clip, the function returns `None` and the clip is checked for next event.

Subsequent Event Placement

For every event after the first, the function computes a candidate start position by taking the end sample of the most recently placed event and adding `GAP_SAMPLES`. This enforces the 500 ms inter-event gap. It then checks whether the candidate end position - candidate start plus event length - falls before the right wall. If it does, the candidate start is returned; if not, `None` is returned, signalling that the clip is too full to accommodate this event and checks for further event.

This design produces a strictly sequential, left-to-right packing of events with guaranteed minimum gaps between them. It is deliberately deterministic in its gap enforcement while still producing variation across clips through the *random selection of events and backgrounds*.

Isolated Events and class balance

The loaded events are then organised into a dictionary keyed by class label, so that the pool for each sound category is accessible independently. This structure supports *class-balanced sampling* during clip generation: rather than drawing events uniformly at random from the entire pool (which would favour more populous classes), the sampling strategy first selects a class uniformly at random, and then selects a random event from within that class.

Generating Mixture Clips

Sequential Event Packing

An inner loop attempts to place up to `MAX_EVENTS_PER_CLIP` events. In each iteration, the class-balanced sampling strategy is used to draw a candidate event. The function is called with the candidate event length and the current list of occupied intervals. If a valid start position is returned, the event waveform is added to the mixture by linear superposition, the interval is appended to the occupied list, and the metadata (start sample, end sample, label) is recorded. If function returns `None`, a different candidate event is drawn and retried, up to `MAX_FIT_ATTEMPTS` times. If no event fits after all attempts, the inner loop terminates early, indicating that the clip's usable space is exhausted. This adaptive termination means that the number of events per clip is not fixed clips with many long events will hold fewer total events than clips with short events - which further diversifies the dataset.

Linear Superposition

The mixing operation is identical to the first pipeline: the event waveform is arithmetically added, sample by sample, to the corresponding region of the background array. Because the placement algorithm guarantees that no two event intervals overlap, no two events contribute simultaneously to the same sample, and the mixture at any given moment contains either pure background or background plus one event.

Saving the Annotation CSV

Because each clip may contain between one and five events, the number of CSV rows will substantially exceed the number of WAV files - a clip containing three events contributes three annotation rows, all sharing the same filename but with different onset, offset, and label values. This one-to-many relationship between audio files and annotation rows is the standard format used by Sound Event Detection evaluation frameworks such as those employed in the DCASE challenge.

Design Principles and Methodological Justification

This pipeline extends the first with three additional design principles.

The first is **ecological validity**. Real-world urban soundscapes rarely contain only a single discrete event against an otherwise unoccupied background. By allowing multiple events per clip, the pipeline produces soundscapes that are structurally closer to real recordings, which should make models trained on this data more robust at inference time.

The second is **guaranteed non-overlap**. The placement algorithm enforces strict temporal separation between events, both from each other and from the clip boundaries. This is a deliberate choice to produce unambiguous ground-truth annotations: because events never overlap, the onset and offset of each event are always acoustically distinct.

The third is **class balance**. The two-stage sampling strategy - class first, then file within class - prevents the dataset from being dominated by whichever sound category happens to have the most recordings in the source. This is particularly important for training a classifier or detector that must perform equally well across all sound classes.

4.2.3 Synthetic Mixture Generation with Multiple Overlapping Events

This section describes the third and most acoustically complex variant of the data generation pipeline. It extends the non-overlapping pipeline Section 4.2.2 by permitting events from *different* classes to overlap freely in time, while still forbidding two events of the *same* class from occupying the same temporal region simultaneously. A further departure from the previous pipelines is the introduction of per-event Signal-to-Noise Ratio (SNR) control: rather than placing all events at a fixed normalised level, each event is scaled to a randomly drawn SNR relative to the background before being mixed. The utility functions, background loading procedure, event pre-loading procedure, clipping guard, PCM conversion, and CSV saving stage are identical to those described in Section 4.2.1 and are not repeated here. Figure 4.7 is the graphical representation of multiple overlapping events.

Additional Configuration Parameters

`SNR_MIN_DB` and `SNR_MAX_DB` define a per-event SNR range above the background level (SNR (Sound to Noise ratio) = EBR (Event to background)). For each event placed in a clip, an SNR value is sampled uniformly from this range, and the event is scaled so that its RMS energy sits exactly that many decibels above the background RMS. This randomisation makes the dataset acoustically diverse: the model cannot rely on a fixed loudness relationship between events and background and must learn to detect events across a range of prominence levels.

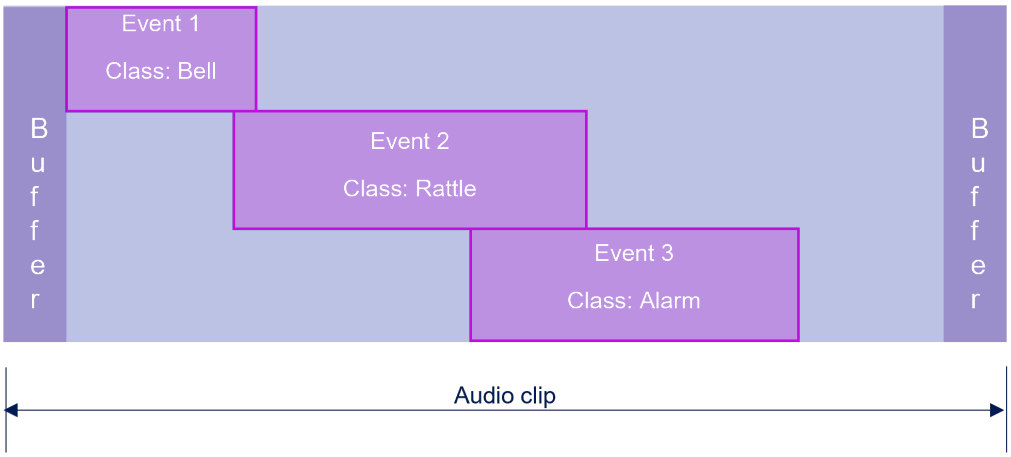


Figure 4.7: Multiple events overlapping mixture representation

Per-Event SNR Scaling

Before an event is mixed into the background, its amplitude is scaled so that its RMS level sits at a target of (`BACKGROUND_RMS_DBFS` + `SNR_dB`) dBFS, where `SNR_dB` is drawn uniformly from given range. Concretely, the target RMS in linear amplitude is

$$A_{\text{target}} = 10^{(\texttt{BACKGROUND_RMS_DBFS} + \texttt{SNR_dB})\,/\,20} \tag{4.3}$$

and the event is scaled by the ratio of this target to its current RMS, with a small numerical guard of 10^{-9} in the denominator.

The Overlapping Placement Algorithm

Overlap Policy

The fundamental rule of this pipeline is that two events of the *same* class may never present in the same clip, but two events of *different* classes may. This policy is motivated by the DCASE data set structure.

Onset Sampling

This stochastic approach contrasts with the deterministic left-to-right packing used in the non-overlapping pipeline. Here, onset positions are drawn uniformly across the full usable window, so events may appear anywhere in the clip rather than accumulating from the left edge. The result is a more uniform distribution of event onsets across time, which is desirable for training a detector that must be equally sensitive at all temporal positions.

Class-Rotation Slot Filling

If the assigned class cannot be placed - either this class is present at all in the existing events, or because the event is too long to fit - the algorithm advances to the next class

in the rotation. If no class can be placed in a given slot after a full rotation, the slot is skipped.

Annotation CSV

The anotations are identical in meaning to those described in Section 4.2.2.

Comparison with the Non-Overlapping Pipeline

Table summarizes the key differences between the three data generation pipelines.

Table 4.3: Comparison of the three synthetic mixture generation pipelines.

Property	**Single-event**	**Non-overlapping**	**Overlapping**
Events per clip	1	0 - N	0 - N
Cross-class overlap	N/A	Not allowed	Allowed
Same-class overlap	N/A	Not allowed	Not allowed
Event placement	Random	Sequential	Uniform random
SNR control	Fixed (20dbfs)	Fixed (20dbfs)	Per-event random

4.3 Supervised Model

This supervised training model is used by the company for current trainning framework using only strongly labelled data.

The supervised model is a convolutional recurrent network trained only on the strongly-labelled portion of the data, with no consistency loss and no teacher-student mechanism. It takes a 500×80 log-mel input (500 frames, 80 mel bands) and applies three convolutional blocks, each consisting of a 3×3 `Conv2D` layer with 64 filters, batch normalisation, a ReLU6 activation, frequency-wise max pooling, and dropout. Pooling is applied along the frequency axis only ($80 \rightarrow 16 \rightarrow 8 \rightarrow 4$), preserving the full temporal resolution of 500 frames required for frame-level localisation. The resulting feature map is reshaped to a 500×256 sequence and passed through two bidirectional recurrent layers (64 units each, outputs averaged across directions), followed by a time-distributed dense layer of 256 units with dropout and a final time-distributed dense layer producing five per-frame class activities. The output is trained with binary cross-entropy under the ACCDOA [62] formulation.

The network has 258,565 total parameters (258,181 trainable), roughly 1/50th the size of the 13.211 M-parameter FDY-CRNN used in the semi-supervised experiments.

4.4 Model and Training Framework for Semi-supervised model

Base Architecture

The model used in this work is the Frequency Dynamic Convolutional Recurrent Neural Network (FDY-CRNN) proposed by Nam et al. [43], built on the CRNN architecture described in Section 2.5.3. The implementation is based on the publicly available FDY-SED reference repository, which we adapt to our synthetic dataset by modifying the data loading, class taxonomy, and evaluation configuration. The model architecture itself is unchanged.

The CRNN backbone consists of seven convolutional blocks followed by a bidirectional Gated Recurrent Unit and a class-wise attention pooling layer. Each convolutional block uses a 3×3 kernel with padding 1 and stride 1. The filter counts across the seven blocks are $[32, 64, 128, 256, 256, 256, 256]$, and frequency-wise max pooling is applied after each block with sizes $[(2,2), (2,2), (1,2), (1,2), (1,2), (1,2), (1,2)]$. The first dimension corresponds to time and the second to frequency; this configuration reduces the frequency axis aggressively while preserving full temporal resolution, which is necessary for frame-level event localisation (see Section 2.5.3). Dropout with rate 0.5 is applied within each convolutional block, and a context-gating activation is used in place of ReLU.

Frequency dynamic convolution (Section 3.3.1) is applied to convolutional blocks 2 through 7; the first block uses standard 2D convolution. The number of basis kernels is 5 for the three-class experiments and 4 for the five-class experiments, with the softmax temperature fixed at 31 as in the original FDY paper [43].

The recurrent module is a two-layer bidirectional GRU with 256 hidden units per direction, with no recurrent dropout. The final classification layer uses class-wise attention pooling rather than time pooling.

Acoustic Features

All audio is processed at a sampling rate of 16,000 Hz. Log-mel spectrograms are computed with 128 mel bands spanning 0 - 8000 Hz, a window length of 2048 samples, and a hop length of 256 samples, following the configuration of the FDY-SED reference implementation. Each input clip is fixed at 10 seconds in duration, matching the synthetic mixture length described in Section 4.2. Normalisation is performed per-instance using min-max scaling along the time and channel dimensions.

Mean Teacher Framework Configuration

The training framework follows the Mean Teacher paradigm [31], in which a student model is trained by gradient descent while a teacher model is maintained as an exponential moving average (EMA) of the student weights. The EMA decay factor is set to 0.999.

Training proceeds, with the first 25% epochs used as an exponential warm-up period during which the consistency loss weight is ramped up smoothly from zero to its maximum value. The optimiser is Adam with a learning rate of 0.001.

Data Augmentation

Three augmentation strategies are applied during training. Mixup [63] is applied to every training batch (rate 1.0) using the soft variant, in which the mixing ratio is propagated to the labels. Time masking is applied with a mask length sampled between 1/20 and 1/5 of the total time frames. FilterAugment [39] is applied with 2–5 frequency bands, a minimum bandwidth of 4 mel bins, and a per-band gain sampled uniformly from $[-4.5, 6]$ dB using the step-filter variant. Independent augmentation realisations are applied to the student and teacher inputs.

Dataset Splits

The synthetic mixtures generated by the pipelines described in Section Section 4.2 are split into five disjoint subsets: strongly labelled training , strongly labelled validation , weakly labelled training, unlabelled training, and held-out test. The weak labels are derived from the strong labels by retaining only the set of event classes present in each clip and discarding all temporal information.

Implementation and Reproducibility

Experiments are run with a fixed random seed of 42. The codebase is implemented in PyTorch and adapted from the FDY-SED reference repository, with modifications to the data loading and configuration files to handle the synthetic mixtures and reduced class taxonomies used in this work.

4.5 ATST-SED: Fine-tuning a Pre-trained Audio Transformer for SED

The semi-supervised model described in learns only from the labelled and unlabelled audio of the target task. A complementary strategy is to import knowledge from a large audio transformer pre-trained on a far larger external corpus, and to adapt it to the target task. ATST-SED, the system of Shao et al. [64], follows exactly this route: it fine-tunes the

pre-trained ATST-Frame model [47] for sound event detection within a Mean Teacher framework. This section describes how the model works.

The Pre-trained Backbone: ATST-Frame

ATST-Frame [47], introduced in Section 3.3.2, is a Transformer encoder pre-trained self-supervisedly on the unbalanced AudioSet ($\sim$ 1.9M clips) with no labels. It is trained under a BYOL-style teacher–student scheme in which a student network predicts the frame-level representations produced by an exponential moving average (EMA) teacher; the difficulty of this pretext task is controlled by applying data augmentation (Mixup and frequency warping) to one branch and BERT-style frame masking to the student. Crucially, ATST-Frame adopts a *frame-wise* rather than patch-wise tokenisation: the 64-bin log-mel input (16 kHz, 64 ms window, 10 ms hop, 60–7800 Hz) is grouped four frames at a time, yielding a frame-level representation at 40 ms resolution. This fine temporal resolution is what makes the resulting embeddings well suited to SED, where the precise onset and offset of events must be recovered, and it is the property that ATST-SED exploits.

Hybrid Architecture for SED

ATST-SED is built on the DCASE 2023 Task 4 baseline, which late-fuses the frame-level embeddings of a frozen pre-trained transformer (originally BEATs [46]) with the feature sequence of a CRNN encoder before the recurrent layer [41]. ATST-SED replaces the frozen pre-trained branch with ATST-Frame and, decisively, makes it trainable. The two streams process the same input in parallel: the CRNN extracts shift-invariant local spectro-temporal features, while ATST-Frame supplies semantically rich frame-level embeddings. The two sequences are merged and passed through the recurrent layer and a class-wise attention-pooling head, producing a strong (per-frame) output and a weak (clip-level) output, exactly as in the baseline two-headed design.

The model is trained semi-supervisedly with the Mean Teacher framework [31]: a supervised binary cross-entropy loss on the strong and weak heads of the labelled clips, plus a consistency loss between student and EMA-teacher predictions computed over all clips, including the unlabelled ones.

Two-Stage Fine-tuning Strategy

The central difficulty ATST-SED addresses is that ATST-Frame is a large model ($\sim$ 86M parameters) whereas the strongly-labelled SED data is small; naively fine-tuning the whole network end-to-end overfits and destroys the pre-trained representation. ATST-SED therefore fine-tunes in two stages [64].

Stage 1 (frozen backbone). The pre-trained ATST-Frame is held frozen and only the newly initialised parts of the network — the CRNN branch, the feature-merging module, and the classification heads are trained under Mean Teacher. This adapts the randomly initialised components to operate on the fixed pre-trained embeddings without disturbing them.

Stage 2 (joint fine-tuning). The ATST-Frame backbone is then unfrozen and the entire network is fine-tuned jointly, again under Mean Teacher, using a substantially smaller learning rate for the pre-trained transformer than for the CRNN. This allows the pre-trained representation to be specialised toward the target sound classes while avoiding the catastrophic overfitting that single-stage fine-tuning would incur on the limited strongly-labelled set.

The fine-tuning is sensitive to effective batch size: the authors report that reducing the batch size, or emulating a larger one through gradient accumulation, degrades performance because of its effect on the batch normalisation statistics of the CRNN branch [64]. On the DCASE Task 4 development set, the fully fine-tuned model attains PSDS1 0.583 and PSDS2 0.810.

Relevance to This Work

ATST-SED is directly relevant to the central question of this thesis. The FDY-CRNN Mean Teacher system can draw only on in-domain data and, as the results of show, the unlabelled portion does not compensate for limited strong labels. ATST-SED injects supervision-free knowledge acquired from a far larger external corpus through the pre-trained ATST-Frame backbone, offering a second, orthogonal route to the strong-label scarcity problem: where the Mean Teacher recipe exploits unlabelled *target* audio, ATST-SED additionally exploits unlabelled *external* audio captured in the pre-trained weights.

5 Experimental Setup

This chapter describes the experimental protocol used to evaluate the FDY-CRNN with Mean Teacher framework on synthetic sound event detection data. Fourteen experiments are organised into five groups, each varying a single axis of the dataset total size, partitioning, class count, or mixture type while holding the model and training configuration fixed. Section 5.1 defines the shared configuration. Section 5.2 describes the hardware environment. Sections 5.3–5.3 describe the five experiment groups.

5.1 Common Configuration

The FDY-CRNN configuration follows Section 4.4. The number of output classes C varies by experiment group ($C = 3$ or $C = 5$); all other model parameters are constant. $K = 5$ frequency-adaptive basis kernels are used per FDY-Conv layer, yielding 13.211 M total trainable parameters. Training proceeds for 100 epochs with a 25-epoch exponential warm-up of the consistency loss, maximum consistency weight $w_{\text{cons,max}} = 0.5$, weak-classification weight $w_{\text{weak}} = 0.1$, weak-consistency weight $w_{\text{weak,cons}} = 1$, Adam optimiser at learning rate 0.001, and EMA teacher decay 0.999.

Augmentation

Three augmentation strategies are applied with independent realisations to student and teacher inputs: soft Mixup at rate 1.0; FilterAugment with 2–5 frequency bands and per-band gains drawn uniformly from $[-4.5, +6]$ dB; and time masking covering between 1/20 and 1/5 of the clip duration. Frequency masking and additive noise are disabled.

Reproducibility

The random seed is fixed at 42. The CUDA backend is not configured for full determinism, so repeated runs differ only in non-deterministic GPU floating-point ordering.

5.2 Hardware

Experiments were conducted on two workstations, both equipped with NVIDIA professional-grade GPUs based on the Ampere architecture. The choice between machines depended on availability; the specific machine used for each experiment is noted in the corresponding experiment table.

Server I: NVIDIA RTX A5000

A workstation hosting a single NVIDIA RTX A5000 GPU. The A5000 has 8,192 CUDA cores, 256 third-generation Tensor cores, and 64 second-generation RT cores, delivering approximately 27.8 TFLOPS of single-precision (FP32) performance. The card has 24 GB of GDDR6 memory on a 384-bit bus, providing 768 GB/s peak memory bandwidth, at a thermal design power (TDP) of 230 W. The software environment is CUDA 13.2 and PyTorch 2.2.2 on Ubuntu Linux.

Server II: NVIDIA RTX A6000

A workstation hosting a single NVIDIA RTX A6000 GPU. The A6000 has 10,752 CUDA cores, 336 third-generation Tensor cores, and 84 second-generation RT cores, delivering approximately 38.7 TFLOPS of FP32 performance. The card has 48 GB of GDDR6 memory on a 384-bit bus, providing 768 GB/s peak memory bandwidth, at a TDP of 300 W (observed peak draw $\approx$ 288 W during training). The software environment is CUDA 13.0 with NVIDIA driver 580.105.08 on Linux. Server II's doubled memory capacity was the primary motivation for migrating to it for the larger five-class and multi-event experiments.

5.3 Data Setups

Group A: Three-Class Single-Event Mixtures

The first experiment group studies the effect of dataset size and partitioning on three-class single-event detection. All five experiments in this group use the same three classes (*Rattle*, *Hiss*, *Bell*) and the single-event mixture pipeline described in Section 4.2.1. Each clip is 10 seconds long and contains one isolated FSD50K event placed against a TUT Sound Events 2017 background normalised to -20 dBFS. Weak labels are derived from strong labels by retaining only the set of event classes present in each clip. Experiments E1–E3 vary the total dataset size while preserving a (strong/val/weak/unlabelled/test) 45/5/15/30/5 partitioning; experiments E4 and E5 fix the total at 8,050 mixtures and vary the proportion of unlabelled training data. (n) being number of trainning runs.

Group B: Five-Class Single-Event Mixtures

Group B extends the single-event task to five classes by adding *Fart* and *Squeak* to the taxonomy. All other mixture generation parameters are identical to Group A. Two experiments are reported: E6 retains the 45/5/15/30/5 partitioning of Group A, while E7 increases both the total dataset size and the proportion of unlabelled training data.

Table 5.1: Group A: three-class single-event experiments. The partitioning column gives strong/val/weak/unlabelled/test percentages of the total.

ID	Configuration	Files	Partitioning (%)	n
E1	Baseline	4,832	45/5/15/30/5	7
E2	Larger dataset	6,440	45/5/15/30/5	6
E3	Largest dataset	10,273	45/5/15/30/5	7
E4	Increased unlabelled share	8,050	35/5/15/40/5	4
E5	Strong-labelled-heavy split	8,050	40/5/15/35/5	4

Table 5.2: Group B: five-class single-event experiments.

ID	Configuration	Files	Partitioning (%)	n
E6	Baseline split	4,000	45/5/15/30/5	7
E7	Unlabelled-heavy	6,000	30/5/15/45/5	7

Group C: Five-Class Non-Overlapping Multi-Event Mixtures

Group C transitions from single-event to multi-event mixtures while keeping the five-class taxonomy. The mixtures are generated using the non-overlapping pipeline described in Section 4.2.2: each clip is 10 seconds long and contains between 1 and 5 events (~ 3 on average), none of which overlap in time. The partitioning is fixed at 30/5/15/45/5 across the group, allocating 45% of clips to unlabelled training. This partitioning differs from Group A's 45/5/15/30/5 scheme: relative to Group A, 15 percentage points are reallocated from strongly-labelled training to unlabelled training.

Table 5.3: Group C: five-class multi-event non-overlapping experiments.

ID	Configuration	Files	Partitioning (%)	n
E8	Small	2,000	30/5/15/45/5	4
E9	Medium	4,000	30/5/15/45/5	4
E10	Large	8,000	30/5/15/45/5	2

Group D: Five-Class Overlapping Multi-Event Mixtures

Group D introduces two methodological refinements relative to Groups A–C, both intended to strengthen the validity of the reported numbers.

First, the test set is constructed from a reference pool of isolated source events held out from training. The available FSD50K events for each class are split 90/10 at the source level into a training-source pool and a test-source pool; mixtures used during training are generated only from the training-source pool, and mixtures used for evaluation are

generated only from the test-source pool. This source-level partition ensures that no isolated event present in training is encountered again at evaluation time, eliminating one form of train–test leakage that was possible in the earlier experiments.

Second, each event is mixed into the background at a per-event Signal-to-Noise Ratio drawn uniformly from $[5, 20]$ dB above the background, introducing acoustic prominence variability that the model must learn to handle.

Mixtures are generated using the overlapping pipeline described in Section ??: each clip is 10 seconds long and contains between 1 and 5 events (~ 3 on average). Events of the same class are not permitted to co-occur within a single clip; events of different classes may overlap freely. The partitioning reserves 10% of the total clips for the held-out test set; of the remaining 90%, 40% is allocated to strongly-labelled training, 15% to weakly-labelled training, and 45% to unlabelled training. The strongly-labelled validation set is drawn as 20% of the strongly-labelled training subset.

Table 5.4: Group D: five-class multi-event overlapping experiments. Validation column gives size as a fraction of the strongly-labelled training subset.

ID	Configuration	Files	Partitioning (%)	n
E11	Medium	12,000	40/(20 of train)/15/45/10	6
E12	Large	18,000	40/(20 of train)/15/45/10	6

Group E: Five-Class Overlapping Multi-Event Mixtures with Increased Unlabelled Proportion

Group E repeats the Group D protocol with a higher unlabelled training proportion. The source-level 90/10 train/test split, the overlapping mixture pipeline, and the per-event SNR range of $[5, 20]$ dB are unchanged from Group D. The variable under investigation is the proportion of unlabelled training clips, which is increased from 45% to 55% at the expense of strongly-labelled training. The partitioning of the training pool is therefore 30% strong, 15% weak, 55% unlabelled, with the validation set drawn as 20% of the strongly-labelled training subset.

Table 5.5: Group E: five-class multi-event overlapping experiments with increased unlabelled training proportion. Partitioning is expressed relative to the training pool (the 90% of clips remaining after the held-out test set is removed).

ID	Configuration	Files (total)	Partitioning of training pool (%)	n
E13	Large	18,000	40/(20 of strong)/15/45	7
E14	Medium	12,000	40/(20 of strong)/15/45	7

5.4 Setup for supervised learning

The supervised model is evaluated on the same dataset as the matched semi-supervised experiment E12 Section 5.3: 18,000 five-class overlapping multi-event mixtures generated with the pipeline of Section 4.2.3, per-event SNR drawn uniformly from $[5, 20]$ dB, and the source-level 90/10 train/test split. The partitioning is identical, reserving 10% for the held-out test set and, of the remaining clips, 40% strongly-labelled, 15% weakly-labelled, and 45% unlabelled.

The crucial difference is the supervision used. Whereas the semi-supervised system exploits all three streams, the supervised is trained on the strongly-labelled training subset alone; the weak and unlabelled streams (60% of the training pool) are discarded. The comparison therefore isolates a single question: whether the semi-supervised use of weak and unlabelled data recovers the performance attainable from strong labels alone.

The model is trained for up to 200 epochs with a batch size of 64, the Adam optimiser (learning rate 3.32×10^{-4}, AMSGrad, EMA momentum 0.926), and a learning-rate scheduler. Evaluation is event-based with a ± 50 ms onset/offset tolerance, a minimum event length and maximum gap of 5 frames, and a posterior threshold tuned on the validation set. One validation-threshold selection rules were considered (best-F1 score); the results below use the F1-optimal threshold, and the other two criteria yield a test-set overall F1 within 0.2 points. All numbers are from a single run.

5.5 Setup for ATST-SED

The ATST-SED model (Section 4.5) is evaluated on exactly the same data as the E12 experiment, so that the only variable relative to the FDY-CRNN semi-supervised system is the model itself. The dataset is the 18,000-mixture five-class overlapping set generated with the pipeline of Section 4.2.3, with the source-level 90/10 train/test split. Of the training pool, 40% is strongly-labelled (with 20% of that strongly-labelled subset held out for validation), 15% is weakly-labelled, and 45% is unlabelled; 10% of the total clips form the held-out test set.

The model is initialised from the ATST-Frame checkpoint pre-trained on AudioSet-2M and fine-tuned with the two-stage strategy described in Section 4.5: a frozen-backbone stage followed by a joint fine-tuning stage, both under the Mean Teacher framework. The acoustic front-end follows ATST-Frame (64 mel bands, 16 kHz, 40 ms frame resolution) rather than the 128-band front-end of the FDY-CRNN; this difference, together with the external pre-training, should be borne in mind when interpreting the comparison.

5.6 Event-to-Background Ratio (EBR) Test Sweep

This experiment probes the robustness of a trained system to event prominence at inference time. Rather than training a separate model per condition, a single semi-supervised model is trained once on the E12 configuration (Section 5.3) - 18,000 five-class overlapping mixtures generated with the pipeline of Section 4.2.3, per-event training SNR drawn uniformly from $[5, 20]$ dB following the synthetic soundscape-synthesis convention of DESED [5] - and then evaluated, without any retraining, on a family of test sets that differ only in event prominence.

Each test set fixes the Event-to-Background Ratio (EBR) to a single constant value, defined as the RMS energy of each event relative to the RMS energy of the -20 dBFS background, in decibels. Nine test sets are generated, spanning

$$\text{EBR} \in \{25,\ 20,\ 15,\ 10,\ 5,\ 0,\ -5,\ -10,\ -15\}\ \text{dB},$$

from events well above the background down to events well below it. The upper values (5–20 dB) lie within the prominence range seen during training, whereas EBR ≤ 0 dB places events at or below the background level, a regime the model never encountered in training; the sweep therefore measures both in-range robustness and out-of-distribution extrapolation to low-prominence events.

6 Results & Discussion

This chapter reports the test-set performance of the experiments described in Chapter 5. The semi-supervised experiments are organised into five groups (A–E) and presented group-by-group, with cross-experiment inferences drawn at the end of each group; the supervised baseline (Section 6.2), the fine-tuned ATST-SED system (Section 6.3), a cross-model comparison (Section 6.4), and an Event-to-Background Ratio (EBR) robustness sweep (Section 6.5) follow in the remaining sections. Six metrics are used throughout: PSDS1 and PSDS2 (Section 2.6.3), event-based F1 with macro and micro averaging (Section 2.6.2), and segment-based F1 with macro and micro averaging (Section 2.6.1). Event-based macro F1 is treated as the primary metric in this work, as it directly measures the system's ability to localise discrete sound event instances while weighting each class equally.

For the Mean Teacher experiments of Groups A–E, the teacher is the default reported model and student results are included in the tables for completeness; all values are reported as mean ± standard deviation across the repeated runs of each experiment, with the number of runs n indicated in each table caption. The supervised baseline, the ATST-SED model, and the EBR sweep are single-run evaluations and are reported as point values. Where a trend across dataset size, partitioning, or mixture type is discussed, an accompanying figure summarises the relevant teacher metrics.

6.1 Results on Semi-Supervised method FDY-SED model

6.1.1 Group A: Three-Class Single-Event

E1: 4,832 mixtures, 45/5/15/30/5

Table 6.1: E1 test results on three-class single-event mixtures (*Rattle*, *Hiss*, *Bell*) with 4,832 clips and 45/5/15/30/5 partitioning. Mean ± standard deviation over $n = 7$ runs.

Metric	Student	Teacher
PSDS1	0.443 ± 0.131	0.583 ± 0.092
PSDS2	0.594 ± 0.255	0.832 ± 0.052
Event F1 (macro)	0.388 ± 0.041	0.445 ± 0.062
Event F1 (micro)	0.375 ± 0.041	0.417 ± 0.078
Segment F1 (macro)	0.773 ± 0.078	0.850 ± 0.026
Segment F1 (micro)	0.805 ± 0.051	0.859 ± 0.025

E2: 6,440 mixtures, 45/5/15/30/5

Table 6.2: E2 test results on three-class single-event mixtures with 6,440 clips. Mean ± standard deviation over $n = 6$ runs.

Metric	Student	Teacher
PSDS1	0.711 ± 0.108	$\mathbf{0.875 \pm 0.029}$
PSDS2	0.922 ± 0.078	$\mathbf{0.991 \pm 0.005}$
Event F1 (macro)	0.558 ± 0.073	$\mathbf{0.691 \pm 0.032}$
Event F1 (micro)	0.484 ± 0.080	$\mathbf{0.659 \pm 0.033}$
Segment F1 (macro)	0.892 ± 0.038	$\mathbf{0.947 \pm 0.005}$
Segment F1 (micro)	0.878 ± 0.041	$\mathbf{0.944 \pm 0.006}$

E3: 10,273 mixtures, 45/5/15/30/5

Table 6.3: E3 test results on three-class single-event mixtures with 10,273 clips. Mean ± standard deviation over $n = 7$ runs.

Metric	Student	Teacher
PSDS1	0.679 ± 0.063	0.784 ± 0.027
PSDS2	0.773 ± 0.075	0.913 ± 0.025
Event F1 (macro)	0.488 ± 0.047	0.552 ± 0.049
Event F1 (micro)	0.465 ± 0.064	0.546 ± 0.046
Segment F1 (macro)	0.865 ± 0.029	0.916 ± 0.010
Segment F1 (micro)	0.883 ± 0.018	0.922 ± 0.009

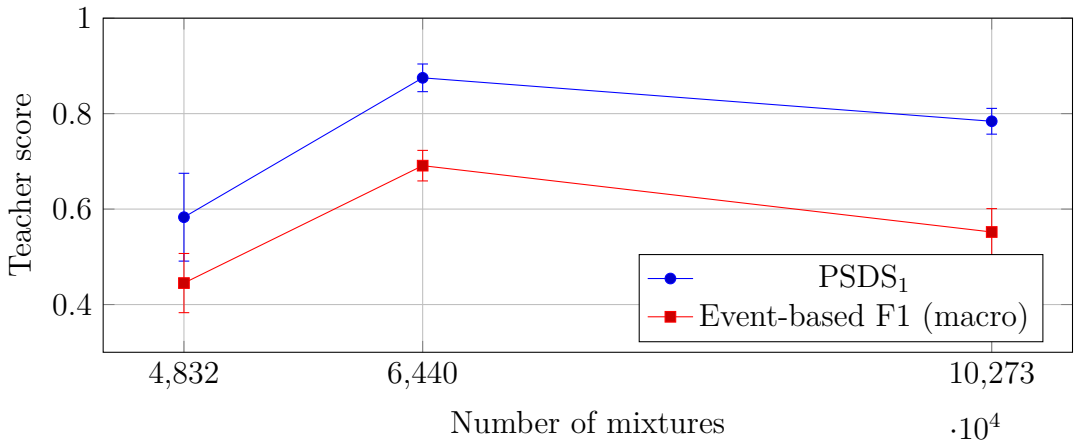


Figure 6.1: Group A: teacher PSDS_1 and event-based macro F1 versus dataset size at fixed 45/5/15/30/5 partitioning (E1–E3); both peak at 6,440 mixtures. Error bars show ± one standard deviation.

Effect of dataset size at fixed partitioning (E1, E2, E3). At the 45/5/15/30/5 partitioning held across E1, E2, and E3, the relationship between dataset size and event-based macro F1 is non-monotonic. Performance rises sharply from 0.445 at 4,832 mixtures (E1) to **0.691** at 6,440 mixtures (E2), and then falls back to 0.552 at 10,273 mixtures (E3). PSDS1 follows the same pattern: 0.583 → **0.875** → 0.784 as shown in Figure 6.1. This can be beacuse at fixed partitioning the unlabelled pool also grows, and under fixed consistency-weight and EMA hyperparameters a larger unlabelled set doesn't necessarily help and can hurt the accuracy.

E4: 8,050 mixtures, 35/5/15/40/5

Table 6.4: E4 test results on three-class single-event mixtures with 8,050 clips and partitioning 35/5/15/40/5. Mean ± standard deviation over $n = 4$ runs.

Metric	Student	Teacher
PSDS1	0.282 ± 0.140	0.502 ± 0.146
PSDS2	0.301 ± 0.180	0.576 ± 0.262
Event F1 (macro)	0.304 ± 0.060	0.418 ± 0.046
Event F1 (micro)	0.270 ± 0.069	0.383 ± 0.066
Segment F1 (macro)	0.706 ± 0.066	0.819 ± 0.049
Segment F1 (micro)	0.760 ± 0.045	0.835 ± 0.041

E5: 8,050 mixtures, 40/5/15/35/5

Table 6.5: E5 test results on three-class single-event mixtures with 8,050 clips and partitioning 40/5/15/35/5. Mean ± standard deviation over $n = 4$ runs.

Metric	Student	Teacher
PSDS1	0.544 ± 0.075	0.662 ± 0.090
PSDS2	0.706 ± 0.196	0.828 ± 0.135
Event F1 (macro)	0.408 ± 0.023	0.481 ± 0.048
Event F1 (micro)	0.386 ± 0.022	0.464 ± 0.038
Segment F1 (macro)	0.840 ± 0.032	0.877 ± 0.032
Segment F1 (micro)	0.856 ± 0.025	0.886 ± 0.027

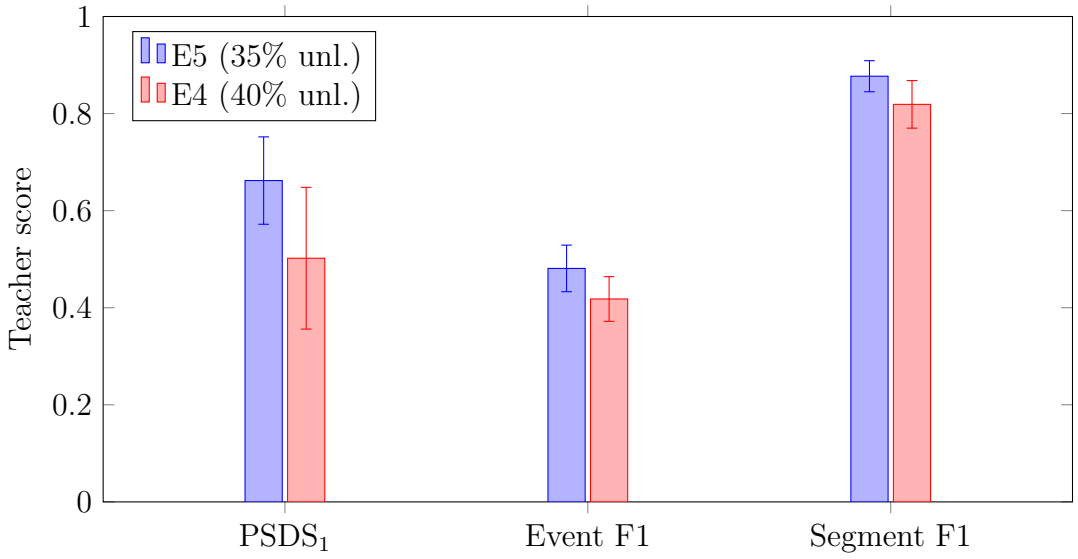


Figure 6.2: Group A: effect of partitioning at fixed 8,050 mixtures. Raising the unlabelled share from 35% (E5) to 40% (E4) lowers every teacher metric. Error bars show ± one standard deviation.

Effect of partitioning at fixed dataset size (E4, E5). At 8,050 mixtures, increasing the unlabelled share from 35% (E5) to 40% (E4) at the expense of strongly-labelled training substantially degrades every metric. Teacher event-based macro F1 drops from 0.481 in E5 to 0.418 in E4; PSDS1 drops from 0.662 to 0.502 as illustrated in Figure 6.2 . The strong-label-heavy split is clearly preferable at this dataset size, indicating that within the three-class single-event regime additional unlabelled clips do not compensate for a reduction in strongly-labelled training data.

6.1.2 Group B: Five-Class Single-Event

E6: 4,000 mixtures, 45/5/15/30/5

Table 6.6: E6 test results on five-class single-event mixtures with 4,000 clips and partitioning 45/5/15/30/5. Mean ± standard deviation over $n = 7$ runs.

Metric	Student	Teacher
PSDS1	0.282 ± 0.060	0.284 ± 0.049
PSDS2	0.337 ± 0.082	0.319 ± 0.060
Event F1 (macro)	0.194 ± 0.041	$\mathbf{0.237 \pm 0.058}$
Event F1 (micro)	0.253 ± 0.036	0.299 ± 0.044
Segment F1 (macro)	0.580 ± 0.030	0.550 ± 0.041
Segment F1 (micro)	0.647 ± 0.036	0.650 ± 0.030

E7: 6,000 mixtures, 30/5/15/45/5

Table 6.7: E7 test results on five-class single-event mixtures with 6,000 clips and partitioning 30/5/15/45/5. Mean ± standard deviation over $n = 7$ runs.

Metric	Student	Teacher
PSDS1	0.335 ± 0.066	$\mathbf{0.421 \pm 0.022}$
PSDS2	0.506 ± 0.069	$\mathbf{0.624 \pm 0.029}$
Event F1 (macro)	0.186 ± 0.027	0.231 ± 0.033
Event F1 (micro)	0.235 ± 0.027	0.284 ± 0.024
Segment F1 (macro)	0.643 ± 0.054	$\mathbf{0.721 \pm 0.017}$
Segment F1 (micro)	0.673 ± 0.052	$\mathbf{0.730 \pm 0.011}$

Effect of class count (Group A versus Group B). Increasing the number of classes from three to five at otherwise similar conditions sharply reduces event-based performance. The closest comparable pair is E1 (three-class, 4,832 clips, 45/5/15/30/5) and E6 (five-class, 4,000 clips, same partitioning): teacher event-based macro F1 falls from 0.445 to 0.237, and PSDS1 falls from 0.583 to 0.284. Segment-based metrics are also reduced but less dramatically (segment macro F1 from 0.850 to 0.550) as shown in Figure 6.3 . The drop reflects the increased difficulty of discriminating five classes from a similar amount of training data per class.

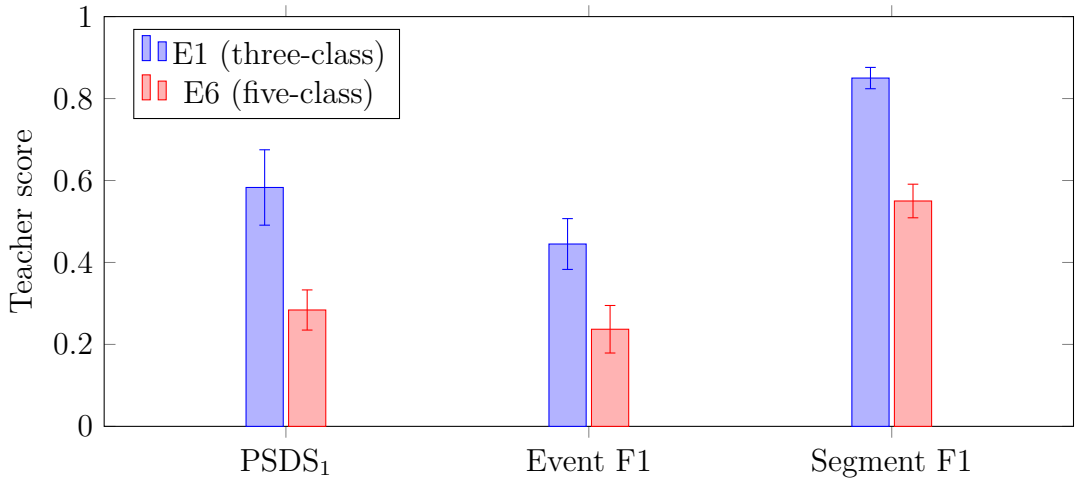


Figure 6.3: Effect of class count at matched 45/5/15/30/5 partitioning and comparable dataset size: three-class E1 (4,832 clips) versus five-class E6 (4,000 clips). Every teacher metric drops with five classes, most sharply on the event-based metrics. Error bars show ± one standard deviation.

Effect of size and partitioning together (E6 versus E7). E6 and E7 differ on both axes: E7 has 50% more clips and a 30/5/15/45/5 split instead of 45/5/15/30/5. PSDS1 and PSDS2 rise clearly from E6 to E7 (PSDS1: $0.284 \rightarrow \mathbf{0.421}$; PSDS2: $0.319 \rightarrow \mathbf{0.624}$), and so does segment-based macro F1 ($0.550 \rightarrow \mathbf{0.721}$). Event-based macro F1 remains essentially unchanged ($0.237 \rightarrow 0.231$). The combined effect of more clips and a larger unlabelled share is therefore positive on coarse-grained metrics but neutral on the strictest event-level metric in this five-class single-event regime.

6.1.3 Group C: Five-Class Non-Overlapping Multi-Event

E8: 2,000 mixtures

Table 6.8: E8 test results on five-class non-overlapping multi-event mixtures with 2,000 clips. Mean ± standard deviation over $n = 4$ runs.

Metric	Student	Teacher
PSDS1	0.240 ± 0.034	0.320 ± 0.018
PSDS2	0.372 ± 0.057	0.443 ± 0.048
Event F1 (macro)	0.377 ± 0.066	0.429 ± 0.015
Event F1 (micro)	0.375 ± 0.058	0.417 ± 0.016
Segment F1 (macro)	0.636 ± 0.071	0.707 ± 0.026
Segment F1 (micro)	0.665 ± 0.057	0.722 ± 0.022

E9: 4,000 mixtures

Table 6.9: E9 test results on five-class non-overlapping multi-event mixtures with 4,000 clips. Mean ± standard deviation over $n = 4$ runs.

Metric	Student	Teacher
PSDS1	0.234 ± 0.069	0.374 ± 0.038
PSDS2	0.325 ± 0.123	0.531 ± 0.092
Event F1 (macro)	0.387 ± 0.061	0.468 ± 0.016
Event F1 (micro)	0.376 ± 0.071	0.464 ± 0.017
Segment F1 (macro)	0.643 ± 0.119	0.768 ± 0.020
Segment F1 (micro)	0.644 ± 0.121	0.768 ± 0.021

E10: 8,000 mixtures

Table 6.10: E10 test results on five-class non-overlapping multi-event mixtures with 8,000 clips. Mean ± standard deviation over $n = 2$ runs.

Metric	Student	Teacher
PSDS1	0.559 ± 0.140	$\mathbf{0.495 \pm 0.211}$
PSDS2	0.746 ± 0.105	$\mathbf{0.603 \pm 0.299}$
Event F1 (macro)	0.610 ± 0.064	$\mathbf{0.577 \pm 0.103}$
Event F1 (micro)	0.607 ± 0.065	$\mathbf{0.574 \pm 0.104}$
Segment F1 (macro)	0.863 ± 0.059	$\mathbf{0.833 \pm 0.080}$
Segment F1 (micro)	0.864 ± 0.057	$\mathbf{0.833 \pm 0.080}$

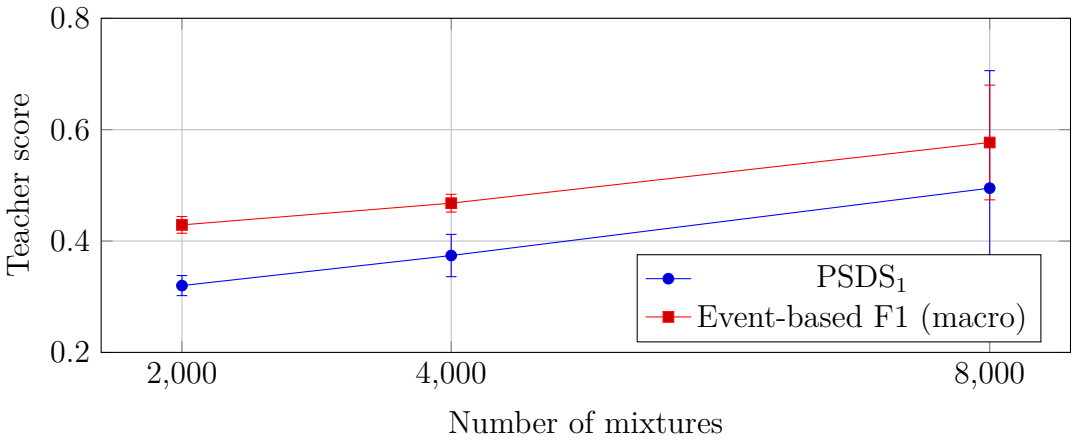


Figure 6.4: Group C: monotonic improvement in teacher PSDS_1 and event-based macro F1 with dataset size at fixed 30/5/15/45/5 partitioning (E8–E10). Error bars show ± one standard deviation.

Effect of dataset size at fixed partitioning. At the 30/5/15/45/5 partitioning held across E8–E10, performance rises monotonically with dataset size on every metric. Teacher event-based macro F1 increases from 0.429 at 2,000 clips (E8) to 0.468 at 4,000 (E9) to **0.577** at 8,000 (E10). PSDS1 rises from $0.320 \rightarrow 0.374 \rightarrow 0.495$ as illustrated in Figure 6.4 . This is a different pattern from the non-monotonic Group A result and indicates that, for multi-event mixtures with this partitioning, the training data was the binding constraint within the sizes tested.

Effect of mixture type (Group B versus Group C). The closest comparable pair across mixture types is E6 (single-event, 4,000 clips) and E9 (non-overlapping multi-event, 4,000 clips), with the caveat that the partitioning also differs (45/5/15/30/5 vs 30/5/15/45/5).

Teacher event-based macro F1 rises from 0.237 (E6) to 0.468 (E9), and segment-based macro F1 rises from 0.550 to 0.768. Multi-event mixtures supply many more labelled event instances per clip than single-event mixtures, and the gain in event-based detection performance is consistent with the model receiving a richer supervisory signal per training clip.

6.1.4 Group D: Five-Class Overlapping Multi-Event

E11: 12,000 mixtures

Table 6.11: E11 test results on five-class overlapping multi-event mixtures with 12,000 clips. Mean ± standard deviation over $n = 6$ runs.

Metric	Student	Teacher
PSDS1	0.224 ± 0.022	0.239 ± 0.022
PSDS2	0.286 ± 0.052	0.284 ± 0.030
Event F1 (macro)	0.305 ± 0.010	0.343 ± 0.018
Event F1 (micro)	0.304 ± 0.008	0.342 ± 0.017
Segment F1 (macro)	0.782 ± 0.005	$\mathbf{0.800 \pm 0.002}$
Segment F1 (micro)	0.781 ± 0.004	$\mathbf{0.801 \pm 0.004}$

E12: 18,000 mixtures

Table 6.12: E12 test results on five-class overlapping multi-event mixtures with 18,000 clips. Mean ± standard deviation over $n = 6$ runs.

Metric	Student	Teacher
PSDS1	0.189 ± 0.022	0.227 ± 0.014
PSDS2	0.272 ± 0.054	$\mathbf{0.329 \pm 0.044}$
Event F1 (macro)	0.338 ± 0.027	$\mathbf{0.358 \pm 0.018}$
Event F1 (micro)	0.335 ± 0.026	$\mathbf{0.355 \pm 0.016}$
Segment F1 (macro)	0.775 ± 0.011	0.793 ± 0.010
Segment F1 (micro)	0.773 ± 0.010	0.793 ± 0.010

Effect of dataset size at fixed partitioning. Moving from 12,000 to 18,000 clips at the same 40/15/45/10 training-pool partitioning improves event-based macro F1 from 0.343 (E11) to **0.358** (E12), and PSDS2 from 0.284 to **0.329**. Segment-based metrics are essentially unchanged across the two experiments (0.800 vs 0.793 on macro, 0.801 vs 0.793 on

micro). Within this group, additional clips help event-level localisation but not frame-level classification.

Effect of mixture type and held-out test set (Group C versus Group D). Group D introduces two simultaneous changes relative to Group C: overlapping events instead of non-overlapping, and a source-level 90/10 held-out test partition. Direct comparison is therefore confounded, but the magnitude of the gap is informative. E10 (non-overlapping, 8,000 clips) reports student event-based macro F1 of 0.610, while E11 (overlapping, 12,000 clips) reports teacher event-based macro F1 of 0.343 on a stricter, source-disjoint test set. The combination of overlapping events and the leakage-controlled test set yields substantially lower absolute event scores, suggesting the earlier non-overlapping numbers were inflated by source-level leakage at evaluation.

6.1.5 Group E: Overlapping Multi-Event, Increased Unlabelled Proportion

E13: 18,000 mixtures

Table 6.13: E13 test results on five-class overlapping multi-event mixtures with 18,000 clips and a 55% unlabelled training share. Mean ± standard deviation over $n = 7$ runs.

Metric	Student	Teacher
PSDS1	0.161 ± 0.062	0.203 ± 0.047
PSDS2	0.198 ± 0.111	0.274 ± 0.069
Event F1 (macro)	0.282 ± 0.028	$\mathbf{0.336 \pm 0.024}$
Event F1 (micro)	0.280 ± 0.029	$\mathbf{0.333 \pm 0.024}$
Segment F1 (macro)	0.764 ± 0.024	$\mathbf{0.785 \pm 0.012}$
Segment F1 (micro)	0.764 ± 0.027	$\mathbf{0.787 \pm 0.013}$

E14: 12,000 mixtures

Table 6.14: E14 test results on five-class overlapping multi-event mixtures with 12,000 clips and a 55% unlabelled training share. Mean ± standard deviation over $n = 7$ runs.

Metric	Student	Teacher
PSDS1	0.172 ± 0.040	0.179 ± 0.046
PSDS2	0.224 ± 0.064	0.216 ± 0.046
Event F1 (macro)	0.303 ± 0.026	0.314 ± 0.024
Event F1 (micro)	0.299 ± 0.028	0.311 ± 0.025
Segment F1 (macro)	0.756 ± 0.022	0.764 ± 0.013
Segment F1 (micro)	0.755 ± 0.025	0.762 ± 0.014

Effect of dataset size at fixed partitioning. Within Group E, the 18,000-clip experiment (E13) outperforms the 12,000-clip experiment (E14) on every metric. Teacher event-based macro F1 rises from 0.314 to **0.336**; segment-based macro F1 rises from 0.764 to 0.785. The size effect is in the expected direction and consistent with Group D.

Effect of increasing the unlabelled share (Group D versus Group E). The cleanest comparison across these two groups is at fixed dataset size:

Table 6.15: Effect of raising the unlabelled training share from 45% (Group D) to 55% (Group E) on teacher metrics at matched total dataset size.

	12,000 clips		18,000 clips	
Metric	E11 (45% unl.)	E14 (55% unl.)	E12 (45% unl.)	E13 (55% unl.)
PSDS1	0.239	0.179	0.227	0.203
Event F1 (macro)	0.343	0.314	0.358	0.336
Segment F1 (macro)	0.800	0.764	0.793	0.785

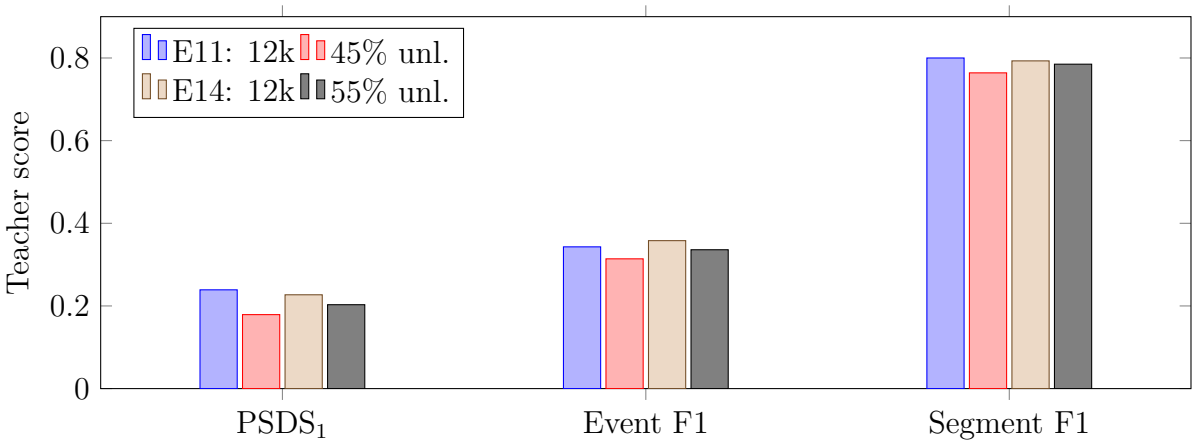


Figure 6.5: Groups D vs E: raising the unlabelled training share from 45% to 55% lowers every teacher metric at both 12,000 and 18,000 mixtures. Values from Table 6.15.

At both 12,000 and 18,000 clips, raising the unlabelled share from 45% to 55% reduces every reported teacher metric. The event-based macro F1 drop is −0.029 at 12,000 clips and −0.022 at 18,000 clips; PSDS1 drops by −0.060 and −0.024 respectively. The result is consistent with the Group A finding from E4 versus E5: within the regimes tested in this work, replacing strongly-labelled training data with additional unlabelled data degrades performance.

6.2 Supervised method with strong labeled Overlapping events

Table 6.16 reports the overall event-based F1 of the supervised model at the F1-optimal validation threshold, aggregated across the five classes. This overall figure corresponds closely to the per-class macro average (the metric treated as primary in this work).

Table 6.16: Supervised strong-label baseline: overall event-based F1 (%) by split, at the F1-optimal validation threshold. Single run on the strong labelled 40% part of 18,000-mixture five-class overlapping dataset.

	Test
Event F1 (overall)	0.571

A plausible explanation for this result lies in the composition of the strongly-labelled partition itself. The supervised model is trained on only the 40% strong-label portion of the 18,000-mixture dataset, yet this portion is large enough that every source event recurs many times across its clips, so the strong supervision alone provides sufficient coverage of each class to reach an overall event-based F1 of **0.571**. Under these conditions the unlabelled pool carries little additional information: the events it contains are already well represented among the strong labels. This is consistent with the Group A (E4 vs

E5) and Group D vs E findings, where enlarging the unlabelled share at the expense of strong labels lowered every metric. When ample strongly-labelled data with repeated event occurrences is available, reallocating part of it to the unlabelled pool removes supervision the model was using and is not compensated by the consistency signal, reducing the overall performance of the semi-supervised system relative to the supervised baseline.

6.3 ATST-SED Results

Table 6.17 reports the test-set performance of the fine-tuned ATST-SED model on the E12 configuration.

Table 6.17: ATST-SED test results on the 18,000-mixture five-class overlapping dataset (E12 configuration).

Metric	ATST-SED
PSDS1	0.367
Event F1 (macro)	0.492
Segment F1 (macro)	0.810

On the same data and split, ATST-SED outperforms the FDY-CRNN semi-supervised system (E12, Section 6.1.4) on every metric. Event-based macro F1 rises from 0.358 to **0.492** and PSDS1 from 0.227 to **0.367**, while segment-based macro F1 improves more modestly, from 0.793 to 0.810. The large gains on the event-based and PSDS1 metrics - both of which reward precise temporal localisation - indicate that the frame-level representations learned by ATST-Frame on large-scale external audio supply localisation information that the in-domain Mean Teacher system cannot recover on its own. The smaller segment-level gain reflects that frame-level presence detection was already near-saturated for both models.

6.4 Cross-Model Comparison

Table 6.18 compares the three models trained and evaluated on the identical E12 data: the FDY-CRNN semi-supervised system (E12 teacher), ATST-SED, and the supervised strong-label baseline (Section 6.2).

Table 6.18: Comparison of the three models on the identical 18,000-mixture five-class overlapping dataset (E12 configuration). Best value in each column in bold. PSDS1 and segment-based F1 were not available for the supervised baseline.

Model	PSDS1	Event F1 (macro)	Segment F1 (macro)
FDY-SED (E12)	0.227	0.358	0.793
ATST-SED	**0.367**	0.492	**0.810**
Supervised	–	**0.571**	–

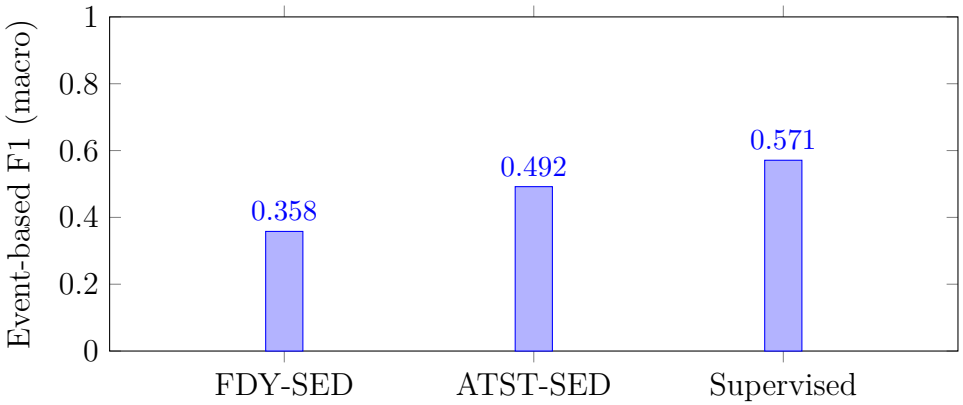


Figure 6.6: Event-based macro F1 on the identical 18,000-mixture five-class overlapping dataset (E12 configuration). ATST-SED narrows the gap between the FDY-CRNN semi-supervised system and the fully-supervised baseline.

Two conclusions follow. First, among the semi-supervised systems, ATST-SED is unambiguously stronger: it improves event-based macro F1 by 0.134 and PSDS1 by 0.140 over FDY-SED on identical data, demonstrating that importing knowledge from large-scale external pre-training is far more effective than relying on in-domain unlabelled data alone. Second, ATST-SED substantially narrows the gap to the fully-supervised baseline: it recovers roughly 63% of the event-based F1 deficit left by FDY-SED ($0.358 \rightarrow 0.492$, against the supervised 0.571) and reaches about 86% of the supervised event-based F1 while using only 40% of the training pool as strong labels. The supervised model remains the strongest on event-based F1, but the margin over ATST-SED is small relative to the strong-label saving, which is the central practical finding of this comparison.

This comparison is indicative rather than fully controlled: the three models differ in architecture and acoustic front-end, the supervised and ATST-SED figures are from single runs, and PSDS1 and segment-based F1 were not computed for the supervised baseline.

6.5 Event-to-Background Ratio (EBR) Test Sweep Results

Table 6.19 and Figure 6.7 report the performance of the fixed E12 model evaluated across the nine EBR test sets of Section 5.6. All three metrics decline as the EBR decreases, confirming that detection becomes harder as events grow quieter relative to the background.

Table 6.19: Performance of the fixed E12 semi-supervised model evaluated on test sets of decreasing Event-to-Background Ratio.

EBR (dB)	Event F1 (macro)	PSDS1	Segment F1 (macro)
25	0.367	0.281	0.790
20	0.369	0.263	0.783
15	0.349	0.223	0.777
10	0.327	0.171	0.760
5	0.288	0.118	0.739
0	0.253	0.079	0.710
-5	0.196	0.047	0.669
-10	0.124	0.019	0.586
-15	0.079	0.007	0.487

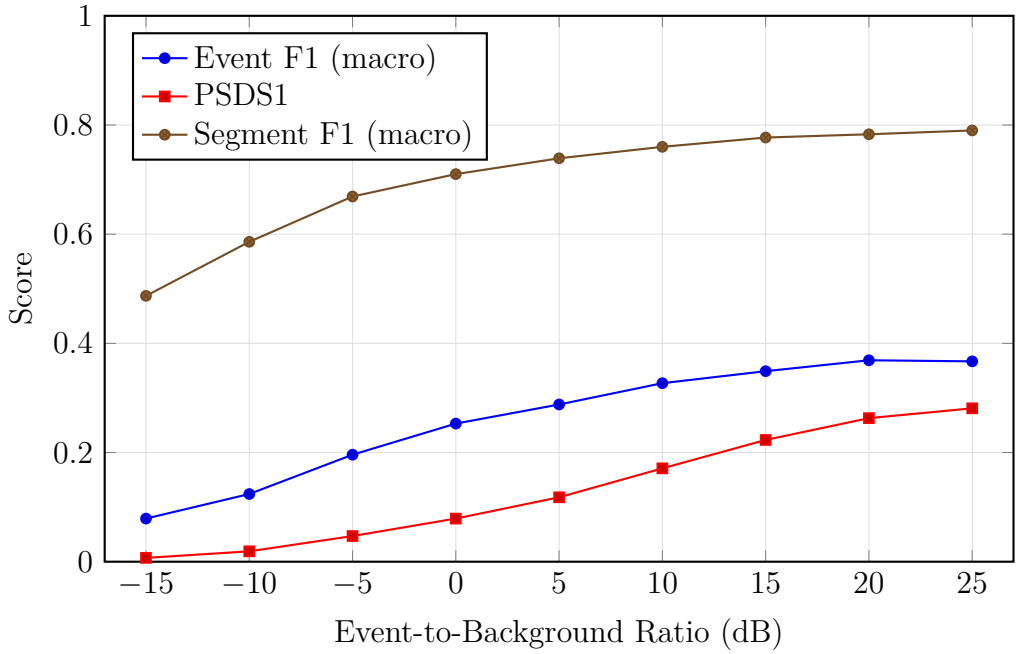


Figure 6.7: Detection performance of the fixed E12 model as a function of Event-to-Background Ratio. All metrics degrade monotonically as events become quieter relative to the background; PSDS1 collapses most steeply, while segment-based F1 is the most resilient.

Three observations follow. First, event-based macro F1 is essentially flat across the top of the range (0.369 at 20 dB, 0.367 at 25 dB) and then falls steadily, reaching 0.079 at −15 dB - a relative drop of roughly 79% from its peak. The plateau coincides with the

$[5, 20]$ dB prominence range used during training, and the decline accelerates once the EBR drops below this range, indicating that the model extrapolates poorly to events quieter than any it was trained on.

Second, PSDS1 is the most sensitive metric, falling from 0.281 at 25 dB to 0.007 at -15 dB. Because PSDS1 rewards precise temporal localisation, this collapse shows that accurate onset/offset estimation is the first capability lost as events recede into the background.

Third, segment-based macro F1 is the most resilient, retaining 0.487 even at -15 dB. Frame-level presence detection therefore survives at low prominence long after instance-level localisation has broken down: the model can still tell that *something* is active in a frame even when it can no longer delimit the event as a discrete instance.

7 Conclusion and Future Work

7.1 Conclusion

This thesis asked whether semi-supervised learning (SSL) can compensate for limited strongly-labelled data in sound event detection (SED) on synthetically generated audio. The investigation used an FDY-CRNN [43] trained within the Mean Teacher framework [?] and evaluated under the DESED / DCASE Task 4 protocol [?, 41], across fourteen experiments (Groups A–E) that varied dataset size, label partitioning, class count, and mixture type, with a supervised strong-label baseline, a fine-tuned ATST-SED system, and an Event-to-Background Ratio (EBR) robustness sweep for reference.

The central finding is negative. Within the regimes tested, reallocating strongly-labelled training data to the unlabelled pool degraded performance rather than improving it, and the semi-supervised system did not match the supervised baseline trained on strong labels alone. Two independent experiment pairs establish this directly: in Group A, raising the unlabelled share from 35% (E5) to 40% (E4) at fixed 8,050 mixtures lowered every metric (teacher event-based macro F1 $0.481 \rightarrow 0.418$); and across Groups D and E, raising it from 45% to 55% reduced every reported metric at both 12,000 and 18,000 mixtures (Section 6.1.5). The supervised baseline, trained on only the 40% strong-label portion of the 18,000-mixture five-class overlapping dataset, reached an overall event-based F1 of 0.571 (Section 6.2), above the matched semi-supervised system (E12 teacher, 0.358). When the strongly-labelled partition is large enough that each source event recurs many times, the unlabelled pool carries little additional information, and the Mean Teacher consistency signal does not recover the supervision lost by shrinking the strong-label set.

The remaining axes refine rather than overturn this conclusion. Dataset size did not act uniformly: Group A (three-class single-event) was non-monotonic, peaking at 6,440 mixtures (Section 6.1.1), whereas Group C (five-class non-overlapping multi-event) improved monotonically with size (Section 6.1.3). The non-monotonic behaviour is consistent with a fixed consistency weight and EMA decay: at fixed partitioning the unlabelled pool grows with the dataset, and a larger unlabelled set under fixed hyperparameters did not reliably help. Increasing the class count from three to five sharply reduced event-based performance at matched conditions (E1 vs E6, teacher event-based macro F1 $0.445 \rightarrow 0.237$), while moving from single-event to multi-event mixtures raised it, consistent with the richer per-clip supervisory signal that multi-event clips provide. The Group D source-disjoint test partition lowered absolute event-based scores relative to the earlier non-overlapping experiments, indicating that part of the earlier performance was attributable to source-level train–test leakage at evaluation.

The two reference systems frame the practical reading of these results. ATST-SED [?], a fine-tuned ATST-Frame transformer [47] pre-trained on large-scale external audio, outperformed the FDY-CRNN semi-supervised system on the identical E12 data on every metric (event-based macro F1 $0.358 \rightarrow 0.492$, PSDS_1 $0.227 \rightarrow 0.367$) and recovered roughly 86% of the supervised event-based F1 while using only 40% of the training pool as strong labels (Section 6.4). Importing knowledge from external pre-training was therefore far more effective than exploiting in-domain unlabelled data through consistency training. Finally, the EBR sweep (Section 6.5) showed that a fixed trained model degrades monotonically as events become quieter relative to the background, with PSDS_1 collapsing first and segment-based F1 the most resilient, and with a plateau confined to the $[5, 20]$ dB prominence range seen during training evidence that the system extrapolates poorly to prominence levels it never encountered.

Taken together, the results map where in-domain SSL helps and where it does not for synthetic SED. In this setup the binding constraint was the quantity and coverage of strong labels, not the availability of unlabelled audio; trading the former for the latter was not a favourable exchange. This is a legitimate null result for the in-domain Mean Teacher approach and a positive result for transfer from external pre-training, and it delineates the conditions under which each route is preferable.

7.2 Future Work

Several directions follow from these findings.

Stronger use of unlabelled data Mean Teacher consistency is one of the weaker mechanisms for exploiting unlabelled audio. Confidence-thresholded pseudo-labelling, curriculum or self-training schedules, and hybrid objectives that combine pseudo-labels with consistency are natural next steps for extracting more signal from the unlabelled stream before concluding that it is uninformative in a given regime.

Combining external pre-training with in-domain SSL The ATST-SED comparison showed external pre-training to be the more effective route. The two strategies are complementary: initialising from a pre-trained frame-level backbone and then applying the semi-supervised pipeline, rather than choosing one or the other may let the unlabelled pool contribute on top of transferred representations. Quantifying whether in-domain unlabelled data adds anything once a strong pre-trained backbone is in place is an open question this work could not answer with a single fine-tuned system.

Leakage-controlled evaluation throughout The Group D source-disjoint test partition lowered absolute scores and suggested earlier numbers were inflated by source-level leakage. Applying the same 90/10 source-level split to the single-event and non-overlapping groups would make cross-group comparisons clean and confirm how much of the earlier single-event and non-overlapping performance survives leakage control.

Transfer to real recordings All experiments used synthetically generated soundscapes. Evaluating the same systems on real DESED recordings, and measuring the synthetic-to-real gap, would establish how far the conclusions drawn here on synthetic data carry over to the conditions the DCASE Task 4 benchmark ultimately targets.

Bibliography

[1] Annamaria Mesaros, Toni Heittola, Tuomas Virtanen, and Mark D. Plumbley. Sound event detection: A tutorial. *IEEE Signal Processing Magazine*, 38(5):67–83, September 2021.

[2] Tuomas Virtanen, {Mark D.} Plumbley, and Dan Ellis, editors. *Computational analysis of sound scenes and events*. Springer, 2018.

[3] `https://www.researchgate.net/figure/Conversion-from-waveform-to-log-Mel-spectrogram-The-waveform-left-shows-amplitude-over_fig1_396787302`, 2023.

[4] Emre Çakır, Giambattista Parascandolo, Toni Heittola, Heikki Huttunen, and Tuomas Virtanen. Convolutional recurrent neural networks for polyphonic sound event detection. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 25(6):1291–1303, 2017.

[5] Nicolas Turpault, Romain Serizel, Ankit Parag Shah, and Justin Salamon. Sound event detection in domestic environments with weakly labeled data and soundscape synthesis, 2019.

[6] Yun Wang, Juncheng Li, and Florian Metze. A comparison of five multiple instance learning pooling functions for sound event detection with weak labeling. In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 31–35. IEEE, 2019.

[7] Annamaria Mesaros, Toni Heittola, and Tuomas Virtanen. Metrics for polyphonic sound event detection. *Applied Sciences*, 6(6), 2016.

[8] Lu JiaKai. Mean teacher convolution system for dcase 2018 task 4. Technical report, Detection and Classification of Acoustic Scenes and Events Challenge, 2018.

[9] Alan V. Oppenheim and Alan S. Willsky. *Signals and Systems*. Prentice Hall, 1999.

[10] Brian et al. McFee. librosa: Audio and music signal analysis in python, 2015. Python library for audio processing.

[11] Aalto University. *Introduction to Speech Processing*. Aalto University, 2023. Online open-access book.

[12] Toni Heittola, Annamaria Mesaros, Antti Eronen, and Tuomas Virtanen. Context-dependent sound event detection. volume 2013, pages 1–13. Springer, 2013.

[13] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[14] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[15] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[16] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

[17] Sharath Adavanne and Tuomas Virtanen. Sound event detection using spatial features and convolutional recurrent neural network. In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 771–775. IEEE, 2017.

[18] Wenjie Wei, Florian Fraisse, and Gaurav Bhatt. Sound event detection via dilated convolutional recurrent neural networks. In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 736–740. IEEE, 2019.

[19] Janek Ebbers, Reinhold Haeb-Umbach, and Romain Serizel. Post-processing independent evaluation of sound event detection systems. pages 21–22, 2023.

[20] Çağlar Bilen, Giacomo Ferroni, Francesco Tuveri, Juan Azcarreta, and Saša Krstulović. A framework for the robust evaluation of sound event detection. pages 61–65, 2020.

[21] Janek Ebbers, Reinhold Haeb-Umbach, and Romain Serizel. Threshold independent evaluation of sound event detection scores. pages 1021–1025, 2022.

[22] Giacomo Ferroni, Nicolas Turpault, Juan Azcarreta, Francesco Tuveri, Romain Serizel, Çağlar Bilen, and Saša Krstulović. Improving sound event detection metrics: insights from dcase 2020. pages 631–635, 2021.

[23] Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien, editors. *Semi-Supervised Learning*. MIT Press, Cambridge, MA, 2006.

[24] Jesper E. van Engelen and Holger H. Hoos. A survey on semi-supervised learning. *Machine Learning*, 109(2):373–440, 2020.

[25] Mehdi Sajjadi, Mehran Javanmardi, and Tolga Tasdizen. Regularization with stochastic transformations and perturbations for deep semi-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.

[26] Samuli Laine and Timo Aila. Temporal ensembling for semi-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2017.

[27] Takeru Miyato, Shin-ichi Maeda, Masanori Koyama, and Shin Ishii. Virtual adversarial training: A regularization method for supervised and semi-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(8):1979–1993, 2019.

[28] Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshop on Challenges in Representation Learning*, 2013.

[29] Eric Arazo, Diego Ortego, Paul Albert, Noel E. O'Connor, and Kevin McGuinness. Pseudo-labeling and confirmation bias in deep semi-supervised learning. In *International Joint Conference on Neural Networks (IJCNN)*, 2020.

[30] Kihyuk Sohn, David Berthelot, Chun-Liang Li, Zizhao Zhang, Nicholas Carlini, Ekin D. Cubuk, Alex Kurakin, Han Zhang, and Colin Raffel. FixMatch: Simplifying semi-supervised learning with consistency and confidence. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

[31] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results, 2017.

[32] Romain Serizel, Nicolas Turpault, Hamid Eghbal-Zadeh, and Ankit Parag Shah. Large-scale weakly labeled semi-supervised sound event detection in domestic environments. In *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2018 Workshop (DCASE2018)*, pages 19–23, 2018.

[33] Lu Jiakai. Mean teacher convolution system for DCASE 2018 task 4. Technical report, DCASE2018 Challenge, 2018.

[34] Nicolas Turpault, Romain Serizel, Ankit Parag Shah, and Justin Salamon. Sound event detection in domestic environments with weakly labeled data and soundscape synthesis. In *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2019 Workshop (DCASE2019)*, New York City, NY, USA, 2019.

[35] Romain Serizel, Nicolas Turpault, Ankit Shah, and Justin Salamon. Sound event detection in synthetic domestic environments. In *ICASSP 2020 – 45th International Conference on Acoustics, Speech, and Signal Processing*, Barcelona, Spain, 2020.

[36] Giacomo Ferroni, Nicolas Turpault, Juan Azcarreta, Francesco Tuveri, Romain Serizel, Çağdaş Bilen, and Sacha Krstulović. Improving sound event detection metrics: Insights from DCASE 2020. In *ICASSP 2021 – 2021 IEEE International Conference on Acoustics, Speech and Signal Processing*, 2021.

[37] Çağdaş Bilen, Giacomo Ferroni, Francesco Tuveri, Juan Azcarreta, and Sacha Krstulović. A framework for the robust evaluation of sound event detection. In *ICASSP 2020 – 2020 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 61–65, 2020.

[38] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

[39] Hyeonuk Nam, Seon-Hee Kim, and Yong-Hwa Park. FilterAugment: An acoustic environmental data augmentation method. In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4308–4312. IEEE, 2022.

[40] Francesca Ronchini, Romain Serizel, Nicolas Turpault, Samuele Cornell, Eduardo Fonseca, and Daniel P. W. Ellis. Description and analysis of novelties introduced in DCASE task 4 2022 on the baseline system. In *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2022 Workshop (DCASE2022)*, 2022.

[41] Samuele Cornell, Janek Ebbers, Constance Douwes, Irene Martín-Morató, Manu Harju, Annamaria Mesaros, and Romain Serizel. DCASE 2024 task 4: Sound event detection with heterogeneous data and missing labels. In *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2024 Workshop (DCASE2024)*, 2024.

[42] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A simple data augmentation method for automatic speech recognition. In *Proc. Interspeech 2019*, pages 2613–2617, 2019.

[43] Hyeonuk Nam, Seong-Hu Kim, Byeong-Yun Ko, and Yong-Hwa Park. Frequency dynamic convolution: Frequency-adaptive pattern recognition for sound event detection, 2022.

[44] Yinpeng Chen, Xiyang Dai, Mengchen Liu, Dongdong Chen, Lu Yuan, and Zicheng Liu. Dynamic convolution: Attention over convolution kernels. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11030–11039, 2020.

[45] Seong-Hu Kim, Hyeonuk Nam, and Yong-Hwa Park. Temporal dynamic convolutional neural network for text-independent speaker verification and phonemetic analysis. In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7517–7521. IEEE, 2022.

[46] Sanyuan Chen, Yu Wu, Chengyi Wang, Shujie Liu, Daniel Tompkins, Zhuo Chen, Wanxiang Che, Xiangzhan Yu, and Furu Wei. BEATs: Audio pre-training with acoustic tokenizers. In *Proc. International Conference on Machine Learning (ICML)*, 2023.

[47] Xian Li, Nian Shao, and Xiaofei Li. Self-supervised audio teacher-student transformer for both clip-level and frame-level tasks. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32:1336–1351, 2024.

[48] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning, 2020.

[49] Pablo Zinemanas, Pablo Cancela, and Martín Rocamora. Mavd-traffic dataset, July 2019.

[50] Jort F. Gemmeke, Daniel P. W. Ellis, Dylan Freedman, Aren Jansen, Wade Lawrence, R. Channing Moore, Manoj Plakal, and Marvin Ritter. Audio set: An ontology and human-labeled dataset for audio events, 2017.

[51] Romain Serizel, Nicolas Turpault, Hamid Eghbal-Zadeh, and Ankit Parag Shah. Large-scale weakly labeled semi-supervised sound event detection in domestic environments. In *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2018 Workshop (DCASE2018)*, pages 19–23, November 2018.

[52] Sarthak Yadav and Mary Ellen Foster. Gise-51: A scalable isolated sound events dataset, 2021.

[53] Hossein Parineh. MELAUDIS: The First Acoustic ITS Dataset in Urban Environment. 9 2024.

[54] L. Fredianelli, F. Artuso, G. Pompei, et al. Environmental noise dataset for sound event classification and detection. *Scientific Data*, 12:1712, 2025.

[55] Karol J. Piczak. ESC: Dataset for Environmental Sound Classification. In *Proceedings of the 23rd Annual ACM Conference on Multimedia*, pages 1015–1018. ACM Press.

[56] Kenneth Ooi, Karn N. Watcharasupat, Santi Peksi, Zhen-Ting Ong, Darren Chua, Hui-Wen Leow, Li-Long Kwok, Xin-Lei Ng, Zhen-Ann Loh, and Woon-Seng Gan. SINGA:PURA (SINGApore: Polyphonic URban Audio), 2021.

[57] J. Salamon, D. MacConnell, M. Cartwright, P. Li, and J. P. Bello. Scaper: A library for soundscape synthesis and augmentation. In *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, pages 344–348, 2017.

[58] Ivo Trowitzsch, Jalil Taghia, Youssef Kashef, and Klaus Obermayer. Nigens general sound events database, February 2019.

[59] Yu Wang, Nicholas J. Bryan, Justin Salamon, Mark Cartwright, and Juan Pablo Bello. Fsd-mix-clips, October 2021.

[60] Eduardo Fonseca, Xavier Favory, Jordi Pons, Frederic Font, and Xavier Serra. Fsd50k, October 2020.

[61] Annamaria Mesaros, Toni Heittola, and Tuomas Virtanen. Tut sound events 2017, development dataset, March 2017.

[62] Kazuki Shimada, Yuichiro Koyama, Naoya Takahashi, Shusuke Takahashi, and Yuki Mitsufuji. Accdoa: Activity-coupled cartesian direction of arrival representation for sound event localization and detection, 2021.

[63] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

[64] Nian Shao, Xian Li, and Xiaofei Li. Fine-tune the pretrained ATST model for sound event detection. In *ICASSP 2024 – 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 911–915, 2024.